

OxideMerge User Guide

Compare, review and merge files and folders

Version 0.9.9 · November 2026

Table of Contents

About this guide . . . **1**

Who this guide is for . . . 1

How this guide is organized . . . 1

Related documents . . . 1

Getting support . . . 2

Revision history . . . 2

1. Introduction . . . **3**

1.1. What OxideMerge does . . . 3

1.2. How to use this guide . . . 3

1.3. Key terms . . . 4

2. Installation . . . **5**

2.1. Windows . . . 5

2.2. macOS . . . 5

2.3. Linux . . . 5

2.4. Building from source . . . 6

3. Getting started . . . **7**

3.1. The start page . . . 7

3.2. Your first comparison . . . 8

3.3. The main window . . . 8

3.4. Opening a comparison . . . 9

3.5. Comparing over SFTP or FTP . . . 11

3.6. Comparison types . . . 19

3.7. Choosing a workflow . . . 20

3.8. Getting help . . . 20

4. Comparing text files . . . **21**

4.1. The text comparison window . . . 21

4.2. Moving between differences . . . 22

4.3. Copying differences . . . 23

4.4. Saving files . . . 24

4.5. Editing text . . . 24

4.6. Find and replace . . . 28

4.7. Bookmarks . . . 28

4.8. Synchronization points . . . 28

4.9. View options . . . 29

4.10. CSV and TSV files . . . 30
4.11. Comparing with a revision . . . 33
5. Merging files and resolving conflicts . . . 35
5.1. Three-way merge . . . 35
5.2. Resolving conflict files . . . 36
6. Comparing folders . . . 38
6.1. The folder list . . . 38
6.2. Display options . . . 39
6.3. The toolbar . . . 41
6.4. Comparison methods . . . 41
6.5. Selecting and opening items . . . 42
6.6. Copying and deleting items . . . 42
6.7. Reviewing all differences . . . 43
6.8. Merging folders automatically . . . 45
6.9. Synchronizing two folders . . . 46
6.10. Documents, spreadsheets and programs . . . 50
6.11. Archives . . . 53
7. Comparing binary files . . . 55
7.1. Moved blocks . . . 55
7.2. Editing bytes . . . 58
8. Comparing images . . . 59
9. Comparing web pages . . . 61
9.1. Comparison modes . . . 61
9.2. The Formatted mode . . . 61
9.3. How pages are loaded . . . 63
9.4. Resource tree limits . . . 63
9.5. Comparing web pages in the console . . . 64
10. Controlling which differences are shown . . . 65
10.1. Comparison options . . . 65
10.2. Compare rules . . . 66
10.3. Creating a rule from a text comparison . . . 71
10.4. Filters . . . 72
10.5. Unpacker plugins . . . 72
11. Saving, sharing and reporting . . . 74
11.1. Projects . . . 75
11.2. Workspaces . . . 75
11.3. Recovering unsaved work . . . 76

11.4. Saved comparisons . . . 77
11.5. Folder reviews . . . 77
11.6. HTML reports . . . 78
11.7. Other reports . . . 81
11.8. Slideshows . . . 82
11.9. PDF reports and printing . . . 83
11.10. Patches . . . 85
12. Analysis tools . . . 86
12.1. Merge verification . . . 86
12.2. Semantic comparison . . . 88
12.3. Review workspace . . . 95
12.4. Release summary / Compare baselines . . . 96
12.5. Potential impact . . . 98
12.6. Limits . . . 100
12.7. Running analyses from the command line . . . 100
13. Command line . . . 102
13.1. Opening comparisons . . . 102
13.2. Options . . . 102
13.3. Scripting and exit codes . . . 107
13.4. Jobs . . . 108
14. Integrating with other tools . . . 112
14.1. Git . . . 112
14.2. Subversion, Mercurial and Perforce . . . 113
14.3. File manager . . . 120
15. Options and configuration . . . 122
15.1. Application options . . . 122
15.2. Configuration files . . . 123
15.3. Portable and separate configurations . . . 124
15.4. Adding a language . . . 124
16. Keyboard shortcuts . . . 125
17. Troubleshooting . . . 128
17.1. Installation and startup . . . 128
17.2. Editing and saving . . . 129
17.3. Comparison results . . . 130
17.4. Web pages . . . 130
17.5. Performance . . . 130
18. Known limitations . . . 132

About this guide

OxideMerge is a fast, safe tool for comparing and merging files and folders on Windows, macOS and Linux. Review changes with confidence, combine edits from several versions and resolve merge conflicts in one clear, side-by-side view. Beyond text, OxideMerge compares binary files, images and web pages, and it fits into your version-control and automated workflows. OxideMerge is built in Rust.

This guide is also available in 繁體中文.

This guide shows you how to install OxideMerge, compare and merge files and folders, and connect OxideMerge to your version-control system and automated workflows. It describes OxideMerge 0.9.9 on Windows, macOS and Linux.

Who this guide is for

This guide is for anyone who needs to see exactly what changed before accepting it: software developers, code reviewers, release and build engineers, system administrators, and authors of documents and configuration files. You do not need programming experience to compare and merge files. The chapters on the command line and on integration with other tools assume that you are familiar with a terminal and with your version-control system.

How this guide is organized

- Introduction describes what OxideMerge does, the conventions of this guide and the terms that it uses.
- The installation and getting-started chapters take you from download to your first comparison.
- The chapters from Comparing text files to Comparing web pages explain each kind of comparison.
- Controlling which differences are shown, Saving, sharing and reporting and Analysis tools explain how to focus a comparison, share the results and analyze changes.
- Command line, Integrating with other tools, Options and configuration and Keyboard shortcuts are reference material.
- Troubleshooting and Known limitations help you when something does not work as expected.

Related documents

- README: an overview of the features, with download and build instructions.

- CHANGELOG: the changes in each release.
- `oxidemerge --help`: a summary of the command-line options.

Getting support

To report a problem or suggest an improvement, open an issue at https://github.com/kuopinghsu/OxideMerge/issues. To help us respond quickly, include:

- The version and build identifier, from **Help** → **About OxideMerge** or `oxidemerge --version`.
- Your operating system and its version.
- The steps that reproduce the problem and, if you can share them, the files that you compared.

Before you report a problem, see Troubleshooting and Known limitations.

Revision history

Version	Date	Changes
0.9.8	Unreleased	Text rewritten for clarity and consistency. Added this preface, the Perforce setup instructions, an example of compare rules, SFTP and FTP comparisons, document and spreadsheet comparison, folder synchronization and job files. New screenshots throughout.
0.9.7	2026-11-01	Added the inline view of text comparisons.
0.9.6	2026-10-30	Added folder reviews, merge verification, semantic compare, the review workspace, compare baselines and impact analysis.
0.9.5	2026-10-29	Reorganized the guide around tasks. Added the first comparison walkthrough, workflows and synchronization points.
0.9.4	2026-10-27	Added comparison types.
0.9.3	2026-10-26	First edition.

Chapter 1. Introduction

OxideMerge shows you what changed between versions of your files and folders, and helps you combine those changes into the result you want. Use it to review changes to source code and configuration files, resolve version-control conflicts, compare two releases, or run comparisons from scripts and as the diff and merge tool for Git. Beyond text, it also compares folders, archives, binary files, images and web pages. This chapter summarizes what OxideMerge can do, explains how this guide is organized and defines the terms that the rest of the guide uses.

1.1. What OxideMerge does

- Compare two or three folders, including archives, and copy, delete or merge their items.
- Compare and edit text files, copy changes between versions and resolve version-control conflicts.
- Compare the bytes of binary files, the pixels of images and the content of web pages.
- Hide differences that do not matter with comparison options and reusable compare rules.
- Export reports and patches, and save projects and comparisons to review later.
- Run comparisons from scripts, and use OxideMerge as the diff and merge tool for Git.

The interface is available in English, Traditional Chinese, Simplified Chinese, Japanese, Korean, German, French, Spanish, Russian, Brazilian Portuguese and Italian.

1.2. How to use this guide

Start with Your first comparison for a short walkthrough. Then read the chapter for the kind of comparison you need. The chapters at the end are reference material: the command line, options, keyboard shortcuts and troubleshooting.

This guide uses these conventions:

- **Bold text** is a name that you see in the program, such as a button or a menu command.
- A menu path such as **File** → **Open...** means: open the **File** menu, then choose **Open...**.
- Command-line examples assume that `oxidemerge` is in your `PATH`. If it is not, use the full path of the program.
- Replace example paths, such as `old-folder`, with your own paths.



Keyboard shortcuts on macOS. This guide uses the key names of Windows and Linux. On macOS, press Command (`⌘`) instead of `Ctrl`, and Option (`⌥`) instead of `Alt`. The one exception is `Ctrl` + `Tab`, which uses the Control key on macOS too.

Menus and tooltips always show the shortcuts for your system.

1.3. Key terms

The following terms are used throughout this guide.

Side

One version in a comparison: the left, middle or right version.

Pane

The area of the window that shows one side.

Difference

A place where the versions are not the same. In a text comparison, one difference can include several lines.

Base (common ancestor)

The original version that two edited versions came from. In a three-way comparison, the base is in the middle.

Conflict

A place where both edited versions changed the same text in different ways. You must choose or edit the result yourself.

Merge

To copy or combine changes into a destination version.

Filter

A rule that selects which files and folders a folder comparison includes.

Compare rule

A rule that changes which text differences count. It does not change the files.



A merge changes its destination. Before you copy or save, check the source, the destination and the output path. Make a side read-only if you only need to look at it.

Chapter 2. Installation

OxideMerge runs on Windows, macOS and Linux. This chapter tells you which package to download for your system, what to do the first time you start the program, and how to build OxideMerge from source.

Download the package for your system and processor from GitHub Releases. Each release also includes this guide as a PDF file, in English and in Traditional Chinese. The release page shows the SHA-256 digest of each download, so you can verify the file.

System	Package
Windows 10 or 11, x64 or Arm64	`oxidemerge-<version>-windows-x64.zip` or `oxidemerge-<version>-windows-arm64.zip`
macOS 11 or later	`oxidemerge-<version>-macos-universal.dmg`, for Apple silicon and Intel
Linux x64 or Arm64	An AppImage (for Ubuntu 20.04, RHEL 8, Debian 10 and later) or a TAR.GZ archive. Choose `linux-x64` for Intel and AMD processors, or `linux-arm64` for 64-bit Arm.

2.1. Windows

OxideMerge for Windows needs no installer. Extract the ZIP file and run `oxidemerge.exe`.

2.2. macOS

To install OxideMerge on macOS:

1. Open the DMG file and drag **OxideMerge** to **Applications**.
2. Start OxideMerge. The app is signed ad hoc and is not notarized by Apple, so macOS can block the first start. If this happens, choose **Open Anyway** in **System Settings → Privacy & Security**. On macOS 14 and earlier, you can also Control-click the app and choose **Open**.

To start OxideMerge from Terminal, use the full path of the program. To use the short command `oxidemerge` instead, create a link in a folder that is in your `PATH`. For example:

```
sudo ln -s /Applications/OxideMerge.app/Contents/MacOS/oxidemerge /usr/local/bin/oxidemerge
```

2.3. Linux

Before you start OxideMerge on Linux, check these requirements:

- Use a desktop session with X11 or Wayland.
- Make the AppImage executable before you start it: `chmod +x oxidemerge-*.AppImage`.
- If your system does not have FUSE, start the AppImage with `--appimage-extract-and-run`.
- File dialogs need `xdg-desktop-portal`, which most desktop environments include.
- To display Chinese, Japanese and Korean text, install a CJK font, such as Noto Sans CJK.

2.4. Building from source

To build OxideMerge yourself:

1. Install a Rust toolchain from https://rustup.rs/.
2. In the repository folder, run `cargo build --release --locked`.

The program is `target/release/oxidemerge` (`target/release/oxidemerge.exe` on Windows). For the requirements and checks, see the development guide.

Chapter 3. Getting started

This chapter takes you from the start page to your first comparison. It then introduces the main window, the ways to open a comparison, including files and folders on a server, and the types of comparison that OxideMerge offers. It ends with suggested workflows for common tasks and with the places to find help in the program.

3.1. The start page

When you start OxideMerge without a comparison, it shows the start page. From here you can start a new comparison, resolve a conflict file, compare two releases or return to a recent comparison.

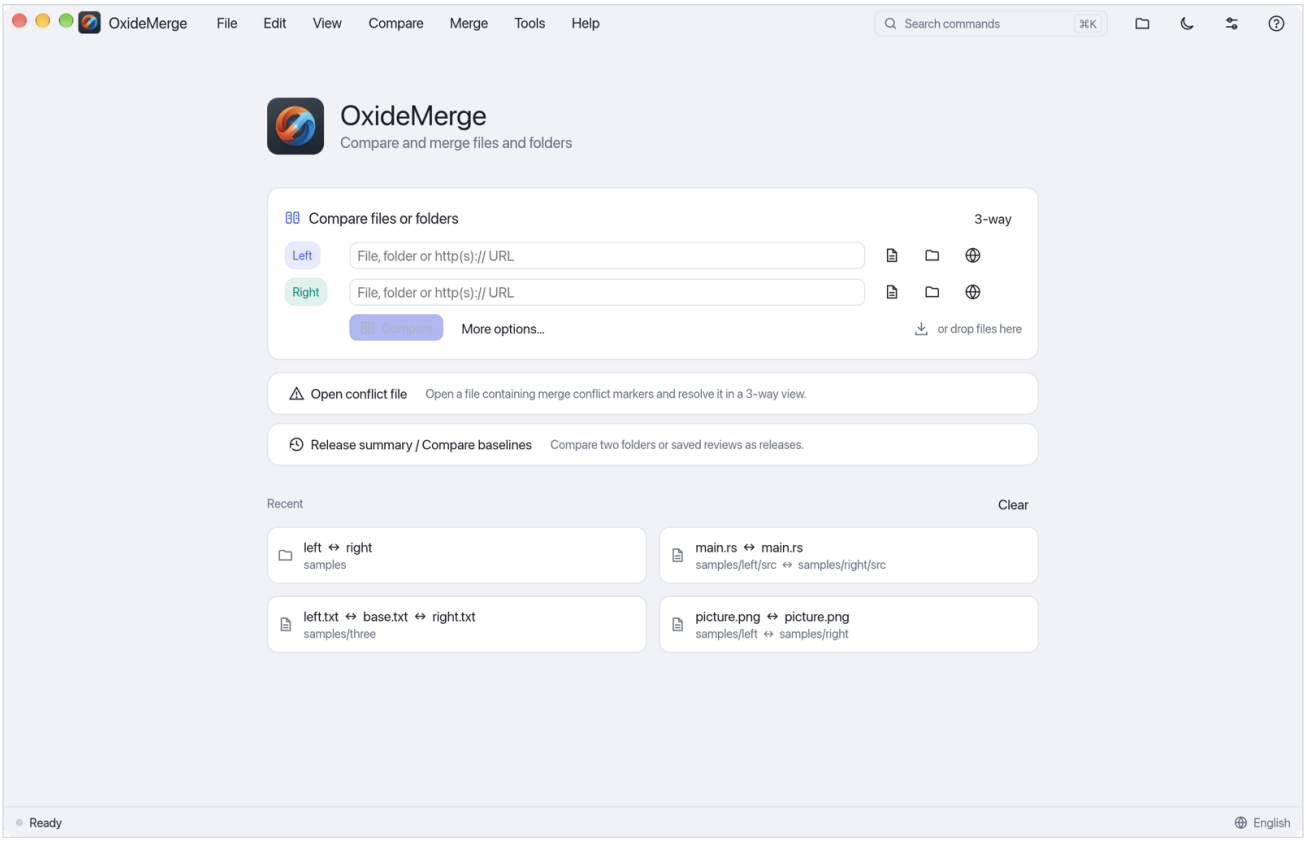


Figure 1. The start page

The start page has these parts:

Compare files or folders

Enter a **Left** path and a **Right** path. You can type or paste a path, or click the file and folder buttons next to the field. Click **3-way** to add a **Middle** path. Click **More options...** to open the **Select files or folders** dialog with these paths.

Open conflict file

Resolve a file that contains conflict markers. See Resolving conflict files.

Release summary / Compare baselines

Compare two folders or saved reviews as two releases. See Release summary / Compare baselines.

Recent

Your last comparisons, shown as cards.

- Click a card to run that comparison again.
- Right-click a card to change its paths (**Change paths…**) or remove it (**Remove from Recent**).
- Click **Clear**, at the right of the **Recent** heading, to empty the list. **File** → **Open recent** → **Clear recent comparisons** does the same. A message then offers **Undo** for a few seconds.

When the recent list is empty, for example on the first start, the start page shows **Open a sample**. This button opens two small sample folders, so that you can see how comparisons work. The sample folders are temporary: OxideMerge removes them when you close the tab, and does not add the sample to the recent list.

3.2. Your first comparison

This short walkthrough compares two versions of a file and copies a difference from one to the other.

To compare two files:

1. On the start page, enter an older file as **Left** and a newer file as **Right**.
2. Click **Compare**. To protect a side from changes, click **More options…** instead and turn on **Read-only** for that side.
3. Press `Alt` + `↓` to go to the next difference.
4. To copy a difference, first check which pane is the destination. Then click the arrow that points to that pane.
5. Check the result. To save it as a new file, choose **File** → **Save as**. To update the file in the pane, choose **Save**.

To review files without changing them, make both sides read-only. To share your review without sending the files, export an HTML report (see Saving, sharing and reporting).

3.3. The main window

Every comparison you open appears in the main window, which has these parts:

Menu bar

File, **Edit**, **View**, **Compare**, **Merge**, **Tools** and **Help**. Commands that do not apply to the current tab are unavailable. A menu that is too tall for the window scrolls, and so does a context menu.

To scroll it, rest the pointer on the arrow at its top or bottom, or use the mouse wheel or the arrow keys.

Title bar

At the right end: **Search commands**, which opens the command palette, **Open**, a switch for the light and dark themes, **Options** and **Help & Shortcuts**. In a narrow window, the search field becomes an icon, and buttons that do not fit are hidden. The menus have the same commands.

Tabs

Each comparison opens in its own tab. If both sides have the same file name, the tab shows the name once, followed by the folders that are different. A dot in place of the close button means that the tab has unsaved changes. Move the pointer over the dot to close the tab.

Status bar

Shows whether the sides are different, and the current line or offset. The interface language is at the right end. The numbers of differences and items are shown in the toolbar and in the folder chips.

To work with tabs:

- `Ctrl` + `Tab` and `Ctrl` + `Shift` + `Tab` go to the next and the previous tab.
- `Ctrl` + `W` or `Esc` closes the current tab. If a dialog, search bar, preview or editor is open, `Esc` closes it first. If the tab has unsaved changes, OxideMerge asks before it closes the tab.
- Right-click a tab to close it, the other tabs, the tabs to its right, or all tabs.

To change the interface language, use **Options** → **General**, or start OxideMerge with `--lang`.

To see the version, click the app logo or the name next to it, or choose **Help** → **About OxideMerge**. The dialog also shows a build identifier:

- A development build shows a short Git hash. `-dirty` is added when tracked files or the Git index had uncommitted changes.
- A clean optimized build of a release tag shows its build date (UTC).

`oxidemerge --version` prints the same identifier.

3.4. Opening a comparison

You can start a comparison in several ways, so you can use the one that fits your work:

- On the start page, enter the paths under **Compare files or folders** and click **Compare**.
- Choose **File** → **Open...** (`Ctrl` + `O`), or click **More options...** on the start page. This opens the **Select files or folders** dialog.
- Drop two or three files or folders on the window. If you drop one item, the dialog opens with that item filled in.

- Choose a recent comparison on the start page, in **File** → **Open recent** or in the command palette.
- Start OxideMerge from the command line (see Command line) or from your file manager (see File manager).
- Choose **File** → **Compare over SFTP/FTP…**, or click the globe button beside a path on the start page or in the **Select files or folders** dialog, to fetch a file or folder from a server over SFTP, FTPS or FTP first. See Comparing over SFTP or FTP.

The **Select files or folders** dialog collects the paths and settings of a new comparison in one place.

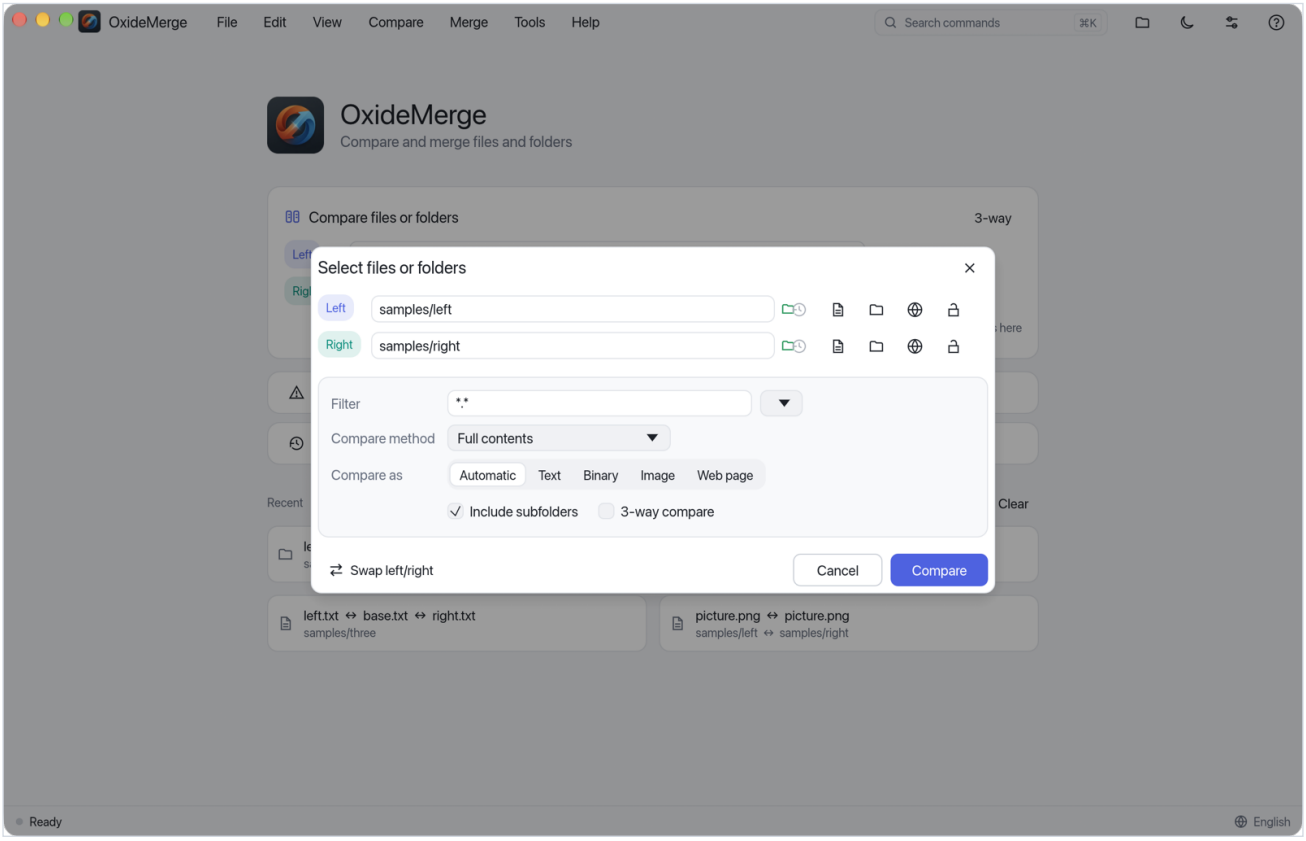


Figure 2. The Select files or folders dialog

To use the **Select files or folders** dialog:

1. Enter or browse for the **Left** and **Right** paths. A path can be a file, a folder, or an address that starts with `http://` or `https://`. To compare three versions, turn on **3-way compare** and enter a **Middle** path.
2. Turn on **Read-only** next to a side to protect it from changes. **Swap left/right** exchanges the two sides.
3. For folders, choose **Include subfolders**, a **Filter** and a **Compare method**.
4. In **Compare as**, choose the type of comparison: text, binary, image or web page. **Automatic** chooses the type for you (see Comparison types).
5. Click **Compare**.



To compare text that is not in a file, choose **File → New text compare** (`Ctrl` + `N`). This opens two empty sides. Paste text with `Ctrl` + `V`, or double-click a line to type.



To compare the clipboard's text with something you are about to paste, choose **File → Compare clipboard**. This opens a text comparison with the clipboard's current text on the left and a blank, writable right side; paste the second text with `Ctrl` + `V`. **Save** works as in any other unsaved comparison, because there is no file to write back to.

3.5. Comparing over SFTP or FTP

When the files you need to compare are on a server, OxideMerge can fetch them for you and write your changes back. **File → Compare over SFTP/FTP...**, and the globe button beside a path on the start page or in the **Select files or folders** dialog, connect to a server to fetch a file or folder to compare, instead of browsing the local disk.

To fetch a file or folder from a server:

1. Choose the **Protocol**. Changing the protocol changes the **Port** to that protocol's usual one, unless you typed another. The protocols are:
   - **SFTP (over SSH)**.
   - **FTPS (explicit TLS)**, which connects to the FTP port (21) and switches to TLS before signing in.
   - **FTPS (implicit TLS, port 990)**, for servers that expect TLS from the start.
   - **FTP (not encrypted)**.
2. Enter the server's **Host**, **Port** and **User**. For SFTP, the host can be a `Host` alias of your `~/.ssh/config`, and you can leave the user empty, as with `ssh` (see SSH configuration, jump hosts and sign-in questions).
3. For SFTP, choose how to sign in: **Password**, **Private key** (which can have a **Passphrase**), or **SSH agent**. **SSH agent** uses the keys that `ssh-agent` holds (Pageant on Windows) and the keys that `~/.ssh/config` names, so you type nothing here. FTP and FTPS sign in with a password.
4. If needed, open **Advanced** for a proxy, FTP's active mode or an FTPS client certificate (see Proxies, active mode and client certificates).
5. Enter the **Remote path** of the file or folder to fetch, or click the folder button beside it to choose it from a list of the server's folders and files.
6. Click **Connect**.

The folder button connects and opens a chooser in place of the dialog, listing the folder of the **Remote path**, or your home folder on the server if the **Remote path** is empty. The chooser keeps the same size however many entries a folder has: the list scrolls inside it, and a name too long for

it is cut short. Folders come first, then files with their sizes; hidden entries, whose names start with a dot, come last in each group. Click a folder to open it, or click a file to choose it. The path at the bottom of the chooser follows what you click: the folder you are in, or the file you chose. **OK** returns to the dialog with that path as the **Remote path**, ready for **Connect**; **Cancel** or Esc returns with the **Remote path** as it was. Double-click a file to fetch it immediately. The bar above the list names the server and each folder on the way to the one shown; click any of them to open it. The up-arrow button opens the folder above, and the refresh button lists the folder again. With the keyboard, the arrow keys, Page Up, Page Down, Home and End move through the list, Enter opens the folder you are on or accepts the file you chose, and Backspace opens the folder above. A path that does not exist lists the nearest folder above it. If the server refuses a folder or the connection drops, the list says why; refresh or **Retry** asks again. The chooser stays connected while it is shown, so moving between folders does not sign in again each time. A folder shows its first 10,000 entries.

The fetched path then fills in the **Left**, **Middle** or **Right** path, the same as a local path. To compare two remote locations with each other, connect again for the other side. To compare a remote path with a local one, leave the other side local. OxideMerge remembers the last ten servers you connected to, in its settings file, so they are there the next time you start it: the protocol, host, port, user, how you signed in and the key file's path. The dialog starts with the last one filled in, so that you can fetch a second path from the same server, and **Recent** at the top of the dialog lists the others; choose one to fill in its settings. The button beside **Recent** removes the server shown from the list. A password or a key's passphrase is never saved or filled in this way; type it each time. While waiting, the window says what it is doing and for how long: connecting to the server, then fetching, with the bytes and, for a folder, the number of entries that have arrived. The folder chooser shows the same while it lists a folder, with the entries received so far.

A fetched **file** is writable. When you edit it, **Save** uploads it back to the server over a fresh connection in the background. If the server's path is a link, the file it points to is updated. If the server's file changed or disappeared after you fetched it, the upload is refused before writing.

Over SFTP, the upload goes to a hidden temporary file beside the original, named `.NAME.oxidemerge-new-⋯` and ending in the original's extension (`.config.oxidemerge-new-⋯.php` for `config.php`), so a web server never shows a script's source as text. Only you can read that copy until it is checked. OxideMerge then reads that copy back and compares it with the file here. Only a complete, matching copy is given the original's owner, group and permissions (set-user-ID and set-group-ID bits included) and replaces the original. If someone else puts another file or a link in the copy's place meanwhile, the save is refused. Just before the swap, OxideMerge checks again that nobody else changed the file on the server. So a dropped connection, a full disk or **Cancel** leaves the server's file as it was, and the message says so.

Most SFTP servers, OpenSSH's included, cannot rename one file over another in one step. OxideMerge then renames the original aside first (`.NAME.oxidemerge-old-⋯`), renames the new copy into its place, and deletes the original. For that moment the file is missing, but it is never partly written. If the connection drops between the two renames, the message names the file that holds the other version, so you can check the server and rename it back if needed. A temporary file that could not be removed is named the same way, and you can delete it. If only the original renamed aside cannot be deleted once the new copy is in its place, the file is saved all the same,

and a warning names the copy to delete. A connection that drops just after the new copy took the original's place does not fail the save either.

A copy that replaces the original is a new file. It keeps the original's owner, group and permissions, but not its access-control lists (ACLs). It is not one of the original's hard links either. SFTP cannot show these, so OxideMerge cannot tell when a file has them. For a server where that matters, turn on **File → Save remote files in place**: **Save** then writes over the original in place, which keeps them.

OxideMerge also writes in place, without asking, in four cases:

- The file belongs to someone else.
- No new file can be created in its folder.
- The server does not show who owns the file.
- The server does not rename files.

Over FTP, **Save** always writes in place, because FTP cannot set a new file's owner or permissions.

An upload in place can be cancelled until it starts writing. After that, a dropped connection can leave the file partly written on the server. The message says so, and choosing **Save** again finishes writing it.

Your edits stay in the tab, and you can compare them with the server's version. Until an upload succeeds, the tab counts as unsaved, so closing it or quitting asks first, and **Save** tries again. When you close a tab with **Save**, the tab waits for its upload to finish before it closes.

Comparing SFTP folders from listings

By default, **Connect** fetches a folder whole and it opens in the usual folder comparison, which shows every difference at once (see below for saving back to the server). For folders too large to fetch, select **List folders without downloading files** in the SFTP connection dialog; it is off by default. Use it to compare two folders, with either one or both on SFTP servers. The comparison shows rows as the listings arrive. **Metadata match** means that size and modification time match within the configured tolerance. Use **Verify Selected** or **Verify All** to check the contents of paired files. **Binary contents** compares the bytes, and **Full contents** applies the comparison options for the file type.

The list is a table with a column for the name, the state and the size on each side. Each state has its own color and mark: red for contents that differ, amber for metadata that differs, green for contents read and found the same, and the side's color for a file only one side has. The row above the list counts the files in each state, those that differ first. While something is running, a strip below the toolbar shows how far it is: the number of entries listed so far, the files verified out of those to verify, or the bytes fetched of a file being opened. Until the server is reached, the strip says that it is connecting. The files a verification has yet to read show a spinner in their state. **Stop** in the strip ends it.

Verifying does not open a comparison: it marks each file's state in the list. When it is done, a line

over the list says how many files differ and how many match, and after **Verify All** the first file that differs is selected and scrolled into view; double-click it to compare. What a verification fetched is kept until you refresh. **Verify All** reads only the files not verified since the last refresh, so clicking it again fetches nothing; **Verify Selected** reads the selected file again. Opening a verified pair uses the copies the verification fetched, without another download. Files larger than 32 MB compared as **Binary contents** are compared as they arrive and are not kept, and at most 512 MB of copies are kept; those are fetched again when opened. **Refresh**, or saving a file to the server, discards what was kept for it and reads it again.

Double-click a paired file to open it in a separate comparison, or choose **Open File Comparison**. OxideMerge fetches the selected remote files and uploads your edits when you save them. Use **Refresh** in the folder comparison to get an updated listing. For folder-wide copy, delete or save operations, leave **List folders without downloading files** cleared when connecting to each remote folder.

A fully fetched **folder** is writable too. Copy or delete files in the folder comparison, or open a file from it and edit it. **Save** then writes what changed back to the server:

- Files that are new or whose contents changed are uploaded.
- Folders that are new are created.
- Files and folders that were removed are deleted on the server.

Files that you did not change are not sent again. Before each upload or deletion, OxideMerge checks that the server's file is still the one it fetched. A file that changed on the server in the meantime is neither overwritten nor deleted, and a new file whose name was created on the server in the meantime is not overwritten. A file that changed within a few seconds before it was fetched is also checked by its contents, because a second change in the same second would leave its size and time unchanged. The message after the save lists those files. Your copies stay in the tab, so you can compare them with the server's files and save again. The rest is written as usual. Until every change is written, the tab counts as unsaved. If the folder's local copy is gone, or part of it cannot be read, nothing is written, so a file that could not be read is never deleted on the server.

Files are uploaded before the removed files are deleted. As a result, a file that you moved to another folder is deleted from its old place only once its new copy is on the server; if that upload fails, the old copy stays. A symbolic link or special file in the fetched copy, or a name that is not valid Unicode, is never uploaded and never counts as removed. The server's file stays as it is, and the message after the save names it.

So that a very large folder cannot be fetched by mistake, a fetch is limited to 4 GiB, 50,000 files and folders, and folders nested 64 deep. Names that cannot be file names on this computer are left out. So is the second of two names that this computer cannot keep apart, such as `A.txt` and `a.txt` where letter case does not count. No `.bak` backups are made inside a fetched copy, because saving would upload them.

File → Reload from server fetches the server's copy of the current tab's remote file or folder again, over a fresh connection, and shows it in place of the copy fetched before. If this would replace changes that are not saved to the server yet, OxideMerge asks first. Edits of a remote file

are then replaced (**Undo** brings them back), and edits of a local side are kept.

On FTP and FTPS, OxideMerge detects whether a file changed on the server by its size and the modification time that the server reports. For a server that reports no modification time, it reads the file and compares its contents instead. Servers that cannot list folders in the standard machine-readable way (`MLSD`) are read from their plain listing instead, and links are followed to the file or folder they point to. Transfers use passive mode unless **Active mode** is set (see Proxies, active mode and client certificates), and always connect to the server that the sign-in went to.

Over FTPS, a download whose connection ended without TLS's closing message, which anyone in between could cause, is checked against the size that the server reports, and refused if it came up short. Many FTPS servers, such as ProFTPD and vsftpd, require each transfer to resume the TLS session of the sign-in. Each session ticket that a server gives for this is good for one transfer, so when the tickets run out, OxideMerge signs in again before the next transfer.



Plain FTP sends your password and your files unencrypted: anyone on the network between you and the server can read them, and the dialog says so. Use FTPS or SFTP whenever the server offers them.

The connection, including the password or the key's passphrase, is kept only in memory for as long as the tab stays open, and the password is overwritten in memory once it is no longer needed. It is never written to the settings file. Tabs that show a fetched file or folder are left out of saved workspaces (see Workspaces), of the session that OxideMerge reopens on its own at startup, and of the recent paths, because their temporary copies are gone after the app closes. To compare the same remote path again, connect again.

Before the password or key is sent, OxideMerge checks the server's host key. It first reads your own SSH `~/.ssh/known_hosts` the way `ssh` does: host patterns such as `.example.com` or `!host.example.com`, in any letter case, and names hashed by `ssh-keygen -H` all match, and a server on a port other than 22 is found under `[host]:port`. A server listed there with the same key is trusted, provided that the key also matches the one that OxideMerge saved for that server, if there is one. A server listed there with a different key of the same type, or whose key is marked `@revoked` there, is refused, as ssh refuses it. If the server was reinstalled, remove its old key with `ssh-keygen -R host` (or `ssh-keygen -R "[host]:port"`) and connect again. OxideMerge also checks its own known_hosts file, next to its settings file. The first time you connect to a server that is found in neither file, OxideMerge shows the key's type and fingerprint (`SHA256:…`, as ssh shows it) and asks whether to trust it. Compare the fingerprint with the one that the server's administrator gives you before you click `*Trust and connect`. A warning is shown when your `~/.ssh/known_hosts` lists only keys of other types for that server.

A server can also prove its identity with a host certificate, signed by an authority that your `~/.ssh/known_hosts` names in a `@cert-authority` line, such as `@cert-authority *.example.com ssh-ed25519 AAAA…`. OxideMerge asks for a certificate only from a server that such a line names. It trusts the certificate, without asking you first, when that authority signed it, it names the server (or its `HostKeyAlias`) among its principals, it is valid now, and it has no critical options. A certificate that fails any of these checks is refused, and so is one whose authority or key is marked `@revoked`. Certificates signed with Ed25519, ECDSA (P-256 or P-384) or RSA with SHA-2 are checked; an

authority that uses another algorithm, such as RSA with SHA-1 or ECDSA P-521, is refused. A certificate from a server for which no authority is named is checked like its plain key.

On every later connection, the key is checked against the one remembered. If it no longer matches, OxideMerge refuses the connection rather than trust a server that may not be the one you expect. This usually means that the server was rebuilt or reinstalled. If you are sure that this is what happened, remove that server's old entries from OxideMerge's `known_hosts` file (`host:port` followed by its key) and connect again. If either `known_hosts` file exists but cannot be read, or holds a line that OxideMerge cannot read (any line of its own file, or a line of `~/.ssh/known_hosts` for a key type it checks), OxideMerge connects to no server until it can.

FTPS servers are checked the way a web browser checks a website: their certificate must be valid for the host name and vouched for by a well-known certificate authority. Many FTP servers have a self-signed certificate instead. The first time you connect to one, OxideMerge shows the certificate's fingerprint and asks whether to trust it, as for an unknown SSH host key. Once trusted, the certificate is remembered in the `known_certs` file next to the settings file (`oxidemerge-known_certs` in portable use), and a later connection that shows another certificate is refused. If the server's certificate was renewed, remove the server's line (`host:port` followed by the certificate) from that file. In both cases, the certificate is checked before the password is sent, and a server that does not offer TLS is refused for FTPS rather than signed in to without it.

3.5.1. SSH configuration, jump hosts and sign-in questions

OxideMerge reaches an SFTP server the way `ssh` reaches it, as your `~/.ssh/config` (and the system's `/etc/ssh/ssh_config`) specifies. A **Host** such as `prod` can be an alias there, and its block can set the `HostName`, `User`, `Port`, `IdentityFile` (with `IdentitiesOnly`), `HostKeyAlias` (the name under which its key is looked up and certified) and `UserKnownHostsFile`. A `#` that starts a word begins a comment. `Include` is read in place, skipping files whose names start with a dot unless the pattern's file name does too. `Match` blocks on `host` (the `HostName` set so far, as in `ssh`), `originalhost`, `user`, `localuser` or `all` apply; `Match exec` is never run. As in `ssh`, the first value found wins, and a user, or a port other than 22, typed in the dialog wins over the file. In an address on the command line, any port written wins, `:22` included. Without a user, the file's `User` or your own user name is used.

A server that is reached through jump hosts (`ProxyJump bastion` or `ProxyJump user@hop1,hop2:2222`) is connected to through each in turn, up to 8, as `ssh -J` does. A jump host signs in with the SSH agent's keys and the keys that its own block names, and its key is checked like the server's before anything is sent to it. A `ProxyCommand` is run as `ssh` runs it, by the shell, with `%h`, `%p`, `%r` and `%n` filled in, and the connection goes through it. As in OpenSSH, a host or user name that contains characters the shell would act on (quotes, `$`, `;`, `|`, spaces and the like), or that starts with `-`, is refused before any command runs.

A server can ask questions while you sign in (**keyboard-interactive** sign-in), such as a one-time code, a second factor or a new password. OxideMerge shows them in a **Sign-in question** window that names the server. The answers are hidden unless the server says that they may be shown; **Cancel** declines. The password you gave answers a server's single password question automatically. For a private key that is encrypted and was given without a passphrase, the

passphrase is asked for the same way; when the key's `.pub` file is beside it, only after the server has said that it would take the key. A password that the server refused is not sent again. For servers that need more than one way of signing in, the ways are tried in turn, starting with the one you chose.

3.5.2. Proxies, active mode and client certificates

Advanced in the connection dialog holds settings that most servers do not need:

- **Proxy**: connect through a SOCKS5 proxy (`socks5://host:1080`, or `socks5://user@host:1080` with a **Proxy password**) or an HTTP proxy that allows `CONNECT` (`http://host:8080`). A password typed into the address moves to **Proxy password**, where it is hidden, once you leave the field. Both SFTP and FTP use the proxy, every FTP data connection included. For the servers that a jump host or a `ProxyCommand` from `~/.ssh/config` applies to, that is used instead of the proxy; the first jump host is reached through the proxy.
- **Active mode** (FTP): for servers that refuse passive mode, the server connects back to this computer for each transfer (`EPRT`, or `PORT` for a server without it). A data connection is accepted only from the server's address. Active mode cannot go through a proxy, and usually does not work from behind a router that translates addresses (NAT) or a firewall that blocks incoming connections.
- **Client certificate** and **Certificate key** (FTPS): a certificate to present to a server that asks for one, in PEM, with its private key in the same file or in the key file. The key must not be encrypted.

The proxy's address, active mode and the certificate's files are remembered with each recent server, but never the proxy password.

3.5.3. Server paths on the command line and in jobs

On the command line and in job files (see Jobs), a path can also be a file or folder on a server, written as an address:

```
oxidemerge sftp://ann@build.example.com/var/www/site ./site     # SFTP, port 22
oxidemerge ftpes://ann@ftp.example.com/public/index.html index.html  # explicit FTPS, port 21
oxidemerge ftps://ann@ftp.example.com:990/public ./public -r    # implicit FTPS
oxidemerge --cli ftp://ftp.example.com/pub/README README         # anonymous plain FTP
oxidemerge sftp://ann@host/~/notes.txt notes.txt                 # in ann's home folder
```

The address has the form `scheme://[user@]host[:port]/path`. An IPv6 address goes in brackets, as in `sftp://ann@[2001:db8::1]/srv`. A path that starts with `/~/` is in the user's home folder on the server, and other characters can be written as `%XX`. For SFTP, the host can be an alias of `~/.ssh/config` (see SSH configuration, jump hosts and sign-in questions), and without a user name, its `User` or your own user name is used. FTP without a user name signs in as `anonymous`, with `anonymous@` as the password, never the one in `OXIDEMERGE_REMOTE_PASSWORD`.

A password never goes in the address, where anyone who can see the command line or the job file could read it; an address that contains one is refused. Put the password in the `OXIDEMERGE_REMOTE_PASSWORD` environment variable instead. SFTP signs in with `--ssh-key FILE` when it is given (the variable then holds the key's passphrase, if it has one), otherwise with the password in the variable, and otherwise with `ssh-agent` and the keys that `~/.ssh/config` names. FTP and FTPS need the password, unless they sign in as `anonymous`. OxideMerge removes the variable when it starts, so the programs it runs, such as an external tool, do not inherit it. Questions that a server asks while signing in, and the passphrase of an encrypted key given without one, are asked on the terminal without echoing the answer (**Ctrl+C** there turns the echo back on as it stops the program); without a terminal, they are declined.

`--proxy URL` connects through a proxy, as **Proxy** does in the dialog. A proxy password is read from the `OXIDEMERGE_PROXY_PASSWORD` environment variable, which is also removed at startup, and a password in the address is refused. `--ftp-active` uses FTP's active mode. `--client-cert FILE` presents an FTPS client certificate, with its key from `--client-key FILE` when the key is not in the same file.

Servers are checked as in the program, against the same `known_hosts` and `known_certs` files. A server that is not trusted yet is refused, and the error shows its fingerprint. Compare it with the fingerprint that its administrator gives you, then run the command again with `--trust-fingerprint SHA256:⋯` to trust the server; it is then remembered, as when you click **Trust and connect**. A server whose key or certificate changed is refused.

Each server path is fetched to a temporary copy before the comparison and opened like a local path. In the window, **Save** writes changes back to the server as described above. Without a window, what the run changed in a fetched copy, as a `sync` job does in its destination, is written back to the server after the run, with the same checks; a file that the server changed in the meantime is reported as an error (exit code 2). These writes follow **File → Save remote files in place** as saved in the settings. A synchronization that stopped part way without undoing all it changed (exit code 5) is undone in the fetched copy instead, and nothing is written back. A sync plan made with `--sync-plan` names a server folder by its address, so `--sync-apply` applies it to the copy that its run fetches; as for local folders, the plan is refused if the files it plans to change are no longer as planned.



Checked on Linux with OpenSSH 10.5p1 and vsftpd 3.0.5. With a private key, SFTP refuses a server it has not been told to trust and shows that server's fingerprint, the same one `ssh-keygen -lf` prints; `--trust-fingerprint` then connects and the key is remembered. An identical file exits 0, a different file exits 1, and a folder compare names the file that differs. The window opens the fetched file as writable and shows the difference. A sync job uploads a new file. Uploading a shorter file replaces it in place, and an upload is refused when the file on the server has changed since it was fetched. Anonymous FTP used a vsftpd process running as the current user, with no chroot, so the remote path was the real directory. An identical file exits 0, a different file exits 1, and the window opens it as writable. A sync job uploads a file, and a second run updates it. `--ftp-active` compared an identical file and exited 0. An address that contains a password is

refused for both.

3.6. Comparison types

OxideMerge has five types of comparison, each designed for a different kind of content:

Type	What it shows
Text	Lines side by side, with the changed words highlighted. You can edit and merge the text. Comparison options and compare rules decide which changes count as differences. Use it for source code, configuration files, CSV files and any other readable text.
Binary	The bytes in a hexadecimal view. You can copy and save differences, but the lines are not aligned. For example, one byte inserted near the start moves all the bytes after it.
Image	The pixels: side by side, as an overlay, with a swipe divider, blinking, or as a difference image. You can copy regions that are different from one image to the other.
Folder	The items of two or three folders. You can copy, delete and merge items, and open each file in its own comparison.
Web page	Two or three web pages: as visible text, as formatted Markdown, as HTML source or as downloaded resources. Web pages are read-only. OxideMerge never saves anything to a web server.

Documents, spreadsheets and programs are compared as the text that OxideMerge reads from them. This is a text or table comparison of what was read, rather than a type of its own. Word documents, workbooks and tagged PDF files can also be shown **Formatted**, with their headings, lists and tables; see Documents, spreadsheets and programs.

When **Compare as** is **Automatic**, OxideMerge checks these conditions in this order and uses the first one that matches:

1. Any side is an `http://` or `https://` address: web page comparison.
2. Every side is an HTML file (`.html` or `.htm`): web page comparison. It uses the Formatted mode, unless you chose another mode before. Exception: a merge with an output file (`-o`) or a console patch (`--patch`) compares the files as text.
3. Every side is a folder: folder comparison.
4. Every file is one that OxideMerge reads as text — a document, a spreadsheet or an ELF program (see Documents, spreadsheets and programs) — or one that an unpacker plugin converts (see Unpacker plugins): read-only text comparison of what was read. A spreadsheet that OxideMerge reads itself opens as a table comparison. This rule comes before the archive and binary rules, so `.docx`, `.xlsx` and `.odt` files open as their text rather than as the zip archives

they are. Turning off **Read documents, spreadsheets and ELF files as text** in **Options → Plugins** leaves them to the rules below.

5. Every side is an archive: the archives are extracted and compared as folders.
6. Every file has an image extension, such as `.png`, `.jpg`, `.gif`, `.bmp`, `.tif` or `.webp`: image comparison.
7. A file has a zero byte in its first 8 KB: binary comparison. Exception: UTF-16 text with a byte order mark is still text.
8. Anything else: text comparison.

To choose the type yourself, use **Compare as** in the dialog, **Compare → Compare as** in an open tab, or `-t` on the command line.

3.7. Choosing a workflow

Use this table to find the best starting point for a common task.

Task	What to do
Review two versions	Open two files or folders. Make a side read-only if you do not need to change it.
Combine two edited versions	Open three versions in this order: left, base, right. See Three-way merge.
Resolve a conflict file	Open the file that contains conflict markers. See Resolving conflict files.
Review many changed files	Compare the folders, then open **All differences**. See Reviewing all differences.
Repeat a comparison later	Save a project (see Projects). To keep the text and your review notes as they are now, save the comparison instead (see Saved comparisons).

3.8. Getting help

Help is always available inside OxideMerge:

- **Help → Help & Shortcuts** (`F1` or `?`) lists every keyboard shortcut and mouse action, sorted by view. It also shows where the settings file is.
- **Search commands** in the title bar, or **Help → Command palette** (`Ctrl` + `K` or `Ctrl` + `Shift` + `P`), finds commands, recent comparisons and settings. Type part of a name and press `Enter`. For settings that you can turn on and off, the palette shows the current state.

Chapter 4. Comparing text files

The text comparison places two or three versions of a file side by side, highlights each difference down to the changed words and characters, and lets you copy changes between the versions, edit them in place and save the result. Use it whenever you need to see exactly what changed in source code, configuration files or other text, and to carry selected changes from one version into another.

This chapter covers plain text. For PDFs, documents, spreadsheets, Jupyter notebooks and ELF programs, which OxideMerge also reads as text, see Documents, spreadsheets and programs.

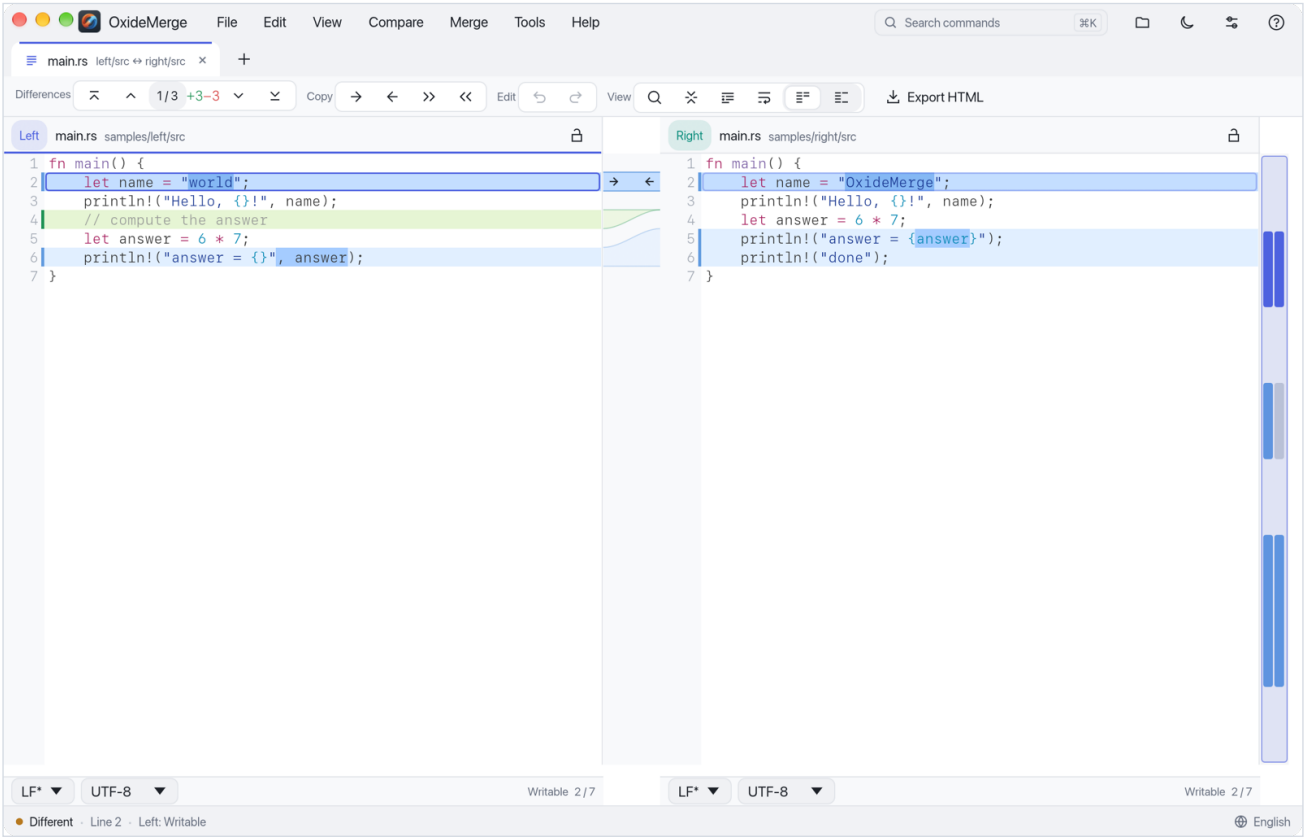


Figure 3. A two-way text comparison

4.1. The text comparison window

Each side of the comparison has a header above the text and a footer below it:

- The header shows the file name and its folder. The lock button in the header makes that side read-only.
- The footer shows the line-ending style and the encoding that OxideMerge uses when it saves the file, together with the current line and the total number of lines.

If you choose a different encoding in the footer, OxideMerge reads the file again using that encoding. After you change the encoding, check the text, especially before you edit or save the file.

Colors identify each kind of difference:

- Changed lines are blue.
- In a two-way comparison, lines that exist on one side only are green.
- In a three-way comparison, unresolved conflicts are red.
- Within a changed line, the words or characters that changed are highlighted.

If you turn on **Color-blind friendly diff colors** in **Options** → **General**, changed lines appear purple and lines on one side only appear blue.

Between the panes, a connector links the two sides of each difference, and the current difference is highlighted. The copy arrows of a difference appear when it is the current difference or when the pointer is over it.

The toolbar arranges its buttons in four groups:

- **Differences**: go to the next, previous, first or last difference.
- **Copy** (**Merge** in a three-way comparison): copy differences between the sides.
- **Edit**: undo and redo.
- **View**: find text, collapse unchanged lines and turn word wrap on or off.

When the window is wide enough, the toolbar also shows the group names. The +/− numbers next to the difference number are the numbers of added and removed lines.

The location pane on the right shows an overview of the whole file; click anywhere in it to go to that place. Binary comparisons, the Formatted mode of web page comparisons and the Formatted view of documents also have a location pane.

4.2. Moving between differences

Use these keys to move through a file:

Keys	Action
`Alt` + `↓`, `Alt` + `↑`	Go to the next or previous difference
`Alt` + `Home`, `Alt` + `End`	Go to the first or last difference
`Alt` + `PgDn`, `Alt` + `PgUp`	Go to the next or previous conflict (three-way comparison)
`Ctrl` + `G`	Go to a line number

If you opened the file from a folder comparison, navigation continues into the other files. After the last difference, `Alt` + `↓` opens the next file that has differences. Before the first difference, `Alt` + `↑` opens the previous one, at its last difference. In a three-way folder comparison, `Alt` + `PgDn` and `Alt` + `PgUp` continue in the same way to the next or previous file that still has an open conflict.

Hold the pointer over a navigation button to see what it moves through, its keyboard shortcuts, and the fact that it stops at the end rather than starting again at the top.

4.2.1. Moving through the differences of a folder

In a folder comparison, the same keys work on the file shown below the folder list and continue through the other files of the folder:

- `Alt` + `↓` goes to the next difference of the file. After its last difference, it selects the next file that has differences and goes to that file's first difference.
- `Alt` + `↑` works in the opposite direction: before the first difference, it goes to the last difference of the previous file.
- `Alt` + `Home` goes to the first difference of the first file, and `Alt` + `End` to the last difference of the last file.
- The status bar shows your position, for example *File 2 / 5 · Difference 3 / 7*.

Navigation covers the files that the folder list shows as different, in the order of the list. Binary files are included; files that could not be read are not. At the last difference of the last file, OxideMerge tells you so instead of starting again at the top.

If you press a key while a file is still loading, OxideMerge remembers it and moves once the file is shown, so no difference is skipped. If a file cannot be read, the pane says so and offers **Retry** and **Skip**. While the folders are still being compared, more files may follow, and the message says so instead of reporting the end. If the comparison was stopped, the message at the end states that it covers only the files that were compared.

A file that changed after the comparison is shown with its current content. If it no longer has differences, the next step moves on to the next file. When the file shown below the list changes on disk, it is loaded again at the same difference.

In a three-way folder comparison, `Alt` + `PgDn` and `Alt` + `PgUp` go through the open conflicts of the files that both sides changed. A file whose changes merge without conflicts is passed over once it has loaded, and a file changed on one side only is not visited. `Alt` + `↓` still stops at every changed file.

4.3. Copying differences

Copying a difference brings a change from one side into the other. OxideMerge changes the text in the destination pane; the file on disk changes only when you save it. Before you copy, check which pane is active and which way the arrow points.

- Click an arrow on the divider between the panes to copy the current difference, or the difference under the pointer.
- Press `Alt` + `→` or `Alt` + `←` to copy the current difference, or the selected lines, to the right or to the

left.

- Press `Ctrl` + `Alt` + `→` or `Ctrl` + `Alt` + `←` to copy all differences to the right or to the left.
- In a three-way comparison, use the **Merge** group of the toolbar to copy the current difference, or all differences, from the left or from the right into the middle. You can also right-click a difference to copy it from the left or the right into the middle, or from the middle to the left or the right.
- Press `Ctrl` + `Z` to undo a change, and `Ctrl` + `Y` or `Ctrl` + `Shift` + `Z` to redo it.

4.4. Saving files

Use these commands to write your changes to disk:

- `Ctrl` + `S` saves all changed panes in the current tab.
- `Ctrl` + `Shift` + `S` saves all changed panes in all tabs.
- **File** → **Save as** saves one side under another name.

OxideMerge saves each file safely, in two steps:

1. It writes the new content to a temporary file in the same folder.
2. It replaces the original file with the temporary file.

If either step fails, OxideMerge shows an error; it never writes directly over the original. For this reason, the folder must allow OxideMerge to create and replace files, even when the file itself is writable.

To keep the previous content of a file, turn on **Keep a .bak copy when saving over a file** in **Options** → **General**. OxideMerge then keeps the old content as `<name>.bak`. The next save replaces this backup, so it is not a version history.

4.5. Editing text

You can edit any pane that is not read-only. Click in the text to place the cursor there and type, as in any text editor.

- `Enter` starts a new line with the indentation of the line above. `Backspace` and `Delete` remove a character, or join a line to the adjacent one.
- `Ctrl` + `V` pastes at the cursor, any number of lines at once.
- To select text, drag over it, `Shift`+click, or move the cursor with `Shift` held down. Double-click selects a word, triple-click selects the line including its line break, and `Ctrl` + `A` selects the whole file. Dragging past the top or bottom of the pane scrolls it and continues the selection. Typing or pasting replaces the selection.
- `Ctrl` + `C` copies the selection and `Ctrl` + `X` cuts it. With nothing selected, they copy or cut the

whole line.

- The arrow keys, `Home`, `End`, `PgUp` and `PgDn` move the cursor. `Ctrl` + `Home` and `Ctrl` + `End` go to the start and end of the file. On macOS, `Cmd` + `←` and `Cmd` + `→` go to the start and end of the line. With word wrap on, the up and down arrows, `PgUp` and `PgDn` move by display line, so a long line takes several presses; `Home` and `End` (`Cmd` + `←` and `Cmd` + `→`) first go to the start or end of the display line, and to the start or end of the whole line on a second press. With `Shift` held down, each of these keys extends the same selection.
- `Tab` types a tab, or indents the lines when the selection spans more than one line or is a whole line; `Shift` + `Tab` outdents the selected lines, or the line with the cursor. `Ctrl` + `]` and `Ctrl` + `[` do the same when the text cursor is in a pane. A file that indents more lines with tabs than with spaces is indented with a tab; any other file is indented with the **Tab size** in spaces. Blank lines are not indented, and outdenting removes one tab or up to the tab size in spaces. A selection that ends at the start of a line leaves that line unchanged, and the selection moves with its text, so `Shift` + `Tab` right after `Tab` restores the same text and selection. Each indent or outdent is one undo step.
- `Ctrl` + `←` and `Ctrl` + `→` (`Option` + `←` and `Option` + `→` on macOS) move by word, and select by word with `Shift` held down. A word move goes to the end of the next word, or to the start of the previous one; from the end or start of a line, it goes to the next or previous line. Letters, digits and `_` form one word (`snake_case` and `camelCase` are each one word), a run of punctuation is a word of its own (`x.len` is three words), and each Chinese, Japanese or Korean character is a word.
- `Ctrl` + `Backspace` and `Ctrl` + `Delete` (`Option` + `Backspace` and `Option` + `Delete` on macOS) delete the word before or after the cursor, or the selection if there is one. A single `Ctrl` + `Z` brings the word back.
- On macOS, while the cursor is in the text, `Option` + `←` and `Option` + `→` move by word instead of copying the difference; press `Esc` to leave the text, or use the copy buttons. Because `Option` with a letter, a digit or a symbol types a character (`Option` + `S` types ß), `Alt` + `C`, `Alt` + `Z`, `Alt` + `P` and `Alt` + `S` wait until the cursor leaves the text; their commands remain available in the menus.
- Each press of `Esc` does one thing, the first of these that applies:
  1. Closes an open menu.
  2. Closes a dialog, or a field you are typing in.
  3. Closes the search bar.
  4. Goes back from several cursors to one.
  5. Ends the selection (the cursor stays where it was).
  6. Removes the cursor, so that the arrow keys move from row to row again.
  7. Closes the tab.
- Input methods work at the cursor, so you can type Chinese, Japanese or Korean.
- In very large files, typing within a line stays fast: OxideMerge compares the files again once you pause for a moment, or before any other key, click or command, so the colors of the line you are typing in may briefly lag behind. A new line, a paste or an undo is compared at once.

- A selection over several lines also selects their rows, so the copy buttons and `Alt` + `→` and `Alt` + `←` copy those lines to the other side.
- `Ctrl` + `]` indents the selected lines, and `Ctrl` + `[` unindents them (with the text cursor, `Tab` and `Shift` + `Tab` work too; see above).
- `Ctrl` + `D` places a copy of the selected lines, or of the cursor's line, below them and selects the copy. `Alt` + `Shift` + `↑` and `Alt` + `Shift` + `↓` move the lines up or down past the adjacent line, together with their selection. Like `Tab`, these commands act on the lines of the text selection (a selection that ends at the start of a line leaves that line out) or, without a text cursor, on the active pane's lines in the selected rows. Each is one undo step, and both are also in the **Edit** menu. Without `Shift`, `Alt` + `↓` and `Alt` + `↑` still go to the next and previous difference.
- When there is no cursor, `F2` or `Enter` places it at the end of the current line.

4.5.1. Several cursors

To type the same text in several places of one pane, give the pane several text cursors:

- `Ctrl`+click (`Cmd`+click on macOS) adds a cursor where you click, or removes the cursor that is there. `Ctrl`+double-click and `Ctrl`+triple-click add a word selection or a line selection.
- `Ctrl` + `Alt` + `↑` and `Ctrl` + `Alt` + `↓` (`Cmd` + `Option` + `↑` and `Cmd` + `Option` + `↓` on macOS) add a cursor on the line above the first cursor or below the last one, in the same column. Some Linux desktops use these keys to switch workspaces; `Ctrl`+click works everywhere.

Typing, deleting, `Enter`, the arrow keys, `Home` and `End` then act at every cursor, and cursors that meet merge into one. Each edit is one undo step for all cursors, and undo puts the cursors back where they were. Pasting as many lines as there are cursors puts one line at each cursor; any other text is inserted at every cursor. Copying gives the selections one per line, in the order of the text. `Tab`, `Shift` + `Tab`, **Duplicate lines** and **Move lines** act on the lines of all the cursors, each line once. A read-only pane refuses the edit at every cursor.

The view follows the cursor that was added last, and an input method shows its composition there. `Esc` first goes back to that single cursor, and so does a click without `Ctrl`. The column selection (`Alt`+drag) is separate: making a column selection removes the extra cursors.

The pane that has the cursor shows a colored line under its header. Where typing goes into the text, the cursor is a solid bar; it is thin and faint in a read-only pane, while a field such as the search bar has the keyboard, and while the window is in the background. Typing into a read-only pane changes nothing, and the status bar tells you which side it is and how to unlock it.

The cursor works in the side-by-side view. In the inline view, double-click a line, or press `Enter` or `F2`, to edit it side by side (see Inline view); in the table view, double-click a cell to edit it.

The bar under each pane shows whether you can edit it:

- **Writable**: you can edit the file and save it.
- **Read-only**: the pane is locked, for example with the lock button in its header, and does not

accept edits.

- **Read-only on disk**: you can edit, but the file is marked read-only, so **Save** refuses to replace it and keeps your edits. Save the file under another name, or clear the read-only mark and save again.
- **Compare only**: the pane shows text that OxideMerge read from the file rather than the file itself, such as a document, spreadsheet, PDF or program, a web page, or a formatted copy of JSON or XML. The pane cannot be unlocked, edited or saved back. To edit the text itself, save it with **File** → **Save as** to a plain file, such as a `.txt` file, which the pane then edits; the text is never saved over a file that is itself read this way, whatever the options say.
- **Modified** appears while the file has unsaved changes.

The status bar at the bottom of the window shows the same information for the active pane. It also shows **Compare only** for the Tree and Formatted views and for web pages shown as Markdown.

To select a column of text, hold down `Alt` (`Option` on macOS) and drag. The selection covers the same columns on every line it crosses. Column selection is not available with word wrap or in the table view.

- Typing replaces the selected columns on every line. A selection with no width acts as a cursor on every line, so typing inserts the same text on each line.
- `Backspace` and `Delete` remove the selected columns, or one character on each line.
- `Ctrl` + `C` copies the columns, one line per row, and `Ctrl` + `X` cuts them.
- `Ctrl` + `V` pastes. A single line is pasted into every selected line. Several lines are pasted one per line, and lines beyond the selection go into the lines below it.
- `Esc` or a click ends the column selection.

Columns are counted as the lines are displayed: a tab extends to the next tab stop. Text placed past the end of a short line is padded with spaces so that it lines up. Characters that are drawn wider than others, such as Chinese characters, still count as one column, so a column selection across them can look uneven.

Each edit, paste, cut or deletion is one undo step; typing within a line counts as one step until the cursor moves. Saving preserves each file's line endings, encoding and byte order mark.

Right-click a line to edit it, insert a line below it, delete it, copy its text, set a bookmark, or create a compare rule from it (see Creating a rule from a text comparison). Right-clicking inside selected text keeps the selection, and **Copy text** then copies only that text; right-clicking elsewhere moves the cursor there first.

Edit markers in the gutter show the lines that have changed since you opened the files.

When another program changes a file, OxideMerge detects the change when its window becomes active again:

- A file without unsaved changes reloads automatically.

- A file with unsaved changes shows a message with a reload option. Reloading replaces your edits with the file on disk.

4.6. Find and replace

- `Ctrl` + `F` opens the find bar. `F3` and `Shift` + `F3` find the next and previous match, and `Esc` closes the bar.
- `Ctrl` + `H` (`Cmd` + `Alt` + `F` on macOS) opens find and replace.
- The search options are **Match case**, **Whole word** and **Regular expression**. With a regular expression, `$1` in the replacement text stands for the first captured group.
- You can undo **Replace all** in a single step.

4.7. Bookmarks

Bookmarks let you mark lines and return to them quickly.

- `Ctrl` + `F2` sets or removes a bookmark on the current line.
- `F4` and `Shift` + `F4` go to the next and previous bookmark.
- **Edit** → **Bookmarks** → **Bookmark comment…** adds a comment to a bookmark. The comment appears when you move the pointer over the gutter, and in HTML reports.
- **Edit** → **Bookmarks** → **Clear bookmarks** removes all bookmarks.

Bookmarks are marked in the gutter and in the location pane.

4.8. Synchronization points

Occasionally the comparison pairs the wrong lines, for example when a function was rewritten and moved. A synchronization point tells OxideMerge which lines belong together.

To set a synchronization point:

1. Place the cursor on a line on one side, then choose **Merge** → **Set synchronization point** (`Alt` + `S`), or right-click the line.
2. Do the same on the matching line on the other side. In a three-way comparison, do it on the third side as well.

OxideMerge shows these lines on the same row and compares the text before them and the text after them separately.

- You can add as many points as you need. OxideMerge refuses a point that would cross another point.
- Synchronization points are marked in the gutter. They stay with their lines when you edit, undo

or reload, and they are kept in saved comparisons.

- To cancel a line that you picked, run the command again on that line.
- To remove a point, run the command again on one of its lines.
- To remove all points, choose **Merge** → **Clear synchronization points**.

4.9. View options

View → **Text compare** offers these options:

- **Location pane** and **Line detail pane**. The line detail pane shows the current line of each side, one above the other, with word wrap. Use it to find small differences in long lines.
- **Line numbers**, **Edit markers**, **View whitespace** and **Syntax highlighting**.
- **Collapse unchanged lines** (`Alt` + `C`) hides unchanged lines behind a bar. Three lines stay visible around each difference; click the bar to show the hidden lines.
- **Word wrap** (`Alt` + `Z`).

View → **Zoom** (`Ctrl` + `+`, `Ctrl` + `-` and `Ctrl` + `0`) changes the text size.

Layout is available in **View** → **Layout**, in **Options** → **General**, and as the **Compact rows** and **Aligned rows** buttons on the toolbar of text and binary comparisons: a pair of icons that show two columns of lines, packed or with a gap:

Compact rows

The default. Each side shows its own lines, without empty space.

Aligned rows

Empty space is added opposite inserted and deleted lines, so that the lines after them stay side by side.

The layout applies to all open two-way and three-way text comparisons and is saved for the next session. The same setting lays out the moved blocks of a binary comparison (see Comparing binary files). The toolbar buttons are unavailable in the inline view and the table view, which show no empty space either way. The layout does not change the files or the compare rules.

4.9.1. Inline view

In a two-way comparison, the **Inline view** button next to **Word wrap** shows the differences in a single column. The command palette offers the same command.

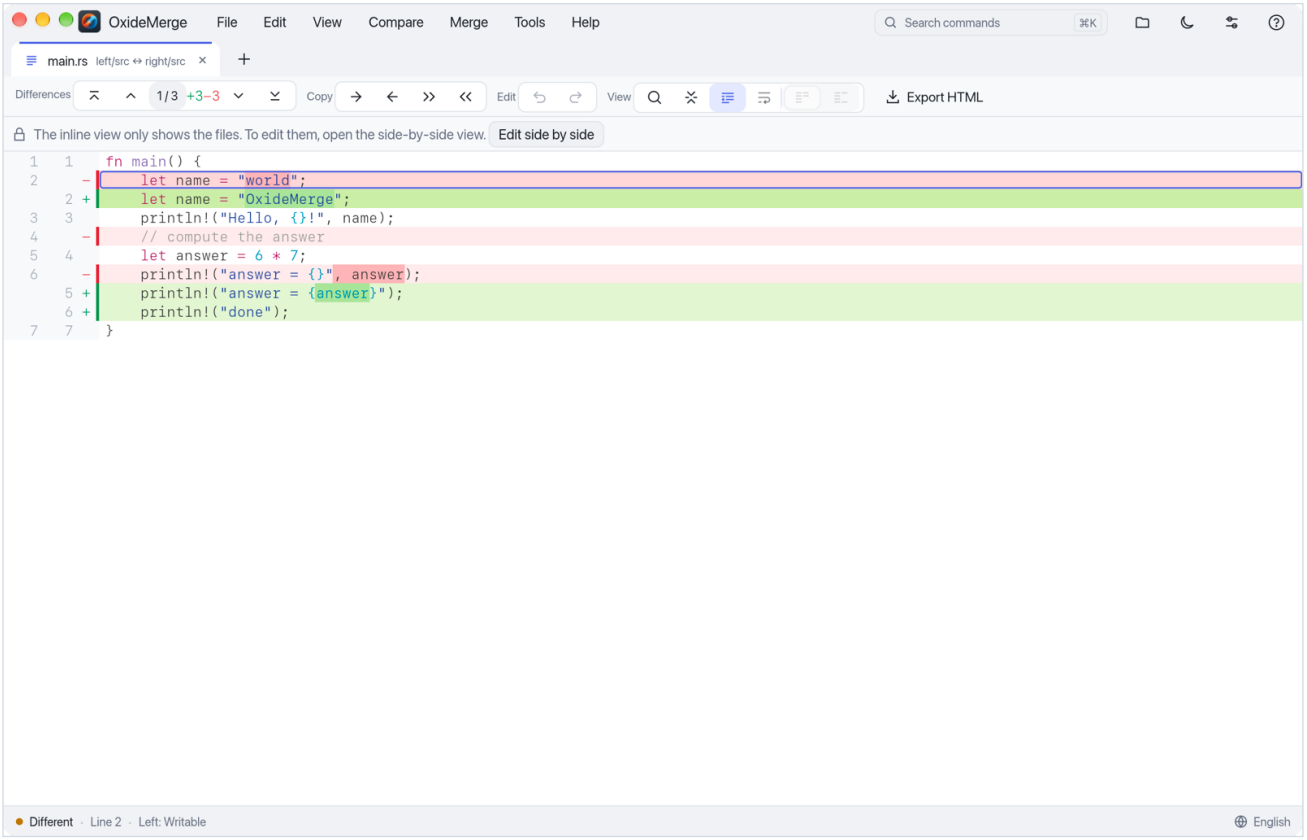


Figure 4. The inline view

- Removed lines come first, followed by the lines that replace them. Each line shows both its old and its new line number.
- When **Collapse unchanged lines** is on, unchanged lines are hidden; click a fold to show them.
- You can move between differences and copy them as usual, and Undo and Redo work. The arrow keys move from line to line, and `Ctrl` + `C` copies the current line.
- The inline view is for viewing only: a bar at its top says so, and typing, pasting, indenting and **Replace** have no effect there. To edit, click **Edit side by side** on that bar, double-click a line, or press `Enter` or `F2`; the side-by-side view opens with the text cursor at the start of the current line.
- Switching between the views keeps every edit and the selected difference, as well as the text cursor and its selection from the side-by-side view, which return when you switch back.

4.10. CSV and TSV files

CSV and TSV files, and spreadsheets, open in the **Table** view. The **Text** / **Table** switch above the panes changes between the lines of the file and the table, in the same place as the **Text** / **Formatted** switch of documents; a workbook has all three. It compares records rather than lines, so a quoted field can contain line breaks.

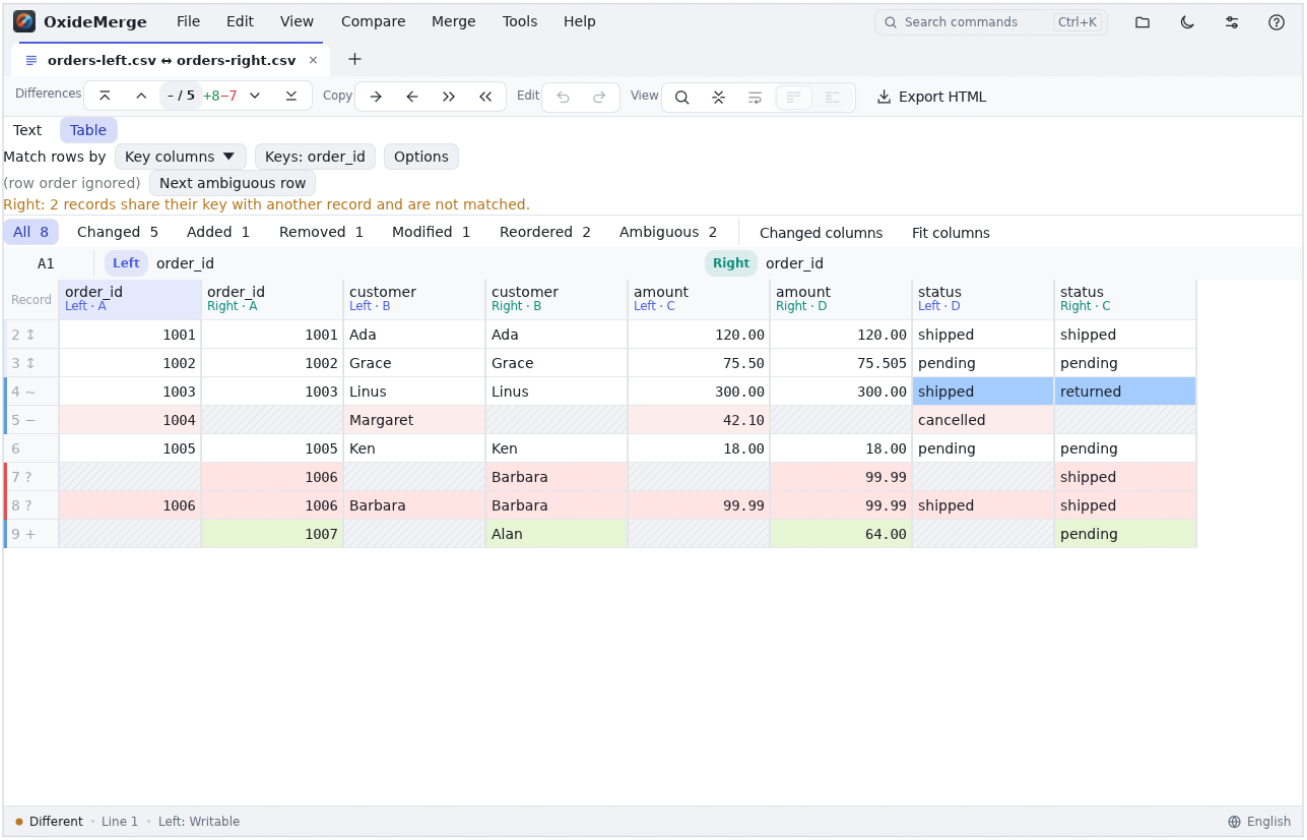


Figure 5. The table view of two CSV files

The table looks like a spreadsheet: a grid with numbered rows and columns headed by their letters, or by their names when the first row is the header.

- Columns that are compared with each other stand side by side, the left one first.
- Click a cell to select it. The bar above the table shows where it is, such as B4, and what each side has there, with the part that differs marked.
- The chips above the table list **All** rows or only the **Changed** ones; when matching by key, also the **Added**, **Removed**, **Modified**, **Reordered** or **Ambiguous** ones, each with its count. Going to a row that is not listed shows all rows again.
- **Changed columns** shows only the columns that hold a changed cell, and the key columns. To hide a column and the one it is compared with, right-click it and choose **Hide column**; **Show hidden columns** brings them back.
- **Fit columns** gives every column the width of its contents, and clicking it again brings back the widths you had; double-clicking a column border does so for one column.
- Numbers are right-aligned. A changed cell is highlighted; a record that only one side has is coloured across its row, and the cells of the side without it are hatched.
- Drag a column border to change the column width.
- When you scroll sideways, the record number and, when matching by key, the key stay at the left edge.
- To edit a cell, double-click it, or right-click it and choose **Edit cell**. You can also select the cell and press `F2` or `Enter`. A cell can contain several lines.

- To copy one field to the matching record on another side, right-click the cell and choose **Copy cell**. Both records must have that column.
- The arrow keys move between records and columns. `PgUp` and `PgDn` move through the records a page at a time, and `Ctrl` + `Home` and `Ctrl` + `End` select the first and the last record.
- **Go to line** uses the line numbers of the file.

In an Excel workbook (`.xlsx` or `.xlsm`), a vertical merge continues across the rows it covers, including gaps created by alignment. Horizontal continuations are shaded and marked in their own pane so that compared columns stay side by side. The anchor displays the value. Hover over a merged cell to see its range, such as `B2:D4`, or select it to see the ranges in the bar above the table. The table treats covered cells as empty and compares cell values. A change to the merge range alone adds no difference. A very large merge can extend beyond the grid, while its range label still gives the worksheet coordinates. Open the two sample `docs/budget.xlsx` files and select the **Merged Cells** tab to see matching, overlapping and disjoint ranges.

When you edit a cell, OxideMerge preserves the quotes and line endings outside that field, and you can undo and redo the change. Converted or read-only documents cannot be edited.

When you copy a difference in the table view, only its selected lines change. If the copy would break the CSV quoting, OxideMerge refuses it and leaves the document unchanged; in that case, copy individual cells or use the text view. If a quoted record is not valid, the table shows an error; switch to the text view to correct it.

4.10.1. Matching rows by key

By default, the table view pairs records by their position, as it does with lines. When records are sorted differently, or some have been added or removed, pair them by a key instead. When most rows differ by position but a column tells the records apart, the bar offers **Match rows by** that column; one click turns matching by key on.

In a two-way comparison, the bar above the table provides these settings; all but the first two are under **Options**:

Match rows by

Position (the default) or **Key columns**.

Keys

The columns that identify a record, such as `id`. Choose one or more.

First row is header

Use the first row for column names. The first row is then not compared; its names head the columns, and double-clicking a name edits it.

Columns by

Pair columns by **Position** or by **Header name**, so that reordered columns still match.

Ignore row order

A record that has only moved is not a difference: next and previous difference skip it, but it keeps its ↕ mark.

Numeric tolerance

Numbers this close to each other count as equal. `0.01` is an absolute difference, and `1%` is a fraction of the larger number. Leave the field empty to compare exactly.

Rows in

Left order (the default) lists the rows in the order of the left table, placing records that exist only on the right after the record that precedes them. **Right order** follows the right table. **Key order** sorts by key: numbers by value and before text, and rows without a key last.

The chips show how many records were added, removed, modified, reordered and ambiguous. The row markers are + for added, − for removed, ~ for modified, ↕ for moved and ? for ambiguous.

- Next and previous difference go through the records that differ.
- **Copy cell** copies into the paired column of the matching record. Copying a whole record replaces the cells of the matching record, or adds or removes the record.
- A key that is empty, or that more than one record uses, is *ambiguous*. Ambiguous records are never copied; **Next ambiguous row** finds them.
- If a key column is missing on one side, OxideMerge matches by position and tells you why.

Tables with more than 100,000 records on both sides together are matched in the background. The bar shows how many records are done and offers **Cancel**. After you cancel, the rows are matched by position, and **Match by key** starts the matching again.

HTML and XML reports of a table matched by key describe how the rows were matched: the keys, the header row, how columns were paired, the row order, the tolerance, the counts and any warnings. XML reports hold this information in a `<table-match>` element.

The settings are saved in projects. Projects created with older versions open with matching by position.

4.11. Comparing with a revision

File → Compare with revision... compares a file or folder with an earlier version from Git, Subversion or Mercurial. The nearest working copy above the path determines which system is used. The dialog lists the recent revisions that changed the path.

The revision you can enter depends on the version control system:

- In Git, the version can be `HEAD`, a branch, a tag or a commit; if you leave it empty, `HEAD` is used.
- In Subversion, it can be `BASE` (the revision the working copy was updated to), `PREV`, `HEAD`, a

number such as `1234` or `r1234`, or a date such as `{2026-11-01}`; if you leave it empty, `BASE` is used.

- In Mercurial, it can be `.` (the working copy's parent), `.^`, `tip`, a number, a hash, a tag, a bookmark or another revset; if you leave it empty, `.` is used.

The older version is shown read-only on the left, and the current file or folder on the right.

Git, Subversion or Mercurial must be installed and on your `PATH`. Git and Mercurial read their revisions from the local repository. Subversion reads `BASE` of an ASCII path from the working copy. On Windows, that build of `svn` keeps only ASCII arguments, so for a path that is not ASCII OxideMerge starts `svn` in the folder, whose name it receives whole. A folder is given to it there as `.`, and is read as an ASCII path is. A file is read by its repository URL, pinned at the revision the working copy has, so an older revision is found even when the file had another name then, as for an ASCII path. `BASE` of such a file is the pristine copy stored in the working copy, with the line endings its committed `svn:eol-style` gives it, as `svn cat` does; a file committed with `svn:keywords` gets `BASE` from the server, which fills the keywords in. Other revisions come from the server. The list of recent revisions also comes from the server, which may ask for a sign-in that OxideMerge cannot provide. Sign in with `svn` itself first. On Windows, Mercurial reads names in the ANSI code page, as `hg` itself does: a name that code page cannot spell cannot be compared. On Windows, `svn` and `hg` write a folder's revision to your temporary folder, and are given the short (8.3) name of that folder when its path is not ASCII. On a drive with short names turned off, set `TMP`, which Windows reads before `TEMP`, to an ASCII folder to compare a folder with a revision; set `TEMP` to the same folder so other programs agree.

The revision is read in the background; **Cancel** stops a server that is slow or does not respond, and the dialog returns. Clicking **Compare** lets Mercurial read the repository's own `.hg/hgrc`, just as Git's filters are honored. The list of recent revisions, which is read while you type, skips that file; skipping it needs Mercurial 5.4 or later. When you compare the top folder of a working copy, the `.git`, `.svn` or `.hg` folder is listed on the right only; the **Exclude Source Control** filter hides it. Exporting a revision's folder is refused if it would exceed 8 GB or 500,000 files, the same limit that archive extraction already has, so that a mistaken or very large revision cannot fill the disk.

On the command line, use `--rev` (or `--git-rev`, which is the same option):

```
oxidemerge --rev HEAD~1 src/main.rs
oxidemerge --rev PREV src/main.c
oxidemerge --rev .^ docs
```

See also Integrating with other tools: you can also set up OxideMerge as the diff and merge tool of Git, Subversion, Mercurial or Perforce, so that they invoke it themselves instead of you opening a revision from within OxideMerge.

Chapter 5. Merging files and resolving conflicts

When two edited versions of a file need to be brought together, a three-way merge compares both with their common base and combines the changes in a single result. OxideMerge copies the changes that do not conflict for you and guides you through the conflicts that remain. You can also open a file that contains version-control conflict markers and resolve it in the same three-way view.

5.1. Three-way merge

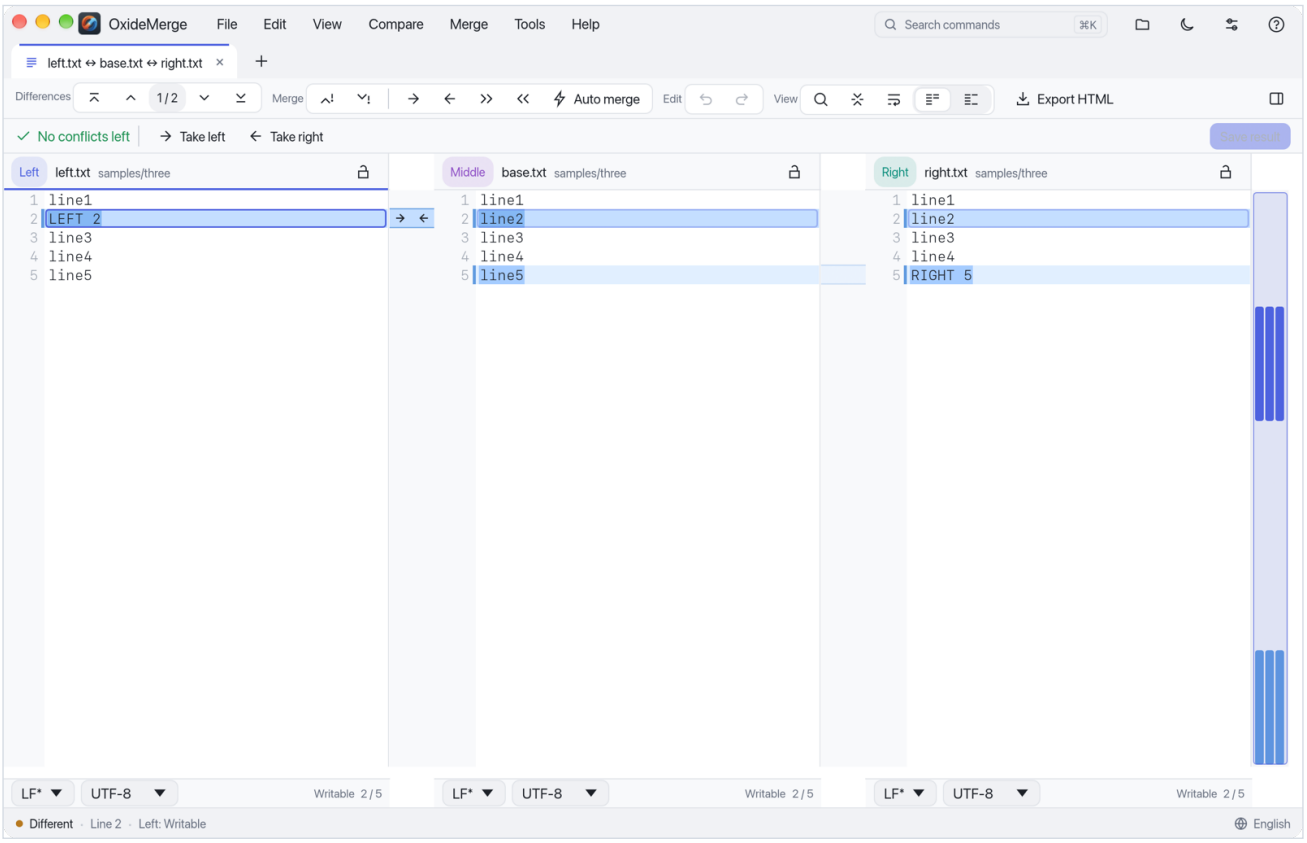


Figure 6. A three-way comparison

A three-way merge combines two edited versions of a file. Open the files in this order: your version, the base, the other version. OxideMerge compares the left and right versions with the base and combines the changes in the middle pane.

5.1.1. Step 1: Choose the output file

By default, saving the middle pane overwrites the base file. To keep the base file unchanged, specify a separate output file when you start OxideMerge:

```
oxidemerge mine.txt base.txt theirs.txt -o merged.txt
```

The middle pane starts with the content of the base file. When you save, OxideMerge writes the result to `merged.txt`. You can also choose **File** → **Save as** to save the result to another file.



If you use the left and right panes only as sources, make them read-only.

5.1.2. Step 2: Merge and resolve conflicts

The merge bar above the panes shows how many conflicts are left and provides these buttons:

- **Next conflict**.
- **Take left**, **Take both** and **Take right**, which apply to the current conflict.
- **Mark resolved**, for a conflict that you have edited by hand. It keeps the middle text as it is and counts the conflict as resolved.
- **Save result**, at the right end, which saves the middle pane. It becomes the main button when no conflicts are left.

To merge the files:

1. Choose **Merge** → **Auto merge**. OxideMerge copies two kinds of changes to the middle pane: changes made on one side only, and the same change made on both sides.
2. Go to each remaining conflict with `Alt` + `PgDn` and `Alt` + `PgUp`.
3. Copy the change that you need from the left or the right, or edit the middle pane.
4. To keep both changes, right-click the difference and choose **Take left, then right** or **Take right, then left**. Then check the result for repeated or conflicting text.
5. Save the result, making sure that the destination path is correct.



Auto merge works only on the text; it does not check that the result is correct. Review the result, and run the checks that your project needs, before you use it.

5.2. Resolving conflict files

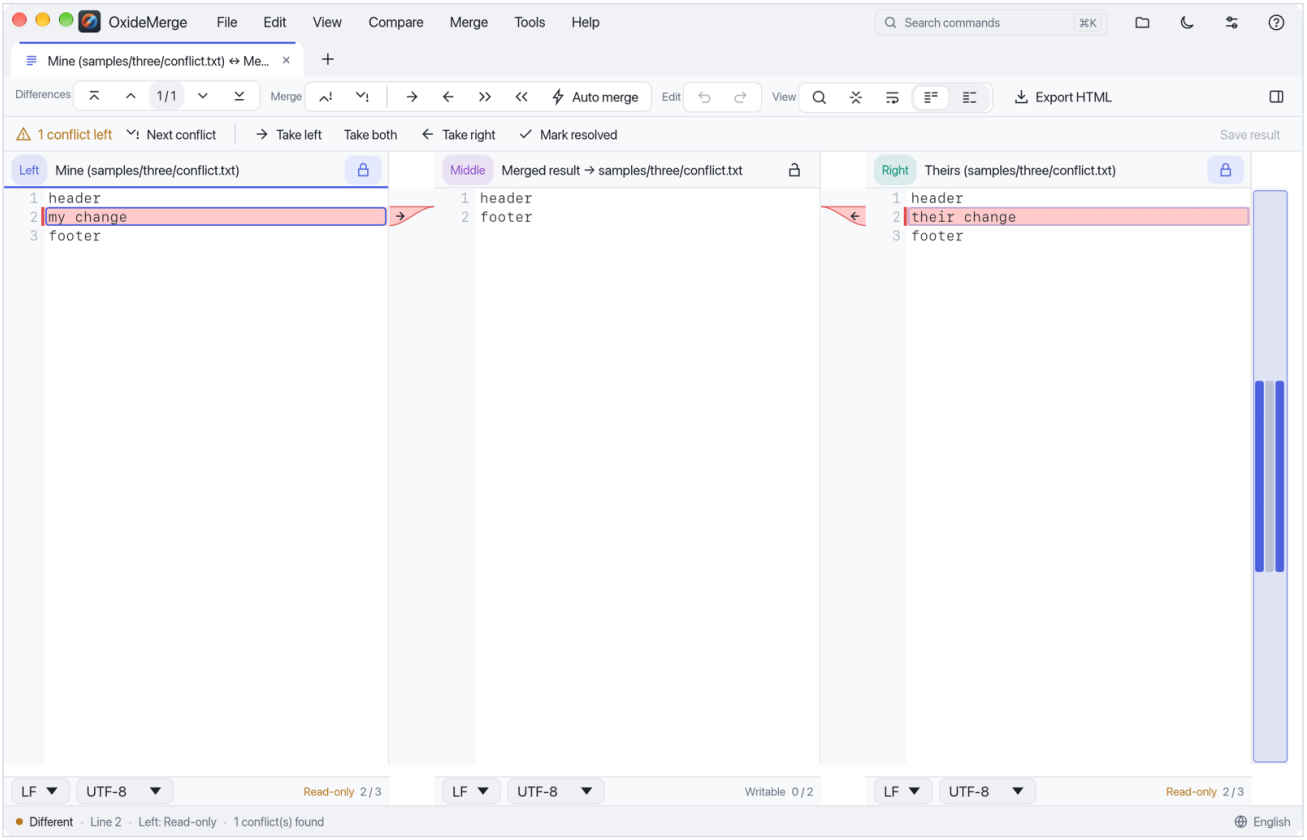


Figure 7. A conflict file in the three-way view

A conflict file is a file that contains version-control conflict markers: `<<<<<<<`, `|||||||`, `=======` and `>>>>>>>`. OxideMerge opens such a file in the three-way view.

To open a conflict file, use any of these:

- **File → Open conflict file...**.
- **Open conflict file** on the start page.
- The command line: `oxidemerge conflict.txt`.

The left side is **Mine**, the right side is **Theirs**, and the middle is the **Merged result**. Resolve each conflict, then save the middle pane. OxideMerge writes the result back to the conflict file.



This differs from opening three separate files. For a conflict file, the original file is the output.

Chapter 6. Comparing folders

A folder comparison shows you how two or three folders differ, item by item. Use it to review the changes across a whole source tree or release, to open any changed file for a detailed comparison, and to copy, synchronize or merge items between the folders.

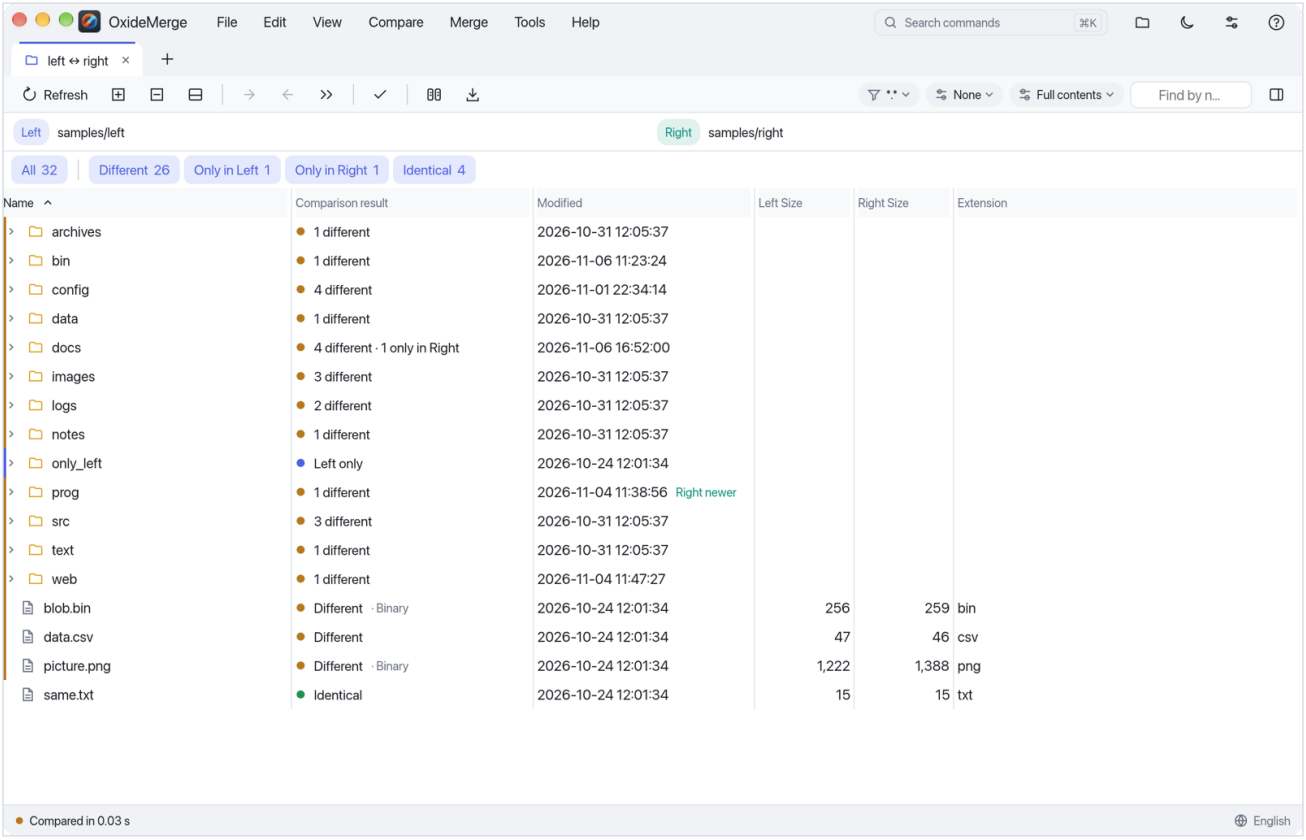


Figure 8. A folder comparison

A folder comparison lists the items of two or three folders together with the result for each item. Results appear while the comparison runs: OxideMerge compares the files during the folder scan and lists each difference as soon as it finds it. If a long comparison finishes while the window is in the background, OxideMerge requests your attention.

6.1. The folder list

Each row of the folder list shows the following columns:

- **Name**.
- **Folder**: the folder that contains the item. This column appears only in a flat list.
- **Comparison result**.
- **Modified**: the newest date among all sides. If the dates differ, the newer side is named after the date. To see the date of each side, move the pointer over the date.
- The size on each side.

- **Extension**.

To sort the list by a column, click its header; click the header again to reverse the order. A colored bar at the left edge of a row marks a difference or an item that exists on one side only.

The comparison result is one of the following:

Result	Meaning
Identical, **Different**	The files are the same or differ. **Binary** marks binary files.
For example, **3 different · 1 only in Left**	A folder whose contents differ. The text summarizes what is different inside it.
Left only, **Middle only**, **Right only**	The item exists on one side only.
Left is different, **Middle is different**, **Right is different**	Three-way comparison: one side differs from the other two.
Missing in Left, **Missing in Middle**, **Missing in Right**	Three-way comparison: the item exists on the other two sides only.
Error	OxideMerge could not read the item.

To see the full description of a result, for example **Text files are different**, move the pointer over it.

The chips under the folder paths show item counts: **All**, **Different**, **Only in Left**, **Only in Right**, **Only in Middle** (three-way only) and **Identical**. Click a chip to hide or show that kind of item. Note the following about the counts:

- In a three-way comparison, an item that is missing from one side counts as different.
- Empty folders, and folders that were not scanned, are counted by the result that they show.
- If **Include subfolders** is off, subfolders are not compared. They are counted only in **All**, and no chip hides them.

The status bar shows how long the comparison took.

6.2. Display options

View → **Folder list** contains the following options:

Tree mode

Shows subfolders as a tree. Turn it off to show a flat list. Press → and ← to expand and collapse a folder. The toolbar also provides **Expand all** and **Collapse all**.

Show

Hides or shows **Identical**, **Different**, **Left only**, **Middle only** and **Right only** items.

File compare below the folder list

Shows the differences of the selected file below the list, without opening a tab. To show or hide this pane, click the pane button on the toolbar or press `Alt` + `P`.

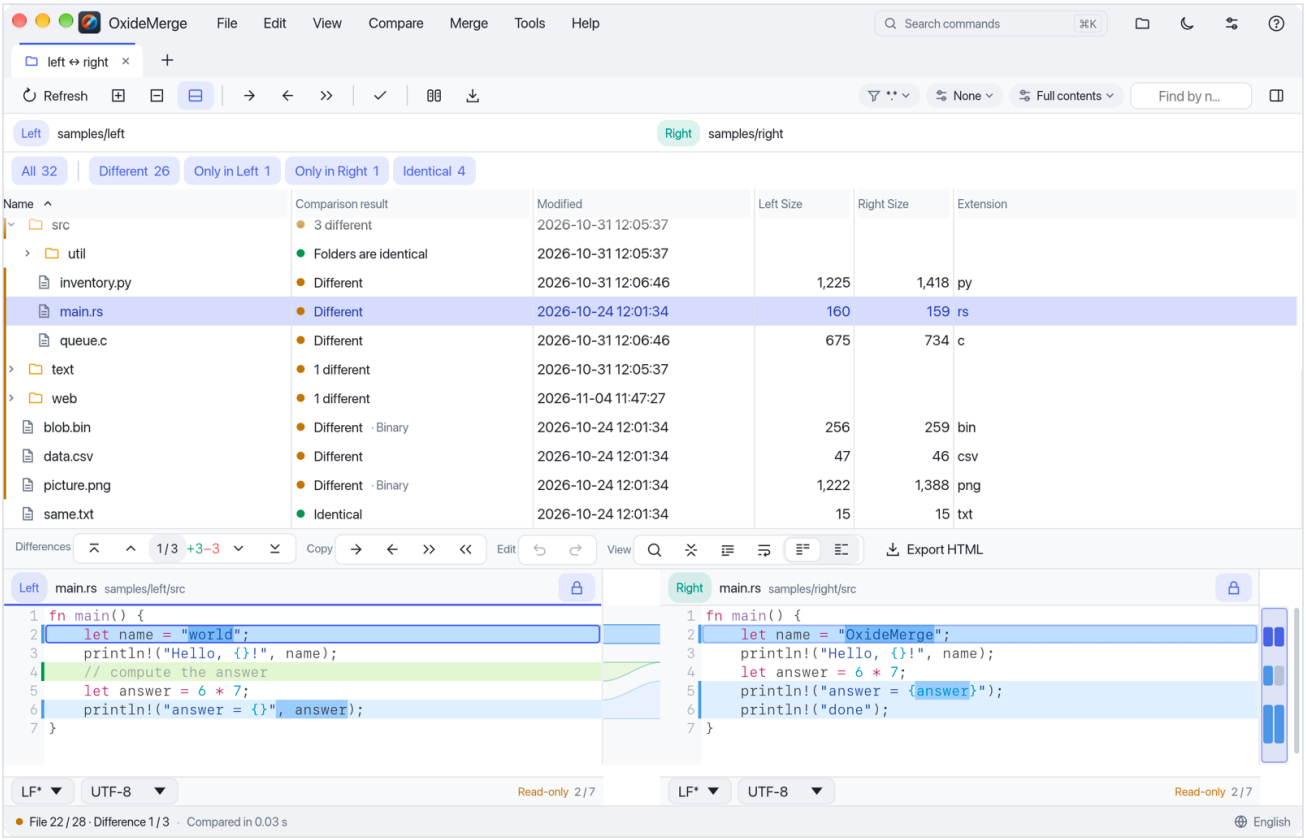


Figure 9. The file compare pane below the folder list

When this pane is shown, the navigation keys behave differently:

Keys	With the file compare pane	Without the pane
`Alt` + `↓`, `Alt` + `↑`	Go to the next or previous difference in the file. After the last or before the first difference, continue to the next or previous file that has differences.	Select the next or previous file that has differences.
`Alt` + `Home`, `Alt` + `End`	Go to the first or last difference in the file.	Select the first or last file that has differences.

The arrow buttons of the pane work the same way as `Alt` + `↓` and `Alt` + `↑`. If you press one of these keys while a file is still loading, OxideMerge waits for the file, so no difference is skipped. If a review filter is active, these keys move only through the files that the filter lists.

6.3. The toolbar

The toolbar contains the following commands, from left to right:

- **Refresh** (`F5`) compares the folders again.
- **Expand all** and **Collapse all** are available in tree mode.
- **File compare below the folder list** (`Alt` + `P`) shows or hides the file compare pane.
- **Copy to right** and **Copy to left** copy the selected items. In a three-way comparison, these buttons are replaced by **Copy from left to middle**, **Copy from right to middle** and **Auto merge folders…**.
- **Synchronize folders…**, available in a two-way comparison, previews and applies a synchronization (see Synchronizing two folders).
- **Select by name…** selects the items whose names match wildcards, such as `*.c; *.h`.
- **All differences** opens all changed text files in a single tab (see Reviewing all differences).
- **Export HTML** saves an HTML report (see Saving, sharing and reporting).
- **Filter** selects which files and folders are compared (see Filters).
- **Rules** selects the compare rules file for this comparison (see Compare rules).
- **Compare method** selects how files are compared (see Comparison methods).
- **Find by name** (`Ctrl` + `F`) shows only the items whose names contain the text you type.
- **Side panel**, at the right end, opens a drawer with the review, verification and baseline panels (see Analysis tools).

In a narrow window, **All differences** and **Export HTML** are shown as icons, and the chips show only their numbers.

6.4. Comparison methods

The comparison method determines how OxideMerge decides whether two files are the same. You choose it with **Compare method** on the toolbar.

Method	How files are compared
Full contents	Text files are compared line by line, using the comparison options and compare rules. Files that differ only in ignored ways count as identical.
Quick contents	The bytes are compared. Files of different sizes are detected without being read.
Binary contents	The bytes are compared, as with **Quick contents**. The result is marked as binary.

Method	How files are compared
Modified date	The modification dates are compared. **Date tolerance (seconds)** in **Options** → **Folder** sets how far apart two dates can be.
Modified date and size	Both the dates and the sizes must match.
Size	Only the sizes are compared.

To choose a method:

- Use **Full contents** when compare rules and ignored differences matter.
- Use **Quick contents** or **Binary contents** to verify that files are exactly the same.
- Use a date or size method for a fast first check. Note that matching dates and sizes do not prove that the contents are the same.

The byte methods read files in small chunks, so large files do not have to fit in memory. A full text comparison can still load changed files into memory.

6.5. Selecting and opening items

You can select items with the mouse or the keyboard:

- Press the arrow keys, `PgUp`, `PgDn`, `Home` or `End` to move the selection. Hold `Shift` to extend it.
- `Ctrl`+click an item to add it to the selection or remove it.
- Press `Ctrl` + `A` to select all items.

To open items:

- Double-click an item, or press `Enter`, to compare it in its own tab. In tree mode, this expands or collapses a folder instead. To open a folder in its own folder comparison, use a flat list, or right-click the folder and choose **Compare**.
- Press `Space` to open **Quick preview**, a color-coded view of the differences of the selected file that does not open a new tab. In the preview, `↑` and `↓` move between items, `Enter` opens the item, and `Space` or `Esc` closes the preview.

6.6. Copying and deleting items

You can copy and delete items directly from the folder list:

- Press `Alt` + `→` or `Alt` + `←` to copy the selected items to the right or to the left.
- Right-click an item for more commands:
  - **Copy** to another side.

- **Copy to folder…**: copies the selected items of one side to another folder, preserving their relative paths.
- **Delete** from a side.
- **Open with associated application**.
- **Show in file manager**.
- **Copy full path**.
- **Select by name…**.

The following settings in **Options** → **General** control file operations:

- **Confirm copy operations**: OxideMerge asks for confirmation before it copies items. It always asks before it deletes items.
- **Use the recycle bin when deleting**: deleted items are moved to the recycle bin or trash. If an item cannot be moved there, OxideMerge does not delete it.
- **Keep a .bak copy when saving over a file**: OxideMerge keeps the previous content of a replaced file.



Copy and delete operations change the files on disk immediately. They do not wait for **Save**, and the **Undo** command of the text editor does not reverse them. Check the selected items and the destination before you proceed.

6.7. Reviewing all differences

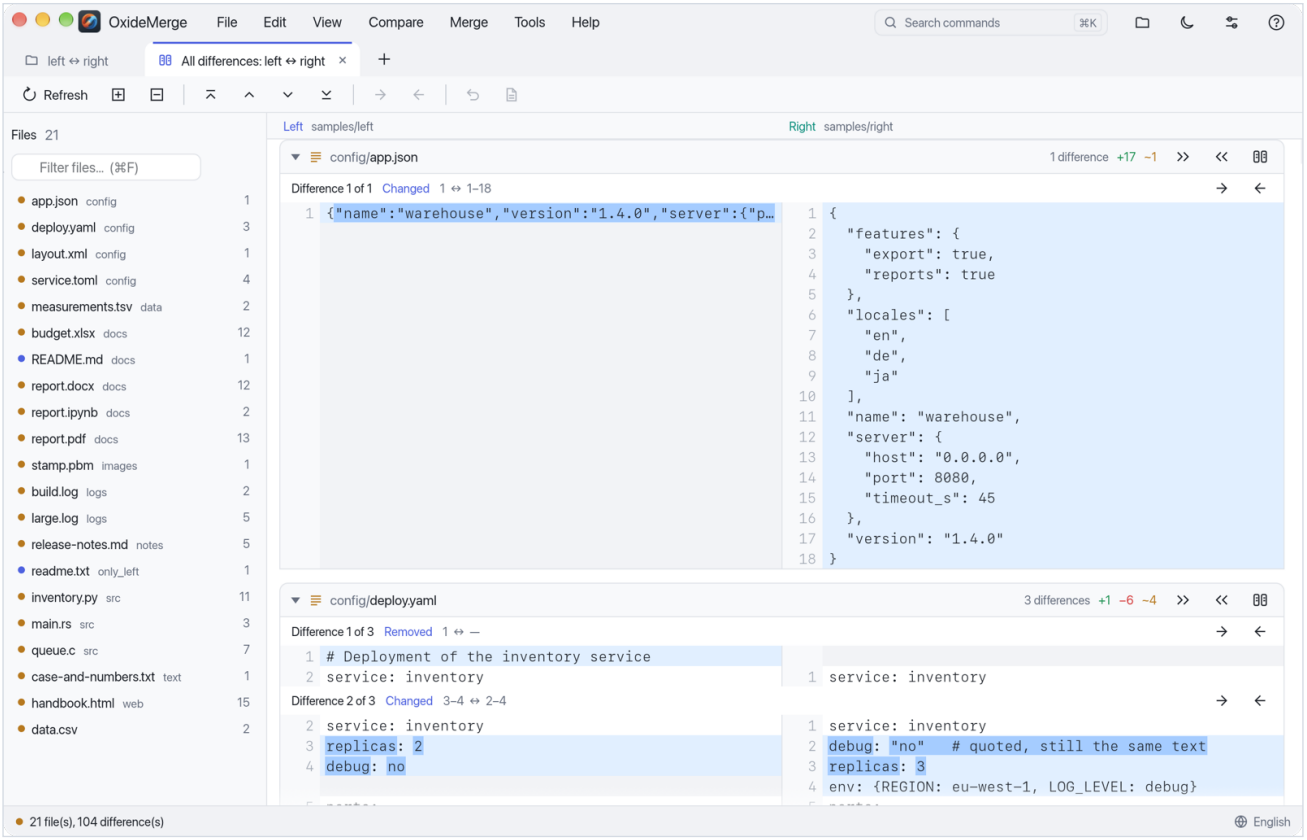


Figure 10. All differences

All differences shows every changed text file of a folder comparison in a single tab, so you can review and edit all of the changes in one place. To open it, click **All differences** on the folder comparison toolbar, or choose **Compare** → **All differences**. The filter and the **Show** settings of the folder comparison apply.

- The list on the left shows each file and its number of differences. To shorten the list, use **Filter files...** (`Ctrl` + `F`). Click a file to scroll to it.
- Each file appears as a card that shows its differences side by side. Click the arrow in a card header to collapse or expand the card. The toolbar collapses or expands all cards.
- `Alt` + `↓` and `Alt` + `↑` move through the differences of all files. When you open **All differences** from a folder comparison, it starts at the difference that was selected there. A card that is still loading is waited for, not skipped. The status bar shows the position, as described in Moving through the differences of a folder.
- The arrows next to a difference copy it to another side. In a two-way comparison, the buttons in a card header copy the whole file.
- **Undo** reverses the last change.
- **Save** writes the changed files and then refreshes the folder comparison.
- The button at the right of a card header opens the file in its own tab.

Note the following:

- If you copy from a side where a file is missing, the file is deleted when you save. Folders that

become empty are deleted as well. If backups are on, the deleted file is kept as `.bak`, so its folder remains.

- If a file changes on disk after it was loaded, you cannot save its card. Instead, the card offers to discard your edits and load the file again. Before you choose this, copy any edits that you need to keep.
- A file marked read-only on disk is not saved, replaced or deleted; its card keeps your edits.
- Cards without edits reload when their files change on disk.
- **Refresh** (`F5`) also reloads the compare rules and the options.
- To include files that changed after you opened **All differences**, save or discard your edits, and then open **All differences** again.

6.8. Merging folders automatically

Automatic folder merging combines the changes made on two sides into a common base folder in a single operation. In a three-way folder comparison, the middle folder is the base. **Merge → Auto merge folders...** brings the changes from both sides into the middle folder:

- An item that was changed, added or removed on one side only is copied or removed.
- A change that is the same on both sides is applied once.
- A text file that was changed on both sides is merged line by line, provided that the changes do not overlap.

6.8.1. Before you start

To prepare for a merge:

1. In **Options → Folder**, turn on **Include subfolders** and **Include contents of unique subfolders**.
2. Wait for the comparison to finish.



The merge writes its result into the middle folder. To keep the original base folder, use a copy of it as the middle folder.

6.8.2. Review the plan

OxideMerge first shows a plan that lists the items it will copy, delete and merge, together with the conflicts that it cannot resolve. Review the plan before you confirm it.

OxideMerge does not change the following files: files with edits that conflict, files with content that it cannot merge, symbolic links, and files that it cannot read. If a file changed after the comparison, refresh the comparison and try again.

6.8.3. How the merge protects your files

The merge is all-or-nothing: either every change is made, or the middle folder stays as it was.

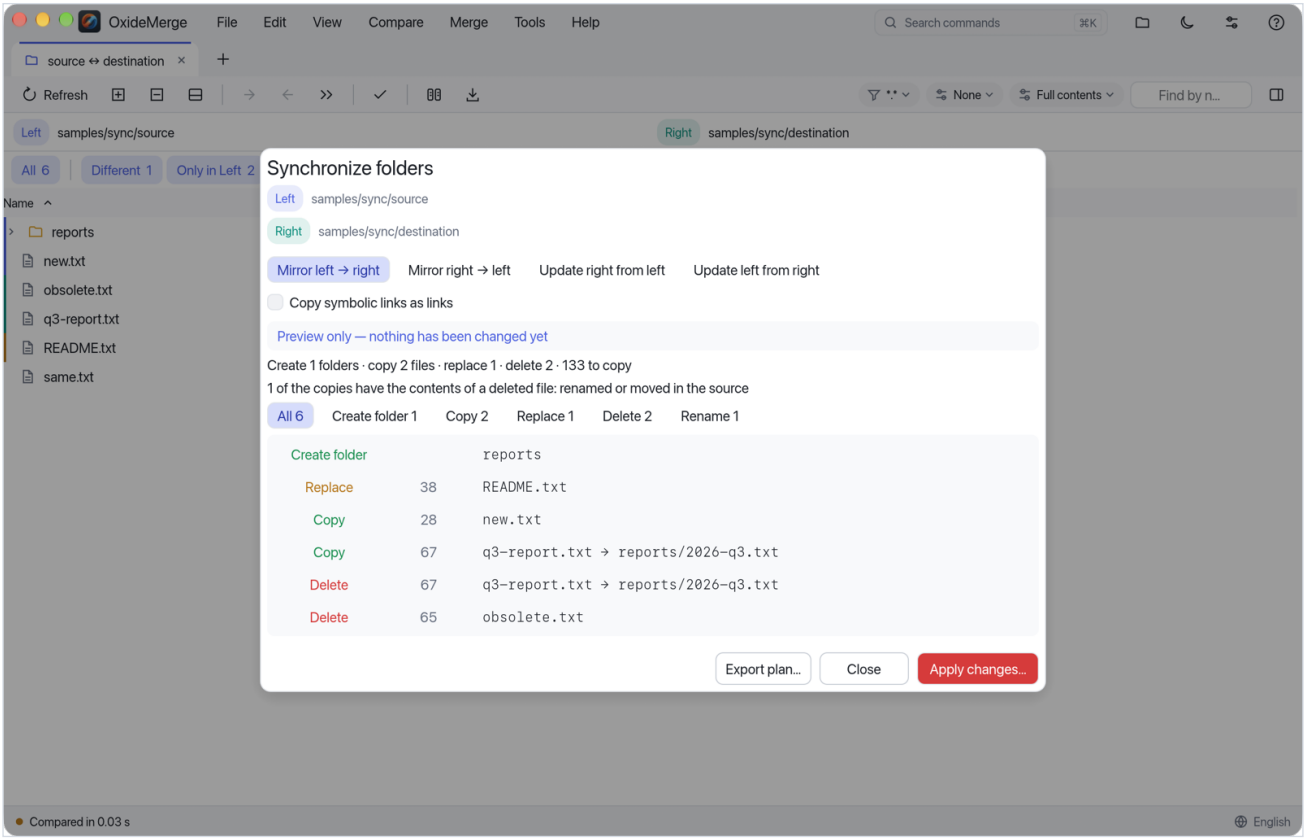
- *Before the merge*, OxideMerge checks every item. Nothing is merged if an item changed after the plan was made, if an item would be written outside the middle folder, or if a backup file would replace a compared file. OxideMerge lists any such items.
- *During the merge*, OxideMerge checks each file again immediately before it writes the file. If another program changes a file, the merge stops. You can also click **Stop** to stop the merge.
- *When the merge stops or fails*, for example because a folder is read-only or the disk is full, OxideMerge undoes all changes made so far. The middle folder is then restored to its previous state, and removed folders regain their permissions. A file that the merge moved to the trash is restored from a copy, so the trash keeps a copy as well.

To make the undo possible, OxideMerge keeps copies of the original files in a `.oxidemerge-undo-⋯` folder at the top of the middle folder. If it cannot create this folder there, it uses the system's temporary folder. Folder comparisons do not list this folder. A deleted file is only moved into this folder, and is moved back if the merge is undone.

If OxideMerge cannot restore a file, it keeps the undo folder and shows its location. The `index.txt` file in that folder lists the original path of each copy, so you can restore the files manually.

6.9. Synchronizing two folders

In a two-way folder comparison, **Synchronize folders…** (on the toolbar and in the **Merge** menu) makes one folder, the *destination*, match the other, the *source*. You see the plan first, and nothing changes until you apply it.



6.9.1. Modes

Mirror left → right, Mirror right → left

The destination ends up with the files of the source. New and changed files are copied, and files and folders that exist only in the destination are deleted.

Update right from left, Update left from right

New and changed files are copied. Nothing is deleted.

A read-only side, such as a Git revision, cannot be the destination. OxideMerge decides by the contents (SHA-256), never by a newer modification time alone. Files that compared as identical but differ in size or time are read to make sure.

6.9.2. Before you start

To prepare for a synchronization:

1. Turn on **Include subfolders** and **Include contents of unique subfolders** in **Options → Folder**.
2. Wait for the comparison to finish. A stopped comparison cannot be synchronized.

The two folders must be different folders, and neither may be inside the other.

6.9.3. The preview

The preview is read-only. It lists each item with its action and the reason for it:

Action	Meaning
Create folder, Copy	The item exists only in the source.
Replace	The contents differ. The destination file is overwritten.
Delete, Remove folder	The item exists only in the destination (mirror modes only). A folder is removed only if it is empty afterwards.
Skip	The item exists only in the destination, and the mode is an update mode.
Conflict	OxideMerge leaves the item untouched because it is not safe to decide.

The following items are conflicts:

- items that cannot be read;
- symbolic links, which are never followed or copied;
- devices;
- a file on one side and a folder on the other;
- names that differ from another name only in letter case, or in how accents on Latin letters are written (composed or not);
- items that changed after the comparison.

A file whose name the two sides spell differently only in case or in accent form, and whose contents are the same on both sides, is not a conflict when mirroring: it is renamed (see below).

On macOS and Linux, select **Copy symbolic links as links** to synchronize links rather than leave them alone. A link is then treated as the link itself, never as what it points to:

- A link that exists only in the source is created in the destination, with the same target text.
- A link whose target text differs is replaced.
- When mirroring, a link that exists only in the destination is deleted; an update keeps it.
- A link facing a file or folder of the same name is still a conflict.

What a link points to is never read, written or deleted, even when it is outside both folders. The links that are changed are kept in the recovery journal like files, so undoing restores them.

Click an item to see its details: the size and the start of the SHA-256 on each side, and whether the action adds to the destination, or overwrites and deletes in it.

A copy whose contents match a file that the plan deletes is a file that was renamed or moved in the source. **Rename** lists such pairs as *old name* → *new name*, and the copy and the deletion also show

the other name. A rename is applied by moving the file in the destination, so nothing is copied or kept in the journal: the file is first renamed to a temporary name, which the journal records, and then to its new name. A rename that changes only the case of a name, or how its accents are written, is made the same way, also on file systems that treat both names as one, such as those of Windows and macOS. When the old and the new location are on different drives, the file is copied and the old one is deleted. A folder renamed only in case is still left alone, and so is a file renamed only in case whose contents also changed.

Warnings appear above the list:

- When a filter or the hidden-file setting left items out, a mirror does not delete them, so the destination may not be an exact mirror. A folder that still holds such items is kept.
- Large deletions are flagged and require a second confirmation.

Export plan… saves the plan as JSON, with a version number and paths relative to the folders. The plan does not contain the absolute paths of the folders. It records the filter and a SHA-256 of the two folder paths, so a plan is applied only to the folders it was made for.

6.9.4. Applying the plan

Apply changes… asks for confirmation and names the destination. When many items would be deleted, you must also select a check box that states their number.

OxideMerge then works in this order: it creates folders, copies and replaces files, deletes files, and finally removes empty folders. The dialog shows the count and the item being changed. For each item, OxideMerge first checks that the source and the destination are still as they were in the preview. Every file is copied to a temporary file next to its destination, checked against the planned SHA-256 and then renamed into place; a renamed file is moved instead. At the end, all copied and moved files are checked again.

If a file changed after the preview, even if its size and modification time are unchanged, nothing is applied, and the message names the file.

A file that cannot be read at the moment is reported as unreadable, not as changed. On Windows, if another program holds a file open without sharing it, the synchronization stops before anything is changed, and the message names that file. Close the other program and preview again.

On Windows, paths longer than the old 260-character limit are copied and deleted like any other path.

Nothing is applied either when the new and replacing files, or the journal copies of the files that are replaced and deleted, do not fit in the free space of their drive. The message names the drive, the space needed and the space free. Files that are only moved within the destination do not count. This check is made on macOS and Linux; on Windows, the free space is not measured. If a drive still fills up partway through, the write fails and the changes are undone.

6.9.5. If something goes wrong

Before it replaces or deletes a file, OxideMerge copies the file into a recovery journal in the `sync-recovery` folder of the settings folder, never inside the synchronized folders.

- If a change fails, or you click **Stop**, OxideMerge undoes the changes made so far, starting with the most recent one.
- If a file was changed again in the meantime, OxideMerge does not overwrite it. Instead, it reports *Partially applied — recovery required* and keeps the journal with the copies.
- If OxideMerge exits before the synchronization finishes, it offers at the next start to undo the synchronization or to keep the folders as they are. Until you choose, these folders cannot be synchronized again.
- This holds wherever OxideMerge stops: while making the journal, while a copy is still under its temporary name, between a rename's two steps, after a deletion, during the final check, or while undoing. Undoing first removes the temporary files the synchronization left, then puts back each change.
- Undoing can be repeated safely: what is already back is skipped. If something still cannot be undone, the message names each path and why. Deal with each one (a program holding the file open, a full disk, a missing permission, or a file changed since), then choose **Undo those changes** again.



Applying a plan to several files is not a single atomic step. Each file is replaced atomically, but another program can see some files already changed. The trash is not used.

6.10. Documents, spreadsheets and programs

Some files are not plain text but contain text. OxideMerge reads these files itself, with nothing to install, so you can compare PDF files, office documents, spreadsheets, programs and notebooks by their content:

Files	What is compared
`.pdf`	The text of each page, in the order in which it is drawn, with a blank line between pages, including the text of forms that a page draws. The characters come from the `ToUnicode` table that the file carries for its fonts, or from the encoding that the font names. An encrypted file is reported rather than read. When part of a file cannot be read, such as a stream in a compression that OxideMerge does not read, or a font without a table of the characters that its glyphs stand for, a line at the end states that the text is incomplete and why. A file whose text cannot be read at all is reported.

Files	What is compared
`.docx`, `.odt`, `.rtf`	The words of the document, one line per paragraph or heading. The cells of a table are separated by tabs, one row per line. Words shown as deleted and field codes are left out.
`.xlsx`, `.xlsm`, `.ods`	Every sheet, one line per row, with its cells separated by commas. Empty rows between rows are kept, so rows keep their places; empty rows after the last row are not. A workbook opens as a table compare. With one sheet, columns, sorting and **Match rows by → Key columns** all apply. With more than one sheet, each sheet has a tab below the table that shows how many of its rows differ; the rows are matched by position, because the sheets do not form one table. Sheets are paired by name, so a sheet that only one side has is one whole difference. **Formatted values** shows numbers and dates as their number formats do, as a spreadsheet program shows them; turn it off to see the values as stored. The rows are compared by the stored values either way. In the text view, each sheet is named before its rows.
ELF programs, shared objects and object files	First, the header: class, byte order, kind, machine, ABI, entry point and the counts of the headers. Next, the program headers, one line per segment, with its type, offset, virtual and physical address, sizes in the file and in memory, alignment and `R`, `W`, `E` flags (plus the interpreter that a dynamically linked program requests). Then the section table, one line per section, with its name, type, address, offset, size, alignment and flags. Finally, the symbols of each symbol table (`.symtab`, `.dynsym`), sorted by name, one line per symbol with its address, size, type, binding, visibility and section (`UND` when the symbol is undefined, `ABS` for an absolute value). As a result, two builds line up function by function, and a function that grew or moved shows as a changed line. The mapping symbols that ARM and RISC-V assemblers add (`$x`, `$d`, …) are left out and only counted, and at most 100,000 symbols of a table are listed.
`.ipynb`	A Jupyter notebook, cell by cell: each cell appears under a line that identifies it as a code cell, a Markdown cell or a raw cell, followed by that cell's own source and nothing else. Cells are not numbered, so adding a cell does not change the label of every cell after it. Outputs, execution counts and other metadata that a notebook collects when it is run are left out, so a notebook that is re-run without changes compares as equal. Older nbformat 3 notebooks are read as well, with their heading cells read as Markdown headings. At most 20,000 cells are read; the end of the text states how many were left out. A notebook that cannot be read this way, such as a broken one, is compared as its JSON text.

A file read this way is **read-only**: the pane states what was read (for example `report.docx [text of`

`the document]`), and nothing can be edited or saved over the original. The text is a reading of the file, not a copy of it, so the layout, fonts, formulas and anything else that the format holds are not compared.

You can also compare two or three `.docx` files **Formatted** by choosing that view above the panes. The documents are then shown as Markdown, block by block and side by side: headings, paragraphs with their bold, slanted and struck-through words, list items, links and tables. Each block that differs is highlighted together with its changed words. Tables are drawn as tables, with merged cells drawn across the rows and columns they cover. A row added inside a merged cell is highlighted where it is, and the rows that a header cell reaches down into are part of the header. An image is shown as its description with a short fingerprint of its picture, so a changed picture shows as a change even when its description is the same. **Next difference** and **Previous difference** step through the changed blocks. As in the text, the Formatted view leaves out headers, footers, notes and comments, and shows tracked changes as accepted.

Workbooks (`.xlsx`, `.xlsm`) can also be compared **Formatted**, whether they have one sheet or several. Each sheet appears as a heading and a table. Sheets are matched by name, so a sheet that was added or removed shows as one whole difference. Merged cells are drawn across the cells they cover. Numbers are shown in their number formats: dates and times in ISO form (`2026-09-30 18:00`), whatever order the format puts them in, and amounts with their decimals, thousands separators, currency and percent signs. A formula shows the result it was saved with. A run of three or more empty rows is shown as one row that states how many rows there are, so a value that moved down a row still shows as a change. Each picture placed on a sheet is listed after its table with its description, the cell where it is placed and a fingerprint of the picture, so a changed or moved picture shows as a change. Fills, fonts, borders, the formulas themselves and charts drawn from data are not compared.

A tagged PDF file can be compared **Formatted** as well. Most word processors tag the PDF files they export, marking which text is a heading, a paragraph, a list item, a table cell or a figure, and which cells span several rows or columns. The view is read from those tags, so a PDF made from a document reads much as the document itself does. Text in a bold or slanted font keeps that formatting, unless a whole paragraph or heading is in that font, as its style already shows it. A figure is shown with its alternate text and a fingerprint of its picture. Running headers, footers and page numbers are left out, because the file marks them as artifacts. A PDF that is not tagged, such as a scan or a file from a program that does not tag, shows a message saying so instead; compare such a file as text.

The **Text** / **Formatted** switch appears above the panes when every side is a `.docx` file, every side is a workbook (`.xlsx`, `.xlsm`) or every side is a `.pdf` file, read by OxideMerge itself. It is not offered for `.odt`, `.rtf` or `.ods` files. A comparison of `.docx` or `.pdf` files opens in the Formatted view; a workbook opens in the Table view. The Formatted view reads the files itself even when a converter is set up for them under **Options** → **Plugins**. Whether a block differs depends on its text alone; the comparison options decide only which words of a changed block are highlighted. The view is rebuilt when a file or an option changes. While the view is shown, the difference counter on the toolbar still counts the differences of the text comparison, whereas **Next difference** and **Previous difference** step through the changed blocks of the Formatted view.

The following limits and rules apply:

- A PDF or RTF file can be at most 256 MiB, because it is read in full. A larger file is reported rather than read.
- Each part of a `.docx`, `.odt`, `.xlsx` or `.ods` file can be at most 64 MiB. A file with a larger part, or with a part that cannot be read, is reported rather than compared without that part.
- An ELF file can be any size: only its headers, section table and symbol tables are read, each up to 64 MiB. Because two different builds can still read the same, a folder comparison by **Full contents** compares programs by their bytes.
- A PDF file can request at most 1 GiB of decoded content in total; a file that needs more is reported rather than read.
- The text read from one file can be at most 16 MiB, in the **Text** and the **Formatted** modes alike. The rest is left out, and a line at the end says so.
- In the **Formatted** mode, a table shows at most 1,048,576 cells and 1,024 columns; the rows beyond that are left out, with the same line at the end. A part of more than 3 million elements is too large to show formatted and is reported; the **Text** mode still reads it.
- A workbook in which none of the listed sheets can be found is reported rather than compared as empty.

A converter set up under **Options** → **Plugins** takes precedence over this built-in reading, so you can always put a better reader in front, such as `pdftotext` for PDF, or Pandoc or Tika for more formats. The output of a converter is compared as text, even for a spreadsheet. If you turn off **Read documents, spreadsheets and ELF files as text** there, these files are compared as the archives or the bytes that they are.



The older Word `.doc` and the older Excel `.xls` formats are not read here yet; use a converter under **Options** → **Plugins** for those. A PDF holds the positions of glyphs rather than words, so columns, tables and ligatures may not come out as they look on the page; a converter such as `pdftotext -layout` reads a complex page better.

6.11. Archives

OxideMerge extracts archives and compares them as folders, so you can review and edit the contents of an archive in the same way as an ordinary folder. The following formats are supported:

- ZIP-based formats: `.zip`, `.jar`, `.apk`, `.docx`, `.xlsx`, `.odt`, `.epub`, `.nupkg` and more, with entries stored or compressed with Deflate, Deflate64, bzip2, LZMA, xz, Zstandard or PPMd.
- `.tar`, `.tar.gz` / `.tgz`, `.tar.bz2` / `.tbz2`, `.tar.xz` / `.txz` and `.tar.zst` / `.tzst`.
- `.7z`.

When you open a password-protected ZIP archive (with either the original ZIP encryption,

"ZipCrypto", or WinZip AES) or 7z archive (including one whose file names are encrypted too), OxideMerge asks for its password in an **Archive password** window. If the password is wrong, it asks again; **Cancel** gives up opening the archive. The password is kept only in memory, only while the comparison is open, and is overwritten there once it is no longer needed.

Edits to the extracted files are saved back into the archive. When you copy, delete, merge or edit a file and then save, OxideMerge writes a new archive in place of the original and keeps its permissions. Only an archive whose files changed is written. The entries that you did not change are kept exactly as they were, with their dates, compression and order; changed entries get the current time. A changed ZIP entry keeps its compression method. An entry compressed with Deflate64 or LZMA, which can only be read, is compressed with Deflate instead, and so is an unchanged LZMA entry, because other tools refuse a copied one. A 7z archive packs entries together into compressed blocks: a block that holds only entries you did not change is copied as it was, and any other block is compressed again with LZMA2, with its entries keeping their dates and attributes.

An encrypted archive stays encrypted. Changed and new entries are encrypted with the password that the archive was opened with, in the same way as the archive already was (AES with the same key strength, or ZipCrypto; the file names of a 7z archive stay encrypted if they were). Unchanged AES-encrypted ZIP entries and 7z blocks are copied as they were; unchanged ZipCrypto entries are encrypted again, because a copy would lose the mark that says they are encrypted.

A `.tar.xz` archive is compressed again as `xz` does by default, and a `.tar.zst` archive as `zstd` does. Compressing a large `.tar.xz` or 7z archive can take a while, so it is written in the background while a window shows which archive is being written; uploads, closing a tab and quitting wait for it. If the archive itself was changed on disk after it was extracted, or is read-only, saving into it is refused instead of losing those changes. No `.bak` backups are made inside an extracted archive. An archive fetched over SFTP or FTP is uploaded again after it is saved.

You can also edit an archive inside an extracted archive. Saving it also writes the archive it came from, and so on out to the outermost one, and an open comparison of that archive is compared again.

The following archives stay read-only, because there is nowhere for their changes to go back to, or because writing them again would break them:

- an archive opened as a snapshot in a review, or a page tree compared from the web;
- signed packages (`.apk`, `.ipa`, `.appx`, `.msix`, `.crx`, `.xpi`, and any zip carrying a `META-INF` signature or an `_xmlsignatures` folder), because rewriting them invalidates the signature;
- an archive inside one of these.

OxideMerge refuses archives that would extract to more than 8 GB or more than 500,000 files. RAR is not supported.

Chapter 7. Comparing binary files

Use a binary comparison to inspect and repair files that are not text, such as compiled programs, firmware images, archives and data files. OxideMerge shows both files byte by byte, points out exactly where they differ, and lets you edit individual bytes and copy differences from one file to the other.

Binary files open in a side-by-side hexadecimal view. Bytes that differ are highlighted, and both sides scroll together.

- `Alt` + `↓` and `Alt` + `↑` go to the next and the previous difference.
- `Ctrl` + `F` finds hexadecimal bytes, such as `DE AD BE EF`, or text.
- `Ctrl` + `G` goes to an offset, which you can write as `0x1F00`, `1F00h` or a decimal number.
- You can copy differences to the other side and save them, as in a text comparison.

To change the number of bytes shown on each line, use **Bytes per line (binary)** in **Options → Image / Binary**.

7.1. Moved blocks

When data is inserted, deleted or rearranged, identical bytes end up at different offsets in the two files. **Moved blocks** on the binary toolbar highlights identical blocks that are found at different offsets, in the hex and text columns and in the location pane.

In the location pane, each side has a column of its own. Moved blocks are shaded in a light color, and solid bars mark the bytes that changed: the differing bytes that no moved block accounts for, such as inserted or overwritten bytes. The bars of the current difference are in the accent color. Click a bar to scroll to it.

To visit the counterpart of a highlighted block, select a highlighted byte and use **Go to matching block**. Your position within the block is kept. You can also tab to the button and press Enter; focus then returns to the byte grid, so you can continue with the arrow keys.

Moved blocks are supported in two-way and three-way comparisons. In a three-way comparison, the button uses the first matching side shown beside it. Turn the checkbox off to see only ordinary differences. OxideMerge remembers the checkbox setting for the next binary comparison.

While the checkbox is on, **Compact rows** and **Aligned rows** on the toolbar lay out the bytes. They are the same **Layout** setting as in text comparisons, also in **View → Layout**:

Compact rows

Each side shows its bytes at their own offsets, so each row has the same offsets on both sides.

Aligned rows

Each moved block is shown on the same rows as its match. Blank cells stand opposite the bytes between the blocks, like the space opposite inserted and deleted lines in a text comparison. Each row starts with the offset of its first byte on that side.

To try it, compare `samples/left/bin/firmware.bin` with `samples/right/bin/firmware.bin`. The left file has an extra byte, `A5`, at offset `0x20`, and the files differ in a few more places further on.

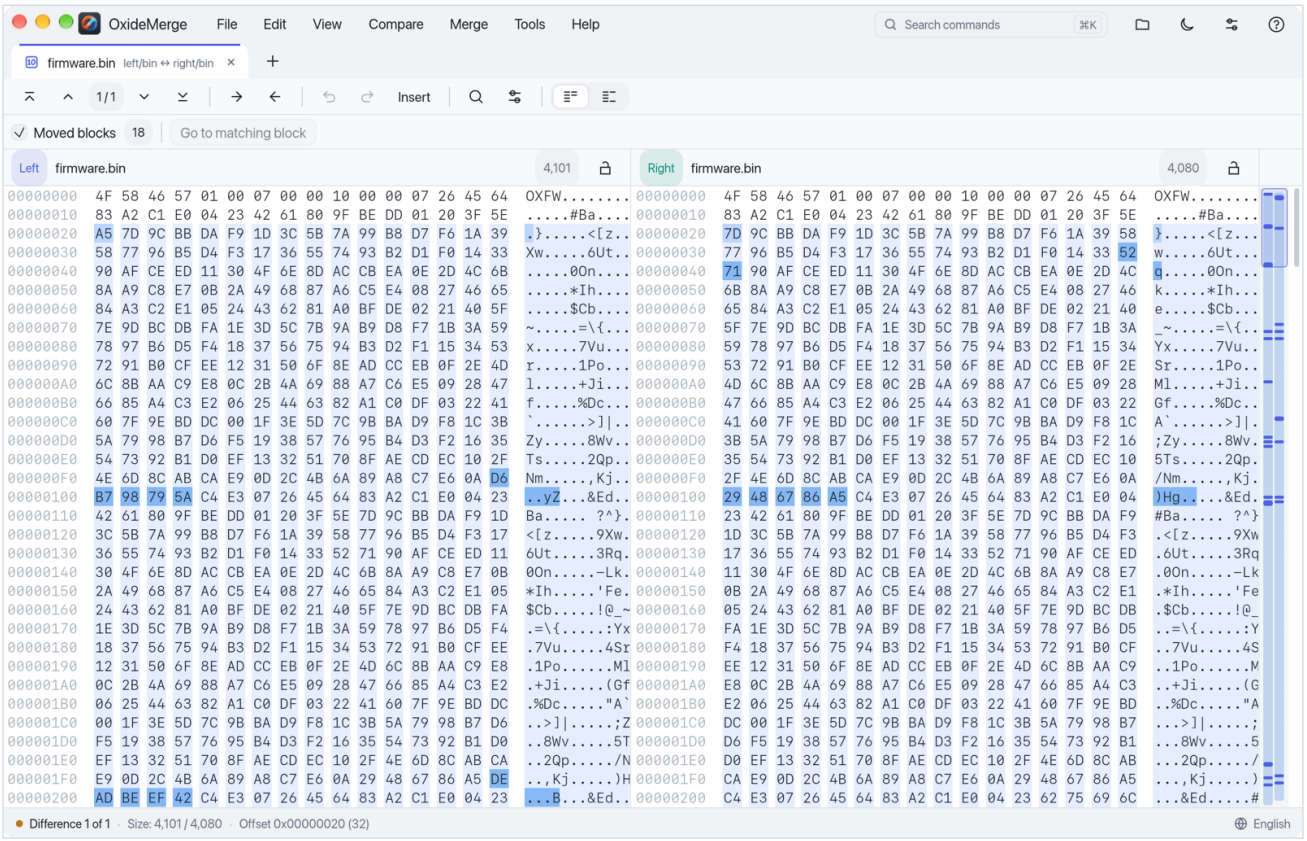


Figure 11. Moved blocks with Compact rows

With **Compact rows**, every byte after `A5` is one offset later on the left than on the right, so from row `00000020` on, the two sides show different bytes on the same row. The bytes that match are highlighted as moved blocks.

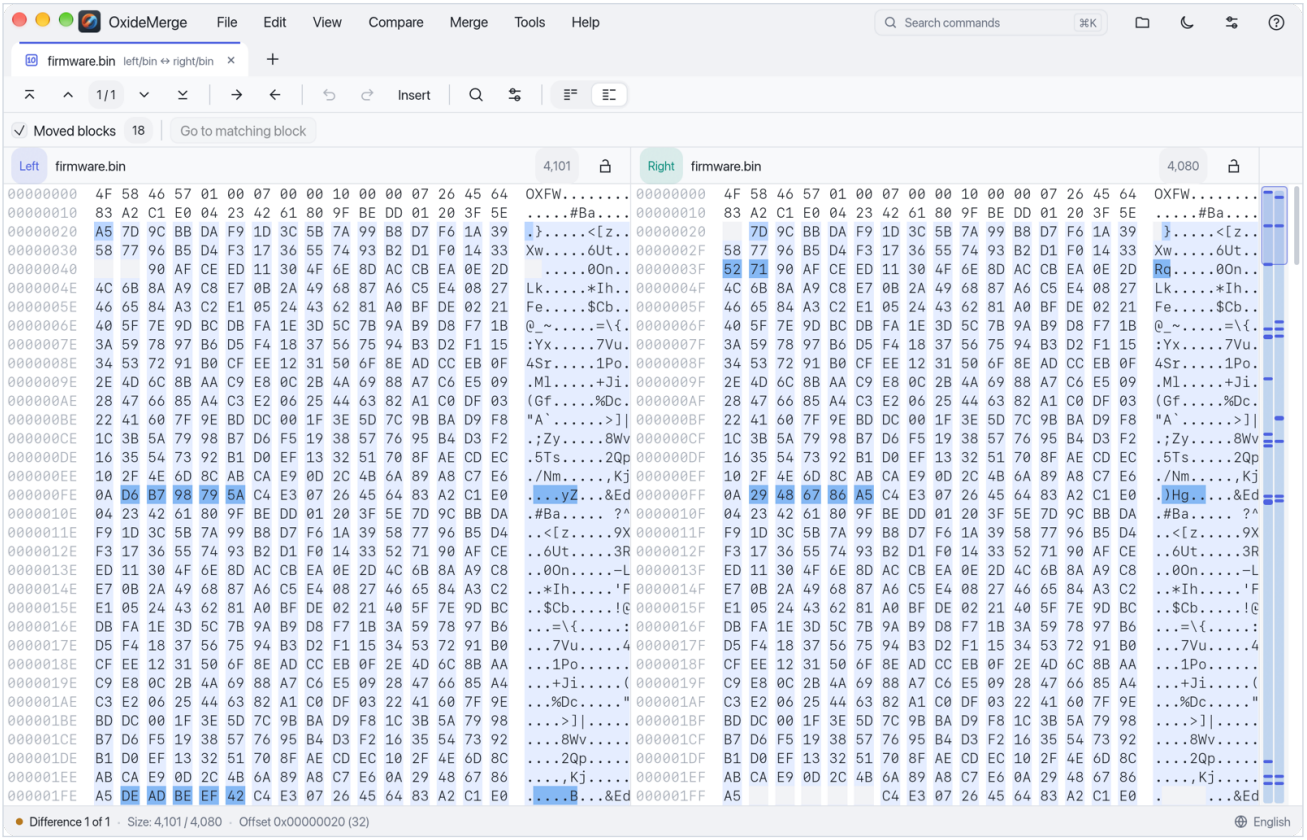


Figure 12. The same files with Aligned rows

With **Aligned rows**, a blank cell on the right stands opposite `A5`, and the bytes after it line up again. At the next difference, two blank cells on the left stand opposite `52 71`, which only the right file has. From there on, the rows of each side start at different offsets, such as `0000004E` on the left and `0000004D` on the right.

Blocks that cross each other, such as two swapped blocks, cannot all be side by side; the longest set that keeps its order on both sides is. In a three-way comparison, the left and right sides are each lined up with the middle side; blocks that only the left and right sides share are highlighted but not lined up. The arrow keys move by the rows shown, and clicking a blank cell selects the next byte of that side. While you edit, the rows stay where they are until the blocks are found again. Difference navigation and the difference count still compare the same offsets of both sides.

Detection finds two kinds of identical bytes:

- *Bytes shifted by an insertion or a deletion*: data near an edit that moved up to 128 bytes forward or back, including short runs between two edits.
- *Blocks moved farther or swapped*: data of at least 16 bytes that occurs only once in the file. Repeated data, such as a run of padding bytes, is not matched this way.



The highlights only add information. The differences, copying a difference, the file contents and the saved bytes stay the same.

Detection runs in the background: once when the comparison opens, and again after an edit, undo, redo or copied difference, as soon as you stop typing for a moment. An edit, or closing the

comparison, stops a scan at once, so detection does not slow down editing or use extra memory. Large files are sampled, so some short matches can be missed. Up to 1,024 blocks are shown for each pair of files.

7.2. Editing bytes

You can change a binary file directly in the comparison, either as hexadecimal values or as characters.

To edit bytes:

1. Click a byte on the side that you want to change. Click in the hexadecimal column to type hexadecimal digits, or in the text column on the right to type characters.
2. Type the new value. In the hexadecimal column, two digits make one byte: the first digit changes the first half of the byte, and the second digit changes the second half. In the text column, each character replaces one byte. Only ASCII characters can be typed.
3. Save with `Ctrl` + `S`.

By default, typing replaces the bytes at the cursor. To insert bytes instead, press `Insert` or click **Insert** on the toolbar. Typing after the last byte always adds bytes.

- The arrow keys, `PgUp`, `PgDn`, `Home` and `End` move the cursor. `Ctrl` + `Home` and `Ctrl` + `End` go to the start and the end of the file.
- `Delete` deletes the byte at the cursor, and `Backspace` deletes the byte before it.
- Pasting (`Ctrl` + `V`) in the hexadecimal column writes hexadecimal bytes such as `DE AD BE EF`; other text is refused there. In the text column, the text is written as it is.
- `Ctrl` + `Z` undoes either the typing since you last moved the cursor, or a paste.

A side that is read-only cannot be changed. To change this, click the lock icon above the side.

Chapter 8. Comparing images

An image comparison shows you exactly what changed between two versions of a picture, such as a screenshot, an icon or a design asset. Several view modes help you spot both large and subtle changes, and you can copy changed regions from one image to the other and save the result.

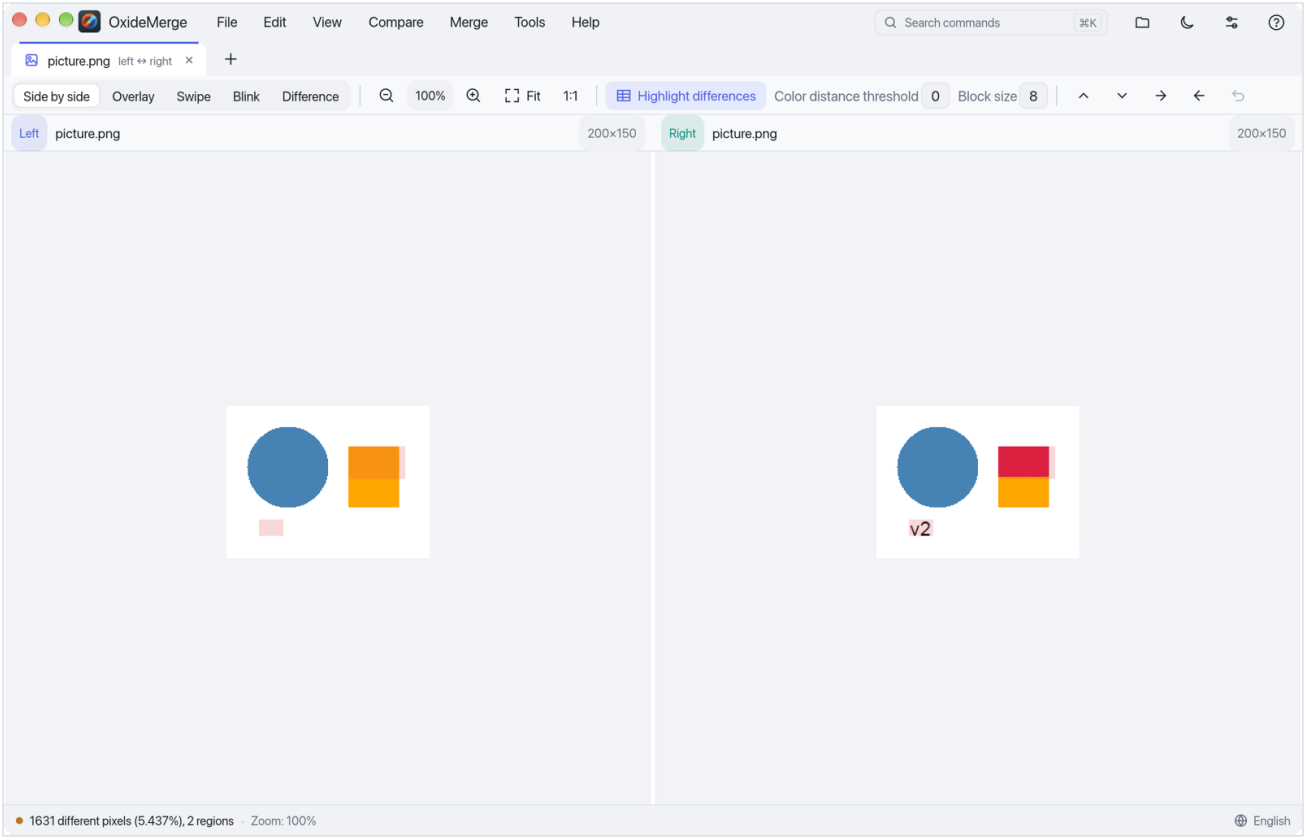


Figure 13. An image comparison

Choose the view mode that suits the change you are looking for:

Side by side

The images are shown next to each other.

Overlay

One image is on top of the other. **Opacity** sets how transparent the top image is.

Swipe

One image is on the left of a divider and the other is on the right. Drag the divider to move it.

Blink

The images take turns, which makes small changes easier to see.

Difference

Only the pixel differences are shown.

To find and copy differences:

- **Highlight differences** marks the regions that are different.
  - **Color distance threshold** sets how different two pixels must be to count as different.
  - **Block size** sets the size of the marked blocks.
- Click a region to select it. `Alt` + `↓` and `Alt` + `↑` go to the next and the previous region.
- `Alt` + `→` and `Alt` + `←` copy the current region to the other image. **Undo** reverses the copy, and `Ctrl` + `S` saves the changed image.

To zoom and move:

- Zoom with the buttons, the mouse wheel or a pinch gesture. The zoom is centered on the pointer.
- **Fit** fits the images to the window, and **1:1** shows them at their actual size.
- Drag, or scroll with two fingers, to move all images together.

The status bar shows how many pixels are different, and in how many regions.

Chapter 9. Comparing web pages

A web page comparison shows you what changed between versions of a page, for example when you check a deployment or review edits to its content. Depending on what matters to you, you can compare the text of the pages, a formatted view of them, their HTML source or the resources that they load.

You can compare two or three web addresses (`http://` or `https://`) or local HTML files. Local `.html` and `.htm` files open as web pages by default. To compare another kind of file as a web page, choose **Compare as** → **Web page**, or use `-t web` on the command line.

9.1. Comparison modes

Each mode looks at the pages from a different angle. Choose a mode in the selector at the top of the tab. Local HTML files start in **Formatted** view. Comparisons that include an online page remember the last mode you used for online pages.

Mode	What is compared
Text	The text of each page, in the text comparison. Each heading, paragraph and list item is one line. The cells of a table row are on one line, separated by tabs. Hidden elements are not included. Changed words are highlighted, and the comparison options, filters and reports all apply.
Formatted	The default mode. Each page is converted to Markdown and shown formatted, side by side. See The Formatted mode.
HTML source	The HTML of each page, in the text comparison. Long minified lines are split between tags.
Resource tree	Each page is downloaded with its style sheets, scripts, images, frames and media. The downloaded files are compared in the folder comparison.

Switching to another mode loads the pages again. After a page changes, click **Reload the pages** (`F5`).

To compare the HTML source of local files line by line, use the HTML source mode, or choose **Compare as** → **Text** (`-t text`).

9.2. The Formatted mode

The Formatted mode lets you review pages much as a reader sees them. It keeps the look of headings, paragraphs, emphasis, links, lists, quotes, code blocks and tables. Images are shown as

links.

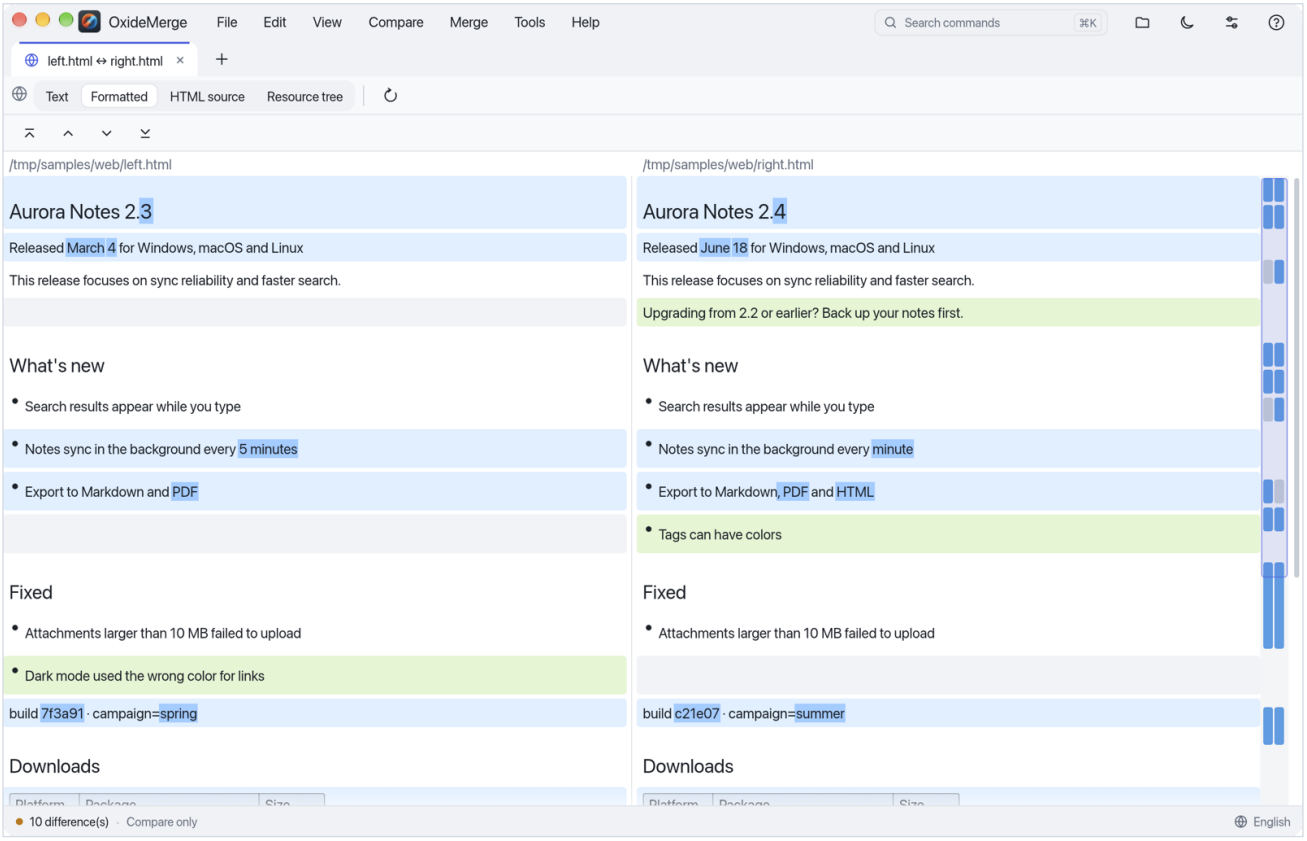


Figure 14. Two web pages in the Formatted mode

OxideMerge compares the pages block by block. A block is a heading, a paragraph, a list item, a table, a quote or a code block.

- Blocks that are different are highlighted. Each one is placed opposite the changed block of the same kind on the other pages.
- A block that is on one page only is placed opposite an empty space.
- Inside a changed block, the changed words or characters have a stronger highlight. The **Inline differences** option controls this, as in the text comparison.
- Tables have lines around their cells, and their rows are aligned. In a row that is different, the changed words are highlighted. If a row or a column is missing on the other page, its cells are highlighted completely.
- A cell that spans rows or columns (`rowspan`, `colspan`) is drawn across the cells it covers, so the other cells of its rows stay in their columns.
- Each changed block is a separate difference, even when it follows another changed block.

To move between differences, use the buttons above the pages, or the same keys as in the text comparison.

- The current difference has a stronger highlight, and the blocks being compared have an outline on every page. The outline is strongest on the page that you clicked last.
- Click a block to make it the current block. The next and previous differences are then found

from that block.

- The pages scroll only when the difference is not visible. The difference is then shown about one third from the top.

The location pane, to the right of the pages, shows all differences. The current difference appears in the accent color, and the visible part of the page has an outline. Click or drag in the location pane to go to that place. To show or hide the pane, choose **View → Text compare → Location pane**.

9.3. How pages are loaded

Every mode uses the HTML exactly as the server sends it, or as the file is saved. OxideMerge does not open the pages in a browser, so they open quickly. Because the scripts of the page do not run, content that scripts add is missing.

The Text and Formatted modes leave out content that the page hides: elements with `hidden`, `aria-hidden="true"`, or an inline `display: none` or `visibility: hidden` style. Content that only a style sheet hides is still compared.

OxideMerge decodes pages the same way as browsers do. It uses the first of these that it finds: a byte order mark, the charset in the HTTP `Content-Type` header, a `<meta>` charset, or a guess from the content.

- If the server returns a status code that is not 2xx, the pane header shows it, and the page that came with it is compared.
- OxideMerge reports a page, together with its address, if the page cannot be reached, takes more than 30 seconds, sends a response that is not valid, or is larger than 50 MB.

9.4. Resource tree limits

The Resource tree mode downloads only the resources that each page names itself. It does not follow links inside style sheets or scripts.

- Each page can have at most 400 resources, and each resource can be at most 20 MB.
- Resources that cannot be downloaded are counted in the status bar. The other resources are still compared.
- For your safety, a web page cannot load local files. Only a page on this computer or on a private network (such as `localhost` or `192.168.x.x`) can load resources from such an address, or redirect to one. Other resources of this kind are skipped and counted in the status bar.
- The addresses that you enter yourself are always loaded.

9.5. Comparing web pages in the console

You can also compare web pages from scripts and the terminal. The console comparison (`--cli` with HTML files, or with `-t web`) uses the last mode that you chose in the window:

- Formatted (the default) compares the Markdown of the pages line by line.
- HTML source compares the HTML.
- The other modes compare the visible text.



No mode runs the scripts of the page. Logins are not supported, so a page that needs a login can show its login page instead. No mode saves changes to a web server.

Chapter 10. Controlling which differences are shown

Not every change between two files matters to you. Reformatted code, timestamps in logs or generated headers can hide the few differences that you actually need to review. OxideMerge lets you decide what counts as a difference, so that a comparison shows only the changes that are relevant to your work.

Three kinds of settings decide what OxideMerge shows as a difference:

- Filters select which files and folders a folder comparison includes.
- Comparison options decide which text changes count, for example whitespace or case.
- Compare rules ignore or replace text that matches patterns.

These settings change only the comparison result. They never change the files.



If a change that you expect is not shown, check these settings before you decide that the files are identical.

10.1. Comparison options

Comparison options tell OxideMerge which kinds of text changes to treat as differences. **Compare** → **Compare options** has the most common options, and **Options** → **Compare** has all of them:

Whitespace

Compare: whitespace counts. **Ignore changes**: a different amount of whitespace does not count. **Ignore all**: whitespace does not count at all.

Ignore blank lines

Extra blank lines do not count.

Ignore case

Uppercase and lowercase letters are treated as the same.

Ignore carriage return differences (Windows/Unix/Mac)

The line endings `CRLF`, `LF` and `CR` are treated as the same.

Ignore numbers

Digits are ignored.

Ignore code page differences

Files in different encodings that contain the same text are treated as the same.

Ignore comment changes

Changes inside comments do not count. This works for languages whose comment syntax is known: C-like languages, scripts, SQL, HTML, XML, CSS, Lua, Haskell and more.

Format JSON and XML before comparing

JSON and XML are indented again, and JSON keys are sorted, so that layout changes do not count. The formatted view is read-only. Turn this option off to edit or merge the files.

Detect moved lines

Blocks of lines that moved are shown as moved.

Diff algorithm

Myers, **Patience** or **LCS**.

Inline differences

How changes inside a line are highlighted: **Off**, **Word level** or **Character level**.

Treat numbers as equal within

Numbers that are at most this far apart, for example `0.001`, are treated as the same. This is useful for measurements and generated data.

In folder comparisons, these options apply to text files when the method is **Full contents**. The other methods do not use them.

10.2. Compare rules

Compare rules handle noise that is specific to your files, such as timestamps, generated sections or license headers, by ignoring or rewriting text that matches regular expressions. A compare rules file (`.oxrules`) contains rules of three kinds:

Ignore lines matching

One regular expression on each line. A difference that contains only matching lines is ignored.

Substitutions

One rule on each line, written as `regex => replacement`. Before comparing, OxideMerge applies the substitutions to both sides in order, each rule to the text that the rules before it left; the changed words within a line are highlighted the same way. Use substitutions for text that is different in an expected way.

Ignore blocks between start and end lines

One block on each line, written as `start regex => end regex`. Differences inside the block are ignored. Use this for a generated section or a license header.

Patterns use the regular expression syntax of the Rust `regex` crate.

10.2.1. Managing rules files

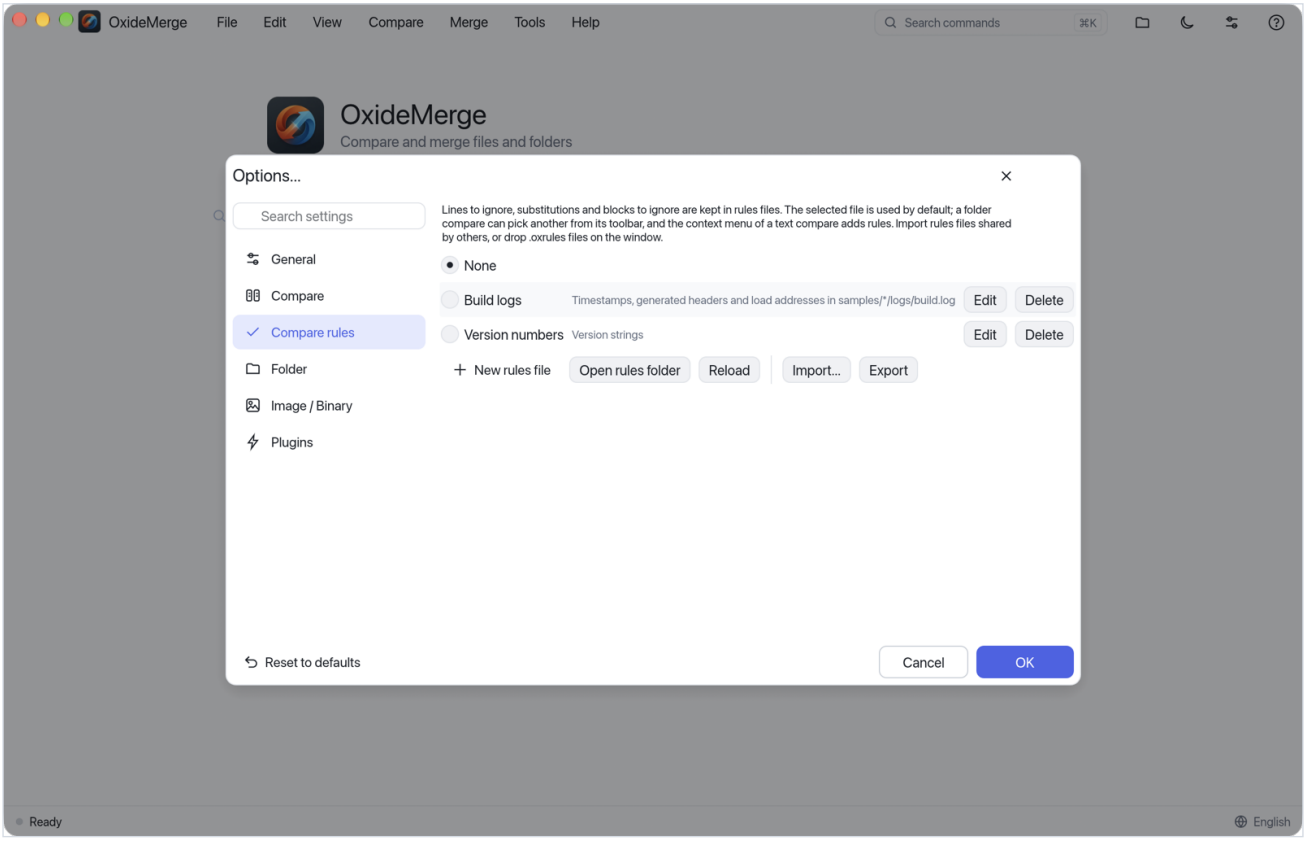


Figure 15. Options → Compare rules

You manage rules files in **Options** → **Compare rules**:

- Choose **None** or a rules file. This file is the default for comparisons that do not choose another rules file.
- **New rules file** creates a file. **Edit** opens a file in the rules editor, and **Delete** removes it.
- **Open rules folder** shows the folder in your file manager. After you change files there, click **Reload**.
- **Import...** adds rules files from other people. **Export** saves one file, or all files to a folder (**All rules files to a folder...**).

A folder comparison can use a different rules file. Choose it from **Rules** on the toolbar, or from **Compare** → **Compare rules**. **Refresh** (`F5`) reads the rules file again.

You can also import rules files by dropping them:

- Drop one `.oxrules` file on the window to import it. If you drop two, OxideMerge compares them instead.
- Drop any number of `.oxrules` files on the options dialog to import them.

When OxideMerge imports a rules file:

- If a file with the same name and the same rules exists, the new file is skipped.

- If a file with the same name but different rules exists, the new file gets a number in its name.
- If a file with the same rules but a different name exists, the new file is added with its own name.

10.2.2. The rules file format

Rules files are stored in the `Rules` folder next to the settings file (see Configuration files). OxideMerge also reads a `Rules` folder next to the program.

A rules file is a JSON file. The name of the rules is the file name without `.oxrules`. This is `samples/rules/Build logs.oxrules`:

Build logs.oxrules

```
{
  "description": "Timestamps, generated headers and load addresses in samples/*/logs/build.log",
  "ignore_lines": ["^// Generated on .*$"],
  "substitutions": [
    { "find": "^\\[\\d{2}:\\d{2}:\\d{2}\\] ", "replace": "[time] " },
    { "find": "\\b0x[0-9a-fA-F]{8,}\\b", "replace": "0xADDR" }
  ],
  "ignore_blocks": [{ "start": "^/\\* BEGIN LICENSE", "end": "END LICENSE \\*/$" }]
}
```

Every field is optional:

`description`
: Shown next to the name in **Options** → **Compare rules**.

`ignore_lines`
: A list of regular expressions. A difference whose lines all match one of them is ignored.

`substitutions`
: A list of `find` and `replace` pairs. Before the files are compared, each match of `find` is replaced by `replace` in both files, in the order of the list. The replacement can use the groups of the match: `$1` is the first group. The panes still show the text as it is in the files.

`ignore_blocks`
: A list of `start` and `end` pairs. A block starts at a line that matches `start` and ends at the next line that matches `end`; both of those lines belong to the block, and differences inside it are ignored. The end can also be later on the start line. A start without an end line is not a block, so nothing is ignored.

Each pattern is matched against one line at a time, so `^` and `$` are the start and the end of the line. In JSON, each backslash of a pattern is written twice: `\\d` in the file is `\d` in the pattern. The rules editor in **Options** → **Compare rules** shows the patterns without that doubling.

10.2.3. Example: build logs

This example shows how a small rules file reduces a noisy comparison to the one change that matters. `samples/left/logs/build.log` and `samples/right/logs/build.log` are two runs of the same build. Without rules, nearly every line differs, because the build host, the date, every timestamp and the load address are different each time.

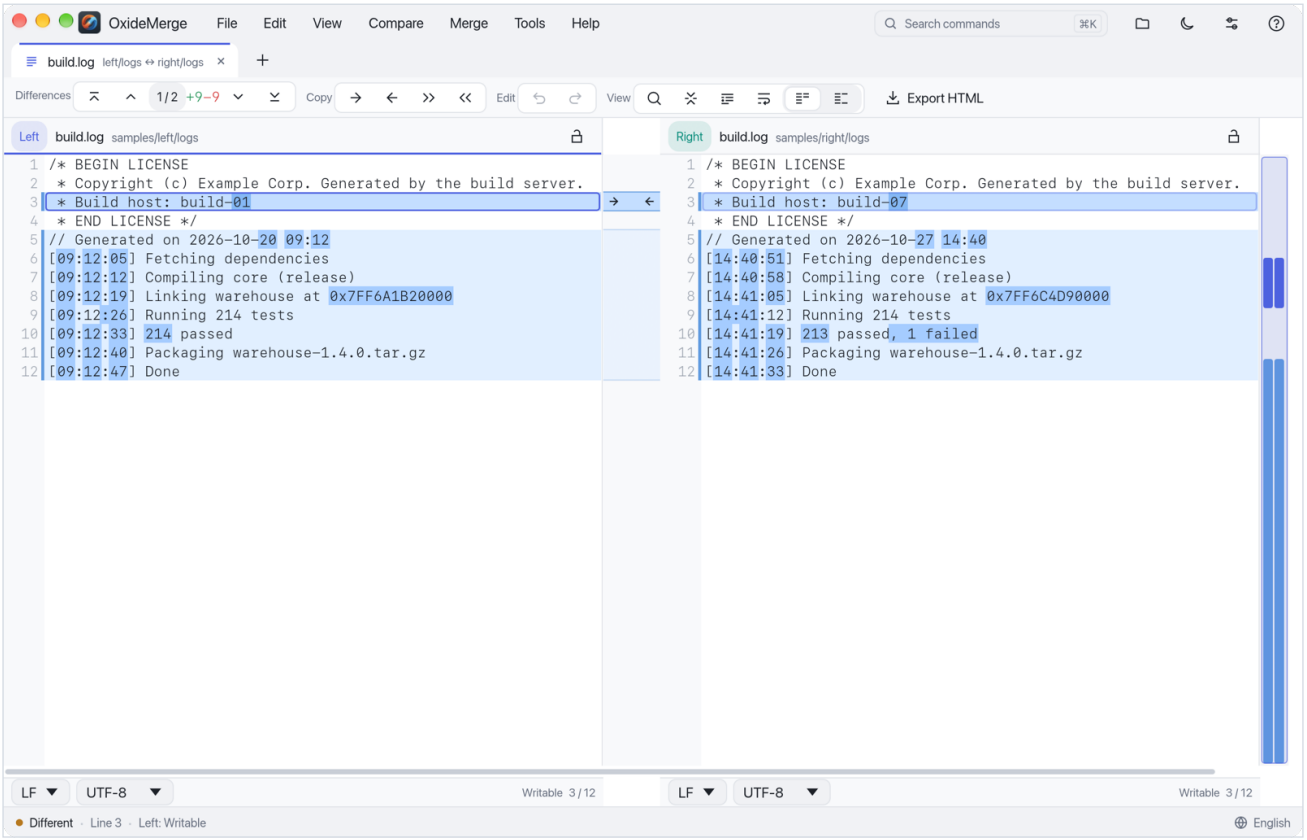


Figure 16. The two build logs without rules

To compare them with the **Build logs** rules, import `samples/rules/Build logs.oxrules` (**Options** → **Compare rules** → **Import…**), select it, and open the two files. Alternatively, choose the rules for one comparison on the command line:

```
oxidemerge --rules "Build logs" samples/left/logs/build.log samples/right/logs/build.log
```

`--rules` also accepts the path of the file, so the rules work without being imported: `--rules "samples/rules/Build logs.oxrules"`.

Each rule handles one kind of noise in the logs:

- The ignore block from `^/\* BEGIN LICENSE` to `END LICENSE \*/$` covers lines 1 to 4, the license header, so `Build host: build-01` and `build-07` no longer count.
- The ignored line `^// Generated on .*$` covers line 5, which holds the date of the run.
- The first substitution replaces the timestamp at the start of each line, such as `[09:12:05]`, with `[time]`. The pattern ends with the space after the timestamp, so the replacement does too.

- The second substitution replaces load addresses of 8 or more hexadecimal digits, such as `0x7FF6A1B20000`, with `0xADDR`, so line 8 is the same on both sides.

What remains is the one real difference: line 10, where one test failed. The ignored lines are shown in grey, and the difference counter shows **1/1**. The timestamps on line 10 are not highlighted, because the substitution makes them the same.

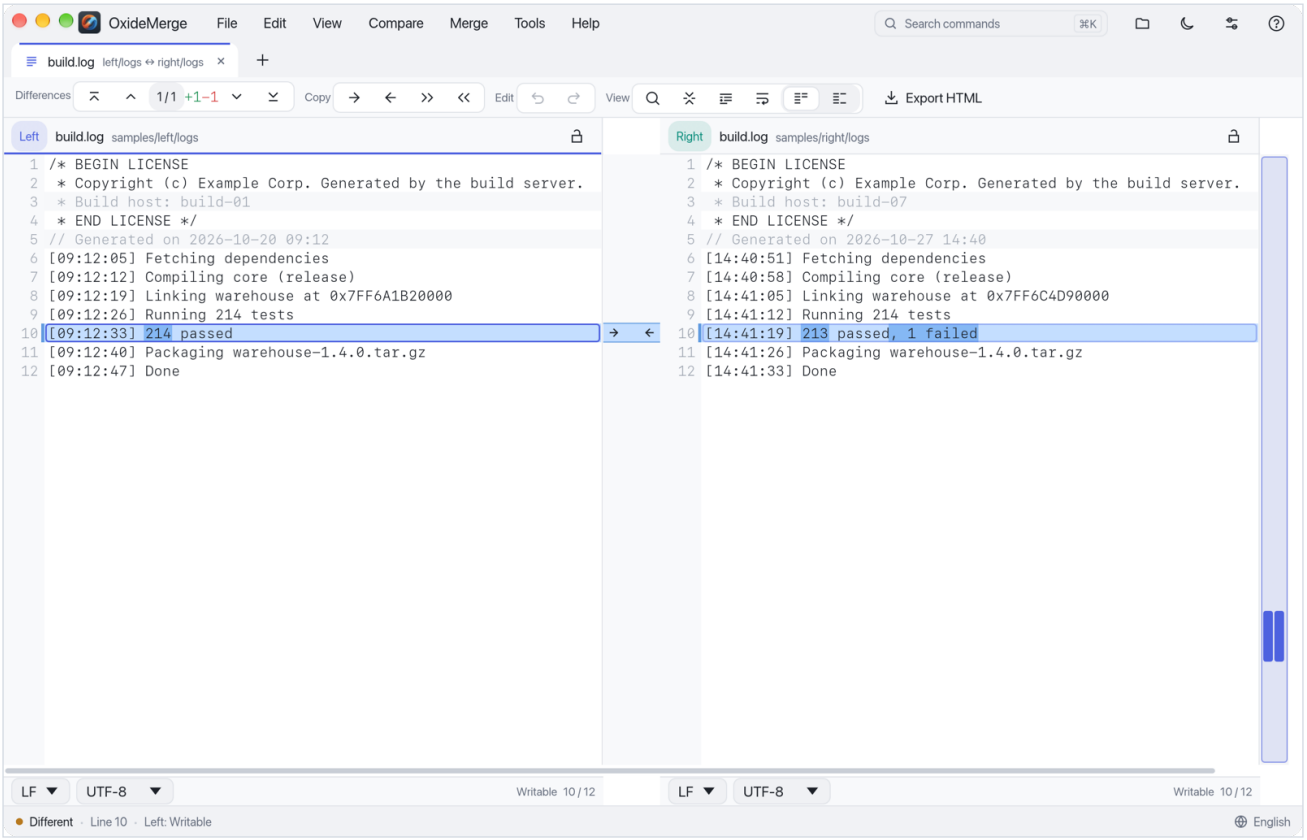


Figure 17. The same build logs with the Build logs rules

To see the same rules in the rules editor, one on each line, click **Edit** next to **Build logs** in **Options → Compare rules**. A substitution is written as `find => replace`. In the first substitution there are two spaces before ⇒, because the pattern ends with a space.

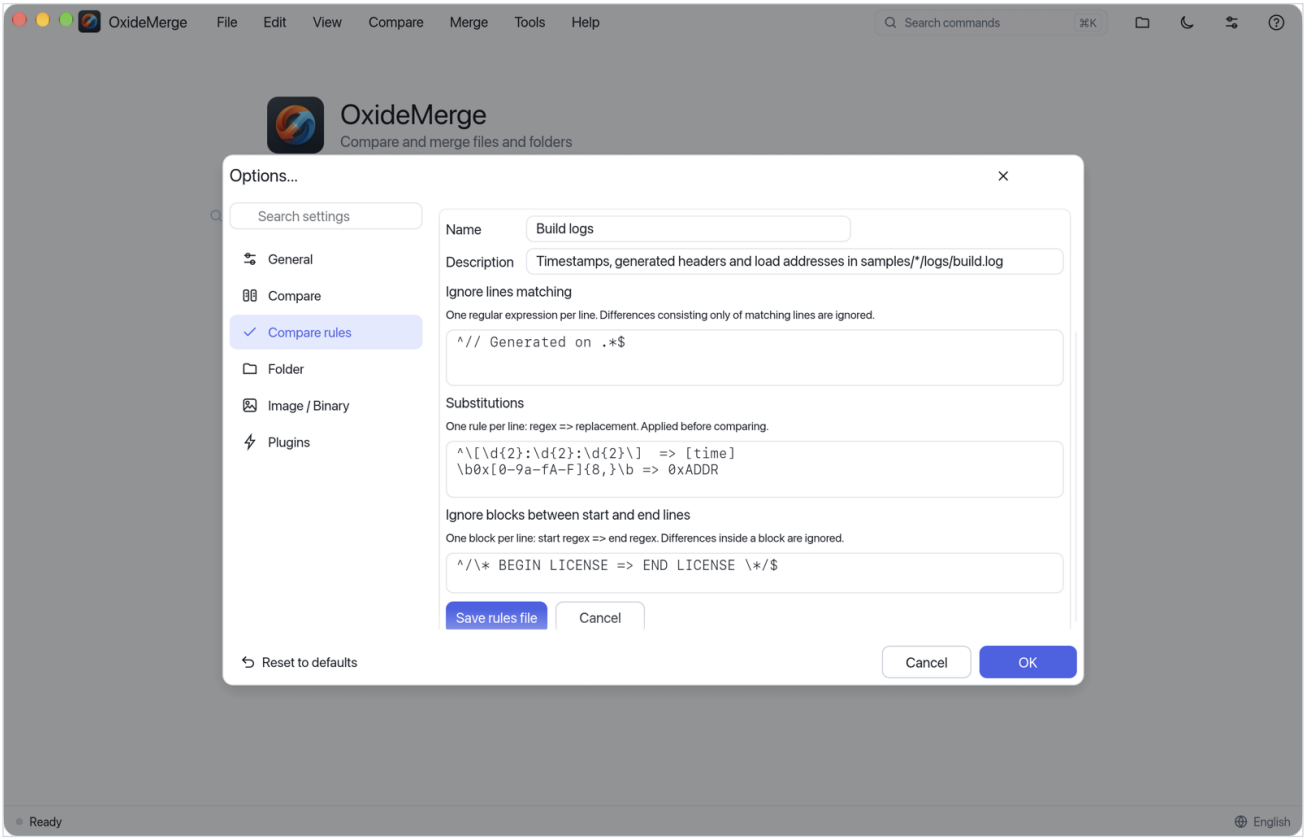


Figure 18. The Build logs rules in the rules editor

In scripts, the same rules decide the exit code. For these logs, `oxidemerge --cli --rules "samples/rules/Build logs.oxrules" ⋯` exits with `1` because of line 10. When the only differences are those that the rules ignore, it prints nothing and exits with `0`.

10.3. Creating a rule from a text comparison

Instead of writing a pattern from scratch, you can create a rule from a line in a text comparison and let OxideMerge suggest the pattern.

To create a rule from a text comparison:

1. In a text comparison, right-click a line and choose **Add compare rule**.
2. Choose the kind of rule:

   Ignore lines like this
   : A pattern that matches the line, or each side of a changed line.

   Substitute
   : A substitution that changes the text of one side into the text of the other side.

   Ignore block
   : A block that starts at this line.

3. Check the pattern that OxideMerge suggests, and change it if necessary.

4. Choose the **Rules file** for the rule, or enter a **File name** to create a new file.

The current comparison uses the rule at once. Other folder comparisons that use the same file use the rule after you refresh them (`F5`).

10.4. Filters

A filter decides which files and folders a folder comparison includes, so you can leave out build output, dependencies and other items that you do not need to compare. Choose a filter from **Filter** on the folder comparison toolbar. To manage filters, choose **Tools** → **Filters...**.

A filter mask is a list of wildcards, separated by `;`.

- A `!` at the start excludes the matching items.
- A `\` or `/` at the end matches folders.

For example, this mask includes Rust and TOML files, and excludes the `target` folder:

```
*.rs;*.toml;!target\
```

OxideMerge also lists WinMerge `.flt` filter files. It reads them from the `Filters` folder in the configuration folder, and from a `Filters` folder next to the program. **Tools** → **Filters...** shows these filters, opens the filters folder and creates new filters.

10.5. Unpacker plugins

An unpacker plugin extends OxideMerge to file formats that it cannot read as text itself. The plugin converts a file to text before the comparison, and OxideMerge compares the text that the program writes to its standard output.

OxideMerge reads PDF, `.docx`, `.odt`, `.rtf`, `.xlsx`, `.ods` and `.ipynb` files, and ELF programs, itself (see Documents, spreadsheets and programs). A plugin is therefore needed only for a format that it does not read, such as the older Word `.doc` and Excel `.xls` formats, or to read one of those formats better than OxideMerge does. A plugin whose pattern matches is always used in preference.

To add a plugin:

1. Install the conversion program, and make sure that the command can find it.
2. In **Options** → **Plugins**, add an entry with a **File pattern**, such as `.docx, and a *Command line`. In the command line, `${file}` is replaced by the path of the file, with quotes.
3. Turn on the entry.

The preset buttons add entries for PDF text (`pdftotext`), Office and OpenDocument files (`pandoc`) and general document extraction (`tika`). These entries start turned off. Move the pointer over the

checkbox of an entry to see which program it needs. This check looks only at the first program name of the command. If the command fails when it runs, OxideMerge shows the output of the converter.

For example, with Pandoc installed:

```
pandoc -t plain ${file}
```

Converters are used wherever files are compared as text: single files, folder comparisons with **Full contents**, the command line, the file compare pane, **All differences** and the differences in folder reports. The size, date, quick and binary folder methods still compare the original files.

The following limits and rules apply:

- The extracted text is read-only. It can never overwrite the original document.
- Each conversion can run for at most 30 seconds, and each output stream can be at most 16 MiB.
- If the program exits with a nonzero code, this is an error, even if it wrote some text.
- If you stop a folder scan, its conversions stop too.

Chapter 11. Saving, sharing and reporting

OxideMerge gives you several ways to keep, share and present the results of your work. You can save a comparison setup to reopen later, keep a complete snapshot for another reviewer, publish the differences as a report for people who do not use OxideMerge, or send the changes as a patch that other tools can apply. Choose the format according to what you need:

Format	File	Use it to
Project	`.oxmproj`	Open the same comparison again later. A project stores paths and settings, not file contents. See Projects.
Workspace	(kept in your settings)	Reopen a named set of tabs at once, alongside the tabs that are already open. See Workspaces.
Saved comparison	`.oxmcmp`	Keep the text of a text comparison, together with its bookmarks and comments. See Saved comparisons.
Folder review	`.oxmreview`	Keep a complete folder comparison, including file contents and review notes. See Folder reviews.
HTML report	`.html`	Share the differences with people who do not have OxideMerge. See HTML reports.
Other reports	`.csv`, `.txt`, `.xml`	Provide summaries for spreadsheets, text files and other programs. See Other reports.
Slideshow	`.html`	Present the differences one at a time in a meeting. See Slideshows.
Patch	`.patch`	Send the changes in a format that other tools can apply. See Patches.



Saved comparisons and folder reviews contain the full content of the files. Reports contain the differences and the lines around them. Check these files for confidential content before you share them.

11.1. Projects

A project lets you return to the same comparison later without selecting the files and options again.

File → Projects and jobs → Save as project... saves what the current tab compares in an `.oxmproj` file. A project stores:

- the paths, the descriptions and the read-only sides;
- the comparison type, the filter, **Include subfolders** and the compare method;
- the compare rules file.

To reopen a project, choose **File → Projects and jobs → Open project...**. OxideMerge can also open WinMerge projects (`.WinMerge`).

Paths are stored relative to the project file, so you can move a project together with its files. Paths on another Windows drive remain absolute, and web addresses are stored unchanged.

A project does not contain the files themselves. When you open it, OxideMerge loads the files that are at those paths at that time. To move a project to another computer, copy its files as well, together with any external rules files that it uses.



To share the text as it is now, together with your review notes, save a comparison instead.

11.2. Workspaces

A workspace saves a set of open comparison tabs under a name of your choice, so that you can reopen all of them at once whenever you need them.

To save the open tabs, choose **File → Workspaces → Save as workspace...** and enter a name of up to 80 characters. OxideMerge saves every open text, binary, image, web and folder comparison tab that has files of its own. The name, together with the paths and settings of the tabs, is kept in your settings rather than in a separate file.

Tabs that have nothing of their own to reopen from are left out, in the same way as they are left out of the session that OxideMerge reopens automatically:

- **All differences** and review tabs;
- the clipboard;
- tabs of temporary copies, such as a file or folder fetched over SFTP or FTP (see Comparing over SFTP or FTP), a revision, or a file opened from inside an archive comparison.

A message reports how many tabs were saved and how many were left out. If none of the open tabs can be saved, the dialog tells you so and nothing is saved.

Workspace names are not case-sensitive. If you save under the name of an existing workspace, OxideMerge tells you so and replaces its tabs only after you click **Replace**.

The **File** → **Workspaces** menu lists the saved workspaces (the first 15; **Manage workspaces…** lists them all). Choose a workspace to reopen all of its tabs at once, alongside the tabs that are already open. A tab whose files no longer exist is not reopened, and a message reports how many tabs could not be reopened.

Manage workspaces… lists the saved workspaces so that you can open or delete one. To restore a deleted workspace, click **Undo** in the message that follows the deletion.



A workspace differs from the single, unnamed session that OxideMerge reopens on its own when it starts. That session is kept and replaced automatically, whereas a workspace is kept, and reopened, only under the name you gave it. When two OxideMerge windows run at the same time, a workspace that you save or delete in one window stays saved or deleted when the other window saves its settings.

11.3. Recovering unsaved work

With **Keep unsaved work for recovery** turned on in **Options** → **General** (it is off by default), OxideMerge keeps a snapshot of its open comparisons while it runs: their paths and settings, the selected difference, the line of the text cursor, the bookmarks, the size and modification time of each compared file, and the whole text of every side that has edits you have not saved. The snapshot is written every 5 seconds while a side has unsaved edits, and every 30 seconds otherwise, to a private `recovery` folder beside the settings file (readable only by you on Linux and macOS), never next to the compared files. In portable mode, that folder is next to the program. Each running window has a snapshot of its own; it is deleted when the window closes normally, or when you turn the option off. Turning the option off, or starting with it off, also deletes the snapshots that windows which did not close normally left behind, without asking.

When OxideMerge starts after a window did not close normally (a crash, a forced quit, a power cut), it asks what to do with the snapshot left behind:

- **Restore** reopens the comparisons and puts the unsaved text back in their sides. The restored sides are marked as modified and are written to the files only when you save them. A message names every compared file that changed since the snapshot, so you can check the restored edits against it before saving. A comparison of the same files already open as the same kind, with no edits, takes the text rather than opening a second time. The unsaved text of a comparison whose files can no longer be opened (moved, deleted, or on a drive not connected) comes back in a new text compare, each side named after its file, to save where you choose. Until you save it, that text is kept in this window's snapshot like any unsaved edit, so it is offered again if this window does not close normally either.
- **Show details** lists the snapshot's comparisons, which of their files changed since, and how many sides have unsaved edits.

- **Discard** deletes the snapshot.
- **Later**, or `Esc`, keeps it, to ask again at the next start.

Limits:

- Only comparisons that can be reopened from their files are kept, as in the session that OxideMerge reopens on its own; tabs of temporary copies (files fetched over SFTP or FTP, revisions, files inside archives) are left out. Unsaved text is kept for text comparisons only.
- At most 32 MB of unsaved text is kept in one snapshot; sides past it are left out, and the details say how many.
- A snapshot is only as recent as its last write: edits made in the last few seconds before a crash can be missing.
- A running window holds a lock on its snapshot, so another window never offers it, however long ago it was written; once the window is gone, the next start offers it at once. Where the lock cannot be taken, a snapshot is taken as left behind once it has not been written for 2 minutes. At most five are kept, the oldest deleted first.
- A snapshot written by a later version of OxideMerge is shown but not restored.

To leave out one comparison, right-click its tab and turn off **Keep for recovery**: from the next snapshot on, it and its unsaved text are no longer written. Turn the option off for work whose unsaved text should not be kept on disk at all.

11.4. Saved comparisons

A saved comparison preserves the text of a text comparison together with your notes, so that you can return to it later or pass it on to another reviewer.

File → Save comparison... saves a text comparison in an `.oxmcmp` file. The file contains the text of each side, the bookmarks and comments, and the synchronization points.

You can open the file later or send it to another reviewer. It opens read-only and does not need the original files. A saved comparison is a snapshot of the text: unlike a project, it does not load the current files from their paths.

11.5. Folder reviews

A folder review packages a complete folder comparison, including the file contents and your review notes, into one portable file that a reviewer can open without the original files.

When a folder comparison has finished, **File → Save comparison...** saves it as an `.oxmreview` file. This portable bundle contains all items that the filter includes.

- To save only some items, select the files or folders, right-click, and choose **Save selected folder review...**. A selected folder includes all of its scanned contents.

- The **Show** settings and the chips change only what you see. They do not remove items from a complete bundle.
- Finish all file operations first. If the files have changed, refresh the comparison before you save.

To add notes, right-click a file or folder and choose **Toggle bookmark** or **Bookmark comment**. The name of a bookmarked item starts with *, and its comment appears when you move the pointer over the item. The notes are saved in the bundle and appear in HTML, XML and plain-text folder reports. CSV summaries do not include them. If you change the notes in an opened bundle, save the comparison again to keep your changes.

To open a bundle, use any of these methods:

- on the command line, run `oxidemerge review.oxmreview`;
- drop the file on the window;
- choose **File** → **Projects and jobs** → **Open project...**.

The bundle opens as a read-only snapshot that uses the comparison options it was saved with. It does not need the original files, named filters or external rules files. Converted documents are stored as their extracted text, and the converter commands are not stored, so the reviewer does not need the converter programs. In the tree view, expand the folders to see their contents.

Folder reviews have the following limits:

- Two or three sides, at most 10,000 items and at most 256 MiB of uncompressed content.
- OxideMerge refuses symbolic links, special files, paths that are not safe on other systems, and files that changed after the comparison.
- A bundle contains the full content of each file, including binary files.

The review workspace adds review decisions to folder reviews; see Review workspace.

11.6. HTML reports

An HTML report lets you share the differences with anyone who has a web browser, whether or not they use OxideMerge.

To create an HTML report:

1. Open a text or folder comparison, and wait until it has finished.
2. Check the comparison options, compare rules and folder filters, because they determine which differences the report shows.
3. Click **Export HTML** on the toolbar, or choose **File** → **Export** → **HTML report...** (`Ctrl` + `Shift` + `E`).
4. Choose the output file. The suggested name is made from the compared names, with _ in place of spaces.

5. OxideMerge opens the report in your browser. Check what it contains before you share it.

The report is a single HTML file that contains everything it needs. Recipients can open it in any browser, without OxideMerge.

- Each difference appears as a card with line numbers, three lines of context and the changed words highlighted.
- The report includes summary cards, an index, a side-by-side or unified view, light and dark themes, a status filter and search.
- **Previous** and **Next** in the header, or `P` and `N`, go to the previous and the next difference across files; a counter shows where you are, and the difference pulses briefly. A link to a difference, such as `report.html#f-3f9a0c12d4e7-d0`, opens its file and scrolls it into view below the header. A file's link is made from its path, not its place in the list, so it still points at the same file in the next report of the same folders. A text report numbers its differences: `#d0` is the first. The view you switch to keeps the difference you were reading in place.
- The search box searches file names and paths, or, with **Search diff content**, the text of the differences shown, whether their files are expanded or not. It shows how many files or differences match, and **Clear** empties it.
- The header carries the OxideMerge icon, and the page animates as it opens: the sections fade in and the counts in the summary cards count up. A browser set to reduce motion shows a still page, and so does printing.
- The report names the compare rules file and counts the ignored differences.
- On a narrow window or a phone, the index folds into a heading you tap to open, and the page fits the screen width.
- Without JavaScript, the report still shows every file, difference and link; only the buttons that need it are hidden.

A folder report includes the differences of each changed text file, and also:

- separate counts of the files with different contents, the files on one side only, and the differences exported or left out;
- an index of every changed or one-sided file; a file shown without its differences says **Summary only** and why, for example because it is binary or too long;
- how each file was compared, such as **Text**, **Spreadsheet cells**, **Document text** or the converter program, with the sheet and rows, the PDF pages, or the notebook cell, of each difference where the file has them, and with the fields that changed in each changed CSV, TSV or spreadsheet row, as in `Alice · score · 90 → 95`. A spreadsheet value reads as the sheet shows it, followed by the value it stores when that differs, as in `741,700 (741700)`;
- the differences of what OxideMerge or a converter reads from a changed binary file, such as the header and sections of an ELF program; when that text is the same, the report says the files differ in bytes it leaves out;
- a **Comparison settings** section with the compare method, file filter, options, line filters,

substitutions and report limits, and which files the folder view showed;

- a note of what the report leaves out or shortens.

11.6.1. How much a report shows

Every HTML report starts with a box that says whether it is complete. **This report shows every difference in full** means nothing was left out. Otherwise the box is headed **Not everything is in this report** and counts what is missing: files listed without their differences, diff blocks left out, and diff blocks shortened. Below that, it states the export policy the report was made with.

File → Export → HTML report detail chooses that policy (`--report-detail` on the command line, `report_detail` in a job file):

Detail	What the report shows	Size and time
Summary	The counts, and where each difference is, without its lines (up to 20,000 differences; the rest are counted). A folder report lists the files and their status, and reads none of them.	About the size of the file list. A folder report takes no longer than the comparison.
Standard (default)	Up to 20,000 rows of differences in all, 400 rows per diff block and 2,000 characters per line.	A few megabytes at most for the differences, however large the files.
Full	Every row of every difference, with whole lines.	Grows with the differences. A report of two large files that differ throughout can be as large as both files together, and slow for a browser to open.

Whatever the detail, a folder report shows the differences of files up to 16 MiB and 50,000 lines. Larger files are listed with the reason they are not shown, and are counted in the box. A folder report also stops showing differences once it holds 1 GiB of them: the rest of that file's diff blocks are counted, and the files after it are listed with that reason and not read. The detail applies to HTML reports. PDF reports have their own limit (see PDF reports and printing), and slideshows show up to 400 rows per difference.

A report is always a single HTML file. OxideMerge does not split a large report into several pages or files: a single file is what you can attach, archive and open anywhere. For a very large comparison, export **Standard** to share and **Full** for your own review, or narrow the comparison with filters first.

When a report is printed, the sides above the rows of a difference repeat at the top of each page, every file is expanded, and the light theme is used. The added, removed and changed rows are told apart by their colors, so on a monochrome printer the kind of each difference is in its heading, and the unified view's `+` and `-` signs show each row's side.

To share a folder report without the paths of your folders, turn on **File → Export → Hide folder paths in reports** (`--report-hide-paths` on the command line). The report then names the folders by their sides, and their paths appear nowhere in it, review notes and the review summary included. HTML reports include review notes. Use **Export review summary…** in the review panel to share the workspace summary as a separate JSON file. File names and the differences shown can still be sensitive, so check the report before you share it.

To save reports without opening them, turn off **File → Export → Open reports in the browser after saving**.



Exporting a report never saves your edits to the compared files.

11.7. Other reports

Besides HTML, you can save comparison results in formats for spreadsheets, text files and other programs. **File → Export → HTML report…** saves these formats as well: choose the file type in the save dialog.

- For a folder comparison: HTML (with the differences of each changed text file), CSV, plain text or XML. CSV and plain-text reports list the files with their status, sizes and dates.
- For a text comparison: HTML or XML.

Tools → Statistics… shows figures for the current comparison:

- for a folder comparison: the numbers of identical, different and unique files, the errors, and the total size of each side;
- for a text comparison: the identical, changed, added, removed and ignored lines, for each pair of sides.

11.7.1. XML reports

XML reports are intended for other programs, such as scripts, review tools or a build that checks what changed. To create one, choose the XML type in the save dialog of **File → Export → HTML report…**, or use `--report <name>.xml` on the command line.

- A text comparison report lists each difference with its kind, whether it is ignored, and the lines of each side with their numbers. The bookmarks and their comments come last.
- The kinds are `changed`, `added` and `removed`. Three-way comparisons also use `left-only`, `right-only`, `both-same`, `compatible` and `conflict`.
- A folder report lists each file and folder with its status, and its date and size on each side.

The following example shows a text comparison report:

```
<comparison format="1" tool="OxideMerge" version="0.9.8" type="text" created="…">
  <side index="0" role="left" name="a.txt"/>
  <side index="1" role="right" name="b.txt"/>
  <statistics differences="1" ignored="0" identical-lines="1" changed-lines="1" added-lines="0"
removed-lines="0"/>
  <differences>
    <difference index="1" kind="changed" ignored="false">
      <lines side="0" start="2" count="1"><line number="2">two</line></lines>
      <lines side="1" start="2" count="1"><line number="2">TWO</line></lines>
    </difference>
  </differences>
</comparison>
```

- Line numbers start at 1.
- If a side has no lines in a difference, its `count` is `0`, and `start` is the line that the text would go before.
- The `format` number changes only if the layout changes in a way that could break a program that reads it.

11.8. Slideshows

File → Export → Slideshow report… saves an HTML page that shows one difference at a time. Use it to walk through a change in a meeting or a review.

- The title slide names the compared sides and counts the differences.
- Each difference then has its own slide, with its lines, the lines around it, the changed words and any bookmark comments.
- For a folder comparison, there is a slide for each difference of each changed text file. Each slide says how the file was compared, such as **Text** or **Spreadsheet cells**, gives the sheet and rows, the PDF pages or the notebook cell of the difference, and lists the fields of a changed CSV, TSV or spreadsheet row.
- The title slide of a folder comparison counts the files as the HTML report does (see HTML reports), with the exported diff blocks, and says what the slideshow leaves out.
- The last slide of a folder comparison lists every changed or one-sided file. Click a file to go to its first slide; a file without slides says **Summary only** and why, such as a binary file. Below the list, **Comparison settings** shows the method, filter, options and limits.
- With **Hide folder paths in reports** on, the sides are named Left, Middle and Right.
- Ignored differences are not included.

In the browser:

- Press `→`, `←`, `Space`, `PgUp` or `PgDn` to go to another slide.
- Press `Home` or `End` to go to the first or the last slide.

- Press `F` to switch to full screen.
- Each slide has its own address, such as `#3` for the third slide, so you can link to it.
- When you print the page, each slide is printed on a page of its own.
- The header carries the OxideMerge icon, which links to the project page; the last slide links there as well.

11.9. PDF reports and printing

A PDF report gives you a printable document of a text or folder comparison, with the same content as the HTML report.

File → **Export** → **PDF report...** saves the report of a text or folder comparison as a PDF file. **File** → **Print...** (`Ctrl` + `P`) writes the same PDF to a temporary file that only you can read and opens it in your system's PDF viewer, from which you print it. The temporary file is deleted when OxideMerge closes. No browser is used.

A PDF report contains what the HTML report contains:

- The compared files, the compare rules file, the counts of differences and of added, removed and changed lines, and the bookmark comments.
- Every difference, side by side, with three lines of context, line numbers and the changed words marked. With the inline view switched on, a two-way report has a single column on portrait pages, with the removed lines above the added ones.
- For a folder comparison, the summary, the note of what the report leaves out, the comparison settings, the index of changed files and the list of all files with their status, sizes and dates, followed by a part for each changed file, the table of the files shown as a summary only, the review notes and the review summary.
- For a table compared by key, what matching by key found.

A PDF folder report has the folder report extras of the HTML report (see HTML reports):

- The summary counts the files with different contents, the files present on one side only, the exported diff blocks and any blocks left out.
- **Not everything is in this report** lists the files shown without their differences, the blocks left out by the size limit and the blocks shortened.
- **Comparison settings** gives the files in the report, the compare method, the file filter, the whitespace and ignore options, the filters and converters, the unchanged lines around each block (or **The whole file**) and the report limits.
- **File differences** lists every changed or one-sided file with the number of blocks drawn, or **Summary only** and why, and how the file was compared, such as **Text**, **Spreadsheet cells** or **ELF header and sections**. Click an entry to go to the part of its file.
- A file that shows no differences, such as a binary file, has no part of its own: these files are

listed together in the **Summary only** table after the parts, each with its status, how it was compared and why it shows none. Its entry in **File differences** goes to its row there.

- The part of each file starts with its status and how it was compared. Each difference names its sheet and rows, its PDF pages or its notebook cell, and a changed CSV, TSV or spreadsheet row lists the fields that changed, as in `Alice · score · 90 → 95`.
- With **Hide folder paths in reports** on, the sides are named Left, Middle and Right, and the report keeps the folder paths out of its notes and settings.

The report starts with its title and the time it was made, a panel that names each compared side (and the compare rules file), and the counts as a row of cards; on a page wide enough for both, up to five cards stand beside the panel. Under them, a line shows the sign and background of removed, added and changed lines (and of conflicts and ignored differences when the report has them) and the hatching of a line that is missing on one side. In the list of all files, each side has a column for the size, with the ends of the numbers in line, and one for the date. Each part has a heading; in a folder report, the settings are a table of labels and values, and the part of each file starts with its name beside a dot in the color of its status. The bar above each difference gives its kind, its lines on each side and its number.

Each page has a header with the OxideMerge icon, the report title and the names of the compared sides (not when the folder paths are hidden), and a footer with **Page n of m** and **OxideMerge**, which links to the project page (https://github.com/kuopinghsu/OxideMerge). The PDF contains a bookmark for each file and each difference. OxideMerge embeds the fonts it uses, with only the characters that the report needs, so Chinese, Japanese and Korean text prints as it appears on screen. Each column is labeled with its side, and on a page that a difference continues onto, with its file as well. In a folder report, the header of the file list is repeated on each page.

A report draws at most 20,000 lines of compared text (about 400 pages) for all files together; a long line that wraps counts once for each line it occupies. A difference shows at most 400 rows, unless the whole file is shown. When a difference is cut, at most 100 unchanged lines are kept above the change, so the change itself is always shown, and a line such as **… 1,200 more lines** states what was left out. Differences after the limit are only counted; a folder report gives their number as **Diff blocks left out**.

Lines read left to right, and Hebrew and Arabic text within them reads right to left, with the numbers and brackets in it kept in place (a date, phone number, percentage or price stays whole) and Arabic and Persian letters joined. This layout is simpler than a browser's, so a line that mixes the two directions in unusual ways can look different from the browser. Code fonts do not join `!=` or `→` into a single symbol. Hebrew, Arabic and Thai print with an installed font whose licence allows embedding a subset of it, such as SF Hebrew, Geeza Pro, Tahoma, Segoe UI or Noto. A character that none of the fonts contains prints as `?`; a note at the end of the report (and on the command line, on standard error) lists such characters by code point.

Color is never the only cue, so a report printed in black and white still shows what changed:

- Each changed line starts with a sign: `-` removed, `+` added, `~` changed, `!` conflict, `=` ignored.
- Changed words are underlined as well as highlighted.

- The position of a line that is missing on one side is hatched.
- When **Color-blind friendly diff colors** is on in **Options** → **General**, blue and orange take the place of green and red, changes are purple and conflicts are yellow. The dots and counts of the file statuses in a folder report follow: identical is blue, different orange, one side only purple and errors yellow.

Pages are A4 landscape. To use Letter pages, select **File** → **Export** → **US Letter pages for PDF reports**. On the command line, `--page-size`, `--portrait`, `--unified`, `--whole-file` and `--context` set the layout (see Command line):

```
oxidemerge old.c new.c --report changes.pdf
oxidemerge -r old new --report review.pdf --report-diffs --page-size letter
oxidemerge old.c new.c --report full.pdf --whole-file --unified --portrait
```

11.10. Patches

Patches let you send changes in a format that other tools can apply, and apply the patches that you receive.

File → **Export** → **Generate patch...** writes the differences between two files or folders as a patch. Choose the format (**Normal**, **Context** or **Unified**) and the number of context lines.

- You can create patches from two-way comparisons only.
- The ignore options still determine which differences are exported.
- Patches support UTF-8 files with or without a byte order mark, with LF, CRLF or mixed line endings, and files without a final newline.
- Files in other encodings, or with CR-only line endings, must be converted first. For these files, OxideMerge shows an error instead of creating a patch that would not reproduce the changes.

File → **Apply patch...** applies a unified diff, including the output of `git diff` and `svn diff`.

- OxideMerge finds each hunk even when its lines have moved. A hunk is one block of changes in the patch.
- Each patched file opens next to its original, so you can check the result and save it.
- Hunks that cannot be applied are reported. Read these messages and check the comparisons before you save. Applying a patch does not save any result automatically.

Chapter 12. Analysis tools

The analysis tools help you check a merge before and after you save it and review larger changes in an orderly way. Use them to confirm that a merged file matches its preview, to see what really changed in structured data files, to keep track of review decisions across a folder, and to summarize what changed between two releases.

OxideMerge provides these optional tools:

- Merge verification previews a merge and checks a saved result.
- Semantic comparison compares JSON, XML, TOML and YAML files by their structure, and Tree view shows two of them as one aligned tree.
- Review workspace records review decisions for each file and difference.
- Release summary / Compare baselines summarizes the changes between two releases.
- Potential impact lists the places where changed files may be used.

The analysis tools are read-only: they never change your files, and they do nothing until you open them. In the window, you find them in the side drawer: click **Side panel** at the right end of the toolbar, then choose a tab. You can also run them from the command line (see Running analyses from the command line).

For the exact rules of each analysis, see the analysis guide.

12.1. Merge verification

Merge verification lets you inspect the result of a merge before you save it and confirm afterwards that the saved file is what you previewed.

12.1.1. Text files

In a three-way text comparison, open the **Merge verification** tab.

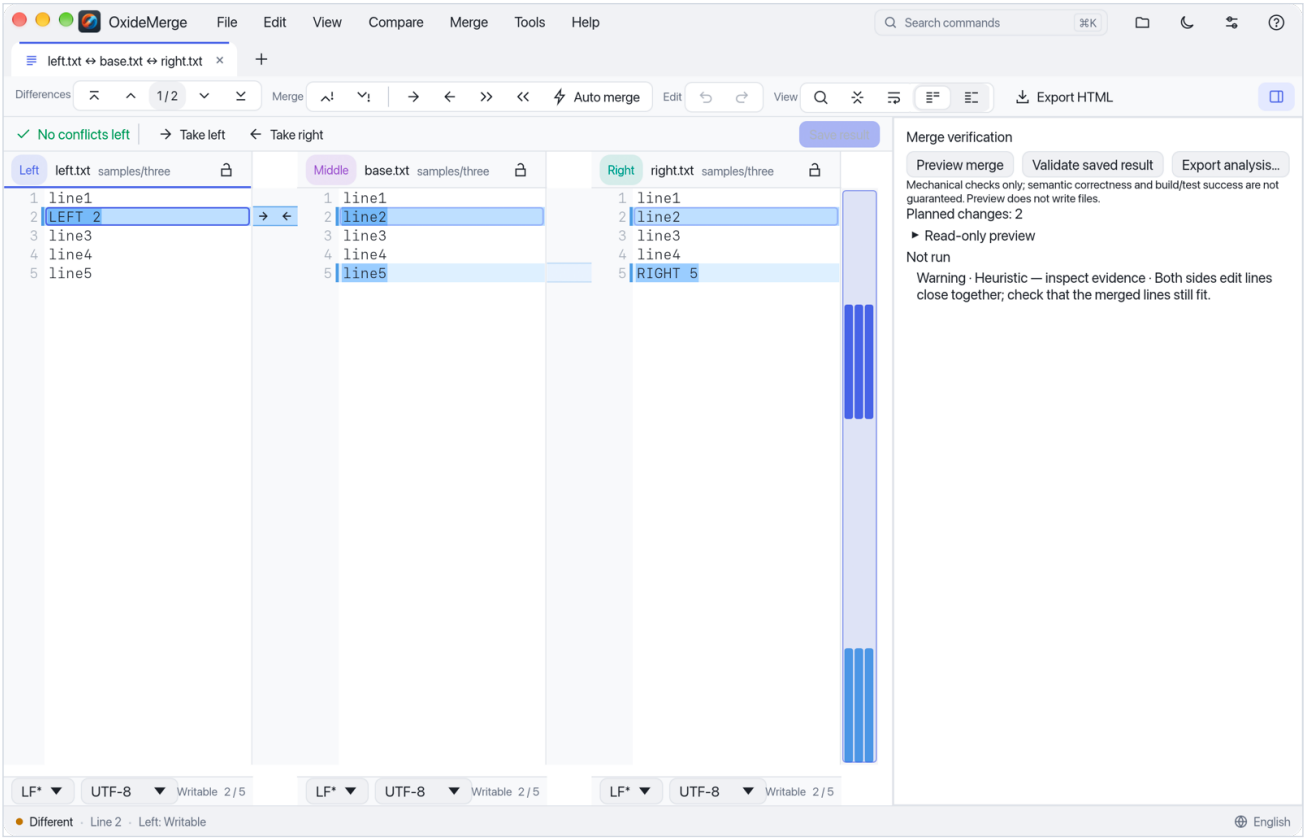


Figure 19. Merge verification after Preview merge

- **Preview merge** shows the result of the merge as a **Read-only preview**; the middle pane does not change. Conflicts keep the base text in the preview; you must resolve them yourself.
- **Validate saved result** reads the saved result file again and compares its bytes, encoding, byte order mark, line endings and final newline with the preview.
- If you edit the text, the validation status on display is no longer valid.

The preview also warns about merges that apply without conflicts but still call for a careful review:

- the two sides edit lines that are at most two lines apart;
- both sides change the same setting in different places, such as `name = value`, `name: value` or `"name": value`.

A warning does not cause the check to fail.



These checks compare text and bytes only. They never prove that the merged result is correct.

12.1.2. Folders

Two command-line options cover folder merges.

`--merge-plan` lists the operations and the conflicts of a folder merge without carrying them out.

`--validate-merge` checks a merge result against the plan. It checks the planned copies, merges and deletions as well as the files that stay unchanged, and it checks again that the source files did not change. The result can be in a separate folder:

```
oxidemerge --validate-merge result-folder left-folder base-folder right-folder
```



Work in this order: preview the merge, run the merge, then validate the result against the plan.

12.2. Semantic comparison

Semantic comparison lets you review structured data files by their content. For JSON, XML, TOML and YAML files, choose **Semantic** to compare the structure instead of the lines. The comparison itself never writes to the files; only the copy buttons described below change a pane. If OxideMerge cannot read a file, the view tells you why, and one click takes you back to the text view.

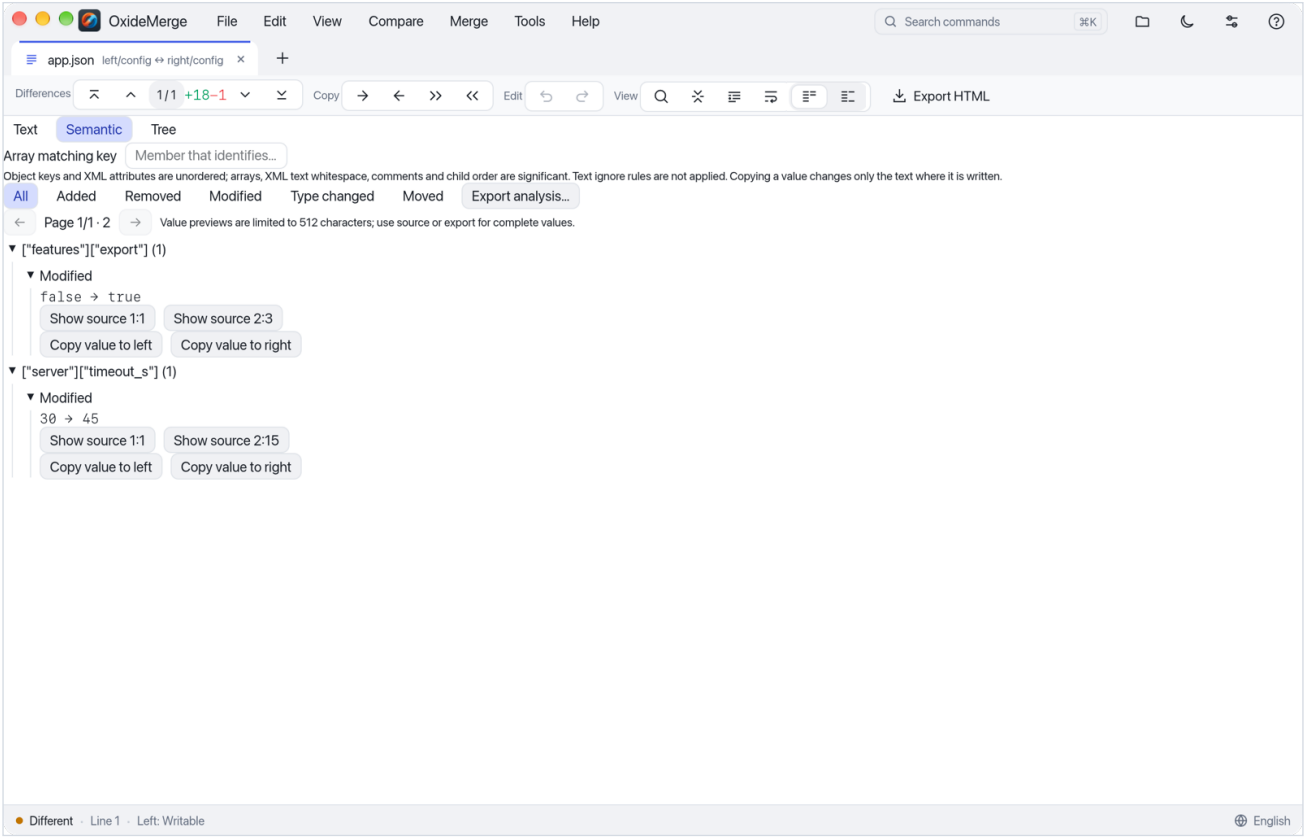


Figure 20. Semantic comparison of two JSON files

12.2.1. What counts as a change

JSON

- The order of object keys does not count.

- The order of array elements counts.
- A missing value and `null` are different.
- Duplicate keys are an error.

XML

- The order of attributes does not count.
- Names are compared together with their full namespace.
- The order of child elements, whitespace in text, mixed content and comments all count.
- DTDs, declared and unknown entities, and XML 1.1 are not supported. Built-in character references are supported.

TOML and YAML

- They are compared like JSON: the order of tables and mappings does not count.
- Numbers are compared by value, also between integers and decimals. For example, `1` equals `1.0`, and in TOML `8_080` equals `8080`.
- Comments and layout do not count.
- TOML date-times with a time zone are compared by the moment in time they name. For example, `10:00+02:00` equals `08:00Z`.
- YAML follows the YAML 1.2 core schema, so `yes` and `on` are text, and `0777` is the number 777, not an octal number as YAML 1.1 reads it. Keys are compared by value too, so `0x10:` and `16:` are the same key.

The following limits apply:

- A TOML or YAML file can be at most 4 MiB.
- A YAML file must contain a single document, without aliases, complex keys or custom tags. A core tag such as `!!int` must match its value.

The compare rules for text are not used as filters in the semantic view.

12.2.2. Reading the results

Changes appear as a tree of their paths, and each branch shows how many changes it contains. Click the source button of a change to go to its line in the file.

Arrays need particular attention:

- If an array has the same elements in a new order, the elements that moved are listed as **Moved**, together with their old position.
- To pair the objects of an array by a member instead of by position, type the member name, such as `id`, in **Array matching key** (JSON, TOML and YAML). An insertion or a new order is then shown as added and moved elements, rather than as a change at every later position.
- If the key cannot pair an array, because an element has no unique simple value for it, a note

above the changes says so, and that array is compared by position.

When array elements were not paired by path alone, each change states how they were paired: by the array matching key, as an exact new order, or by position in arrays of different lengths.

A change is marked **ambiguous** when another explanation fits equally well. Check these changes yourself. Typical cases:

- In a swap of two elements, either element could be the one that moved.
- A moved element has duplicates.
- An insertion shifted the elements, and the change was found by position.

For each change, reports record its `confidence` (`parser_exact`, `array_key`, `reordered` or `position`), `ambiguous` when it is set, and its `notes`.

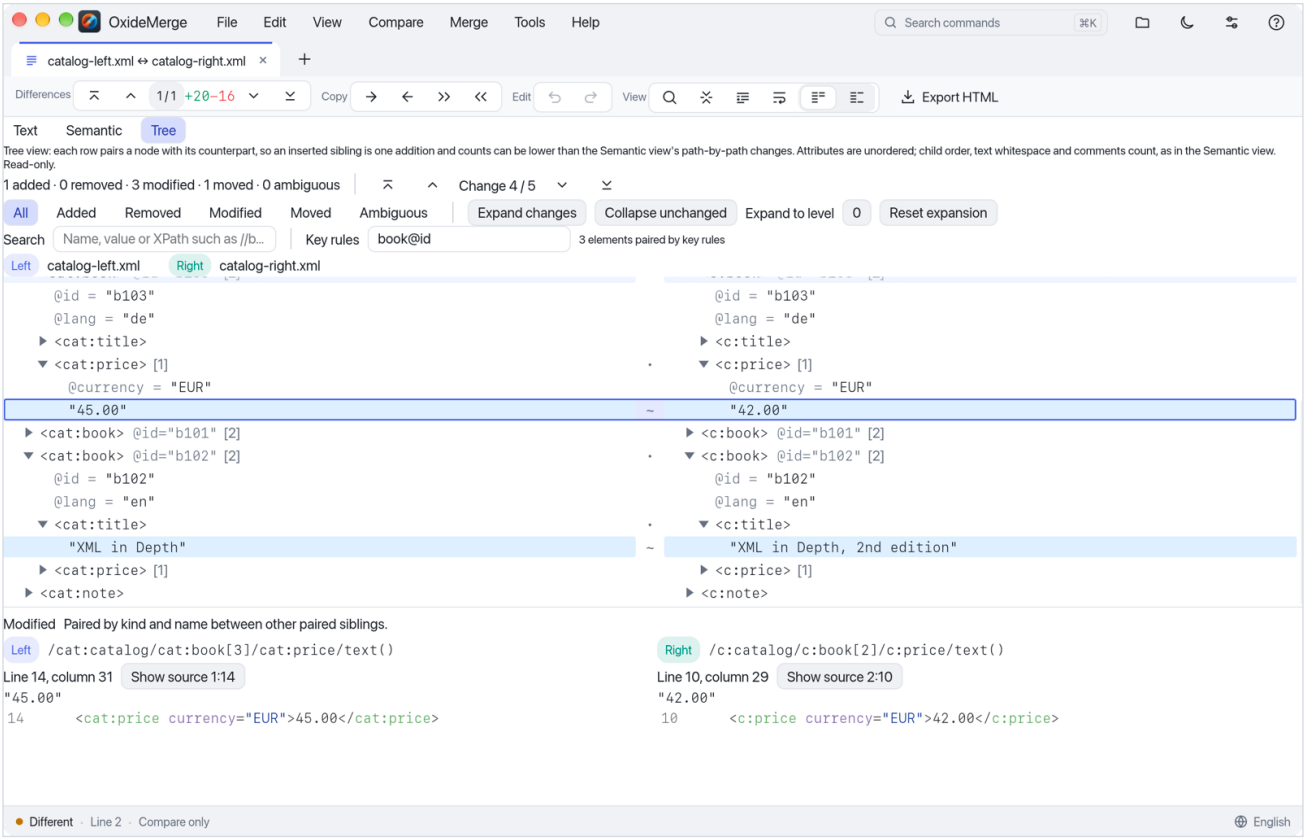
12.2.3. Copying values

Each change has **Copy value to left** and **Copy value to right** buttons for the panes that are not read-only. A copy replaces the value on the other side, adds it where it belongs, or removes it when the source side does not have it.

- Only the text where the value is written changes. Comments, layout and the rest of the file stay as they are. A removed member takes its own lines with it, including a comment at the end of them; a comment on a line of its own stays.
- The value is copied as the source side writes it: its quotes, the digits of a number (even one too long for a 64-bit number) and, for an XML element, its namespace prefix. An element whose prefix the other file does not declare is not copied, because it would get another name there.
- Before the copy is kept, the edited file is read again. If the result is not exactly the expected structure, nothing changes and the view tells you why. Edit such a value in the text view.
- Moves are not copied, because a move changes the order of the whole array.
- Each copy is one step of **Undo**. The file is written only when you save it.

12.2.4. Tree view

For two JSON, YAML, TOML or XML files, **Tree** shows both documents as one aligned tree. A node and its counterpart share a row, with the left file on the left and the right file on the right, so both sides scroll together. A node that exists on one side only has an empty placeholder row on the other side. The view only reads the files.



The column between the sides tells you what each row is, even without color: `+` added, `−` removed, `~` modified, `↕` moved, `•` changes below, and `?` ambiguous. A folded node with changes inside shows how many, such as **3 changes**. What a row shows depends on the format:

- **JSON, YAML and TOML**: a key and its value, such as `port: 8080`; an object or array with the number of its members, such as `env {2}` or `ports [2]`; and an array element by its position, with its `id`, `name` or `key` member, such as `[0] id="report"`.
- **XML**: elements by their name as written, with their `id`, `name`, `key` and `ref` attributes and any changed attributes as short labels. Expand an element to see its attributes, text, comments and processing instructions.

Rows are paired only below paired parents, so a node is never shown under another parent:

- Object members and XML attributes are paired by their key or name, whatever order they are written in.
- Array elements and XML child nodes are paired sibling by sibling:
  1. by a key rule that you give, or, for an array of objects that all have an `id`, `name` or `key` member with a different value, by that member (see **Key rules** below);
  2. as identical content, keeping the order;
  3. as identical content out of order, when that content occurs once on each side: a **Moved** row;
  4. by kind and name between the rows paired so far.

Inserting an element counts as one addition: the elements after it stay paired with their

counterparts. Because the Semantic view compares by position, it can list more changes for the same files. What counts as a change is the same as in the Semantic view:

- **JSON, YAML and TOML**: key order, comments, quoting and block or flow style do not count. In YAML and TOML, how a number is written (`8_080`, `0x1F90`, `1.0`) does not count either; in JSON, `1` and `1.0` differ. A value that changed type, such as a number that became a string, is shown as **Type changed**.
- **XML**: attribute order and namespace prefixes do not count, CDATA is text, and child order, whitespace in text and comments count. Rows of whitespace-only text that did not change are left out, and whitespace added or removed next to an added or removed element, such as its indentation, is part of that element.

Moving between changes

The previous and next difference commands (`Alt+↑` and `Alt+↓`, `Alt+Home` and `Alt+End`) and the buttons above the tree go through the changes in tree order. A change inside a folded node is not skipped: its parents unfold. **Change 3 / 12** shows where you are. The filters show and visit only added, removed, modified, moved or ambiguous changes, and the change counts on folded rows follow the filter.

Keyboard

As is usual for a tree, the tree is a single stop in the tab order: press `Tab` until it has an outline, or click a row. The arrow keys then move the selection: `↑` and `↓` from row to row, `→` to unfold a row, and `←` to fold it or to go to its parent. `Tab` leaves the tree rather than stepping to the next row, and `Esc` releases it. While the tree has the keyboard, a screen reader reads out the selected row: what it is, then each side. Every row is in the accessibility tree and can be read one by one, whether the tree has the keyboard or not.

Folding

Expand changes (also the starting state) unfolds every node with changes inside. **Collapse unchanged** folds the rest, **Expand to level** unfolds the tree to a given depth, and **Reset expansion** returns to the starting state.

Details and source

Below the tree, the selected row shows its path on each side (such as `$.server.port` or `/catalog/book[2]/@id`), how it was paired, its line and column, its value, and the lines of the file where it is written, exactly as they appear in the file. **Show source** opens the Text view at that line. If you choose **Tree** while the text cursor is in a node, including on the line of its key, that node is selected.

Search

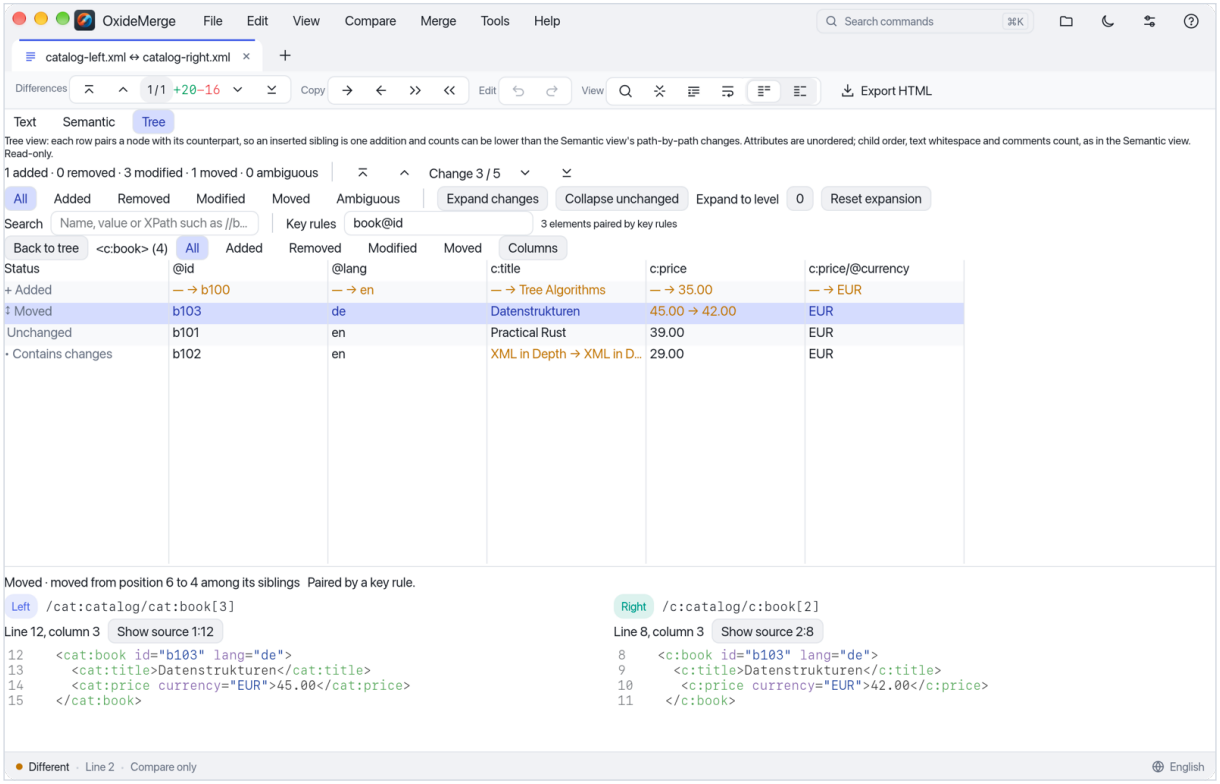
Type a key, name or value to show the matching rows together with their parents. In JSON, YAML and TOML, text that starts with `$` is a path, such as `$.server.port`, and shows that node and everything below it. In XML, text that starts with `/` is a small XPath subset: `/` and `//` steps of element names or `*`, `@name`, `text()` and `comment()`, with `[@a]`, `[@a='value']` and `[n]` conditions, such as `//book[@id='b102']/title`. Other XPath is refused with a message.

Key rules

Repeated nodes are paired by position unless they can be told apart. In JSON, YAML and TOML, the objects of an array that all have an `id`, `name` or `key` member with a different value on each side are paired by that member without a rule, and the toolbar says how many were, such as **3 objects paired by their id member**. Otherwise, write a rule in the form `name@key`. In JSON, YAML and TOML, `servers@name` pairs the objects of arrays named `servers` by their `name` member, `ports@name+protocol` pairs them by two members, and `spec/ports@name` applies only below `spec`; `*@id` covers every array, including a file that is a single array and an array directly inside another array, which has no name. In XML, `book@id` pairs `book` elements by their `id` attribute, `book@id+lang` pairs them by two attributes, and `catalog/book@id` applies only below `catalog`; a rule matches local names, but never pairs elements of different namespaces. Separate several rules with `;`; they take effect once you stop typing. Entries that swapped places are then shown as a move plus the content that changed, not as two replacements. A rule is used for a group of siblings only when every node it covers on both sides has a complete and unique key. Otherwise, a warning gives the number of duplicate and missing keys, and those nodes are paired as they would be without the rule and are marked ambiguous. Key rules belong to this view: they do not change the Semantic view or `--semantic` reports.

Collections

When the selected row holds an array of objects or repeated XML elements, **View as collection** lists them as a table with one row for each entry. Each row shows the status and the value on each side, in a column for each member that holds a single value, or for each attribute, each child element that holds only text, and each attribute of such a child (plus one column for the text when the entries hold only text). Choose the columns and filter by status; click a row to select it in the tree. Entries that hold deeper objects, arrays, elements or mixed content are flagged, because the table does not show that content.



Ambiguous rows

A row is marked ambiguous when another pairing fits as well: several siblings of the same name on one side with fewer on the other, a key rule that could not be used, an addition next to an identical sibling, or a swap where either node could be the one that moved. Check these rows yourself.

The Tree view is built in the background, with a progress display and **Cancel**, within the same limits as the Semantic view (100,000 nodes and 64 levels). It is available for two files; three-way comparisons use the Semantic view.

12.2.5. Three files

In a three-way comparison, **Semantic** compares the left and right files with the middle one. Each path that differs is listed once, by its path in the middle file:

- **Only left differs** or **Only right differs**: one side changed it.
- **Both differ the same way**: both sides made the same change.
- **Conflict**: the sides changed it differently, or one side changed a value inside a value that the other side changed. Changes to different members of one object do not conflict. When both sides changed an array whose elements could be paired in more than one way, such as arrays of different lengths paired by position, the whole array is one conflict.

Use the filters to show only the conflicts. **Use left** and **Use right** copy that side's value into the middle pane, in the same way as **Copy value**. The middle pane is compared as it is now: after you take a side, that path no longer differs from the side you took.

12.3. Review workspace

The review workspace keeps a record of your review of a folder comparison, file by file and difference by difference. The **Review workspace** tab of a folder comparison records:

- a state for each file and each difference (hunk);
- a **Review comment**, a **Rationale** and optional **Tags (comma-separated)**;
- a name in **Reviewer (free-form name)**. This is free text, not a verified identity.

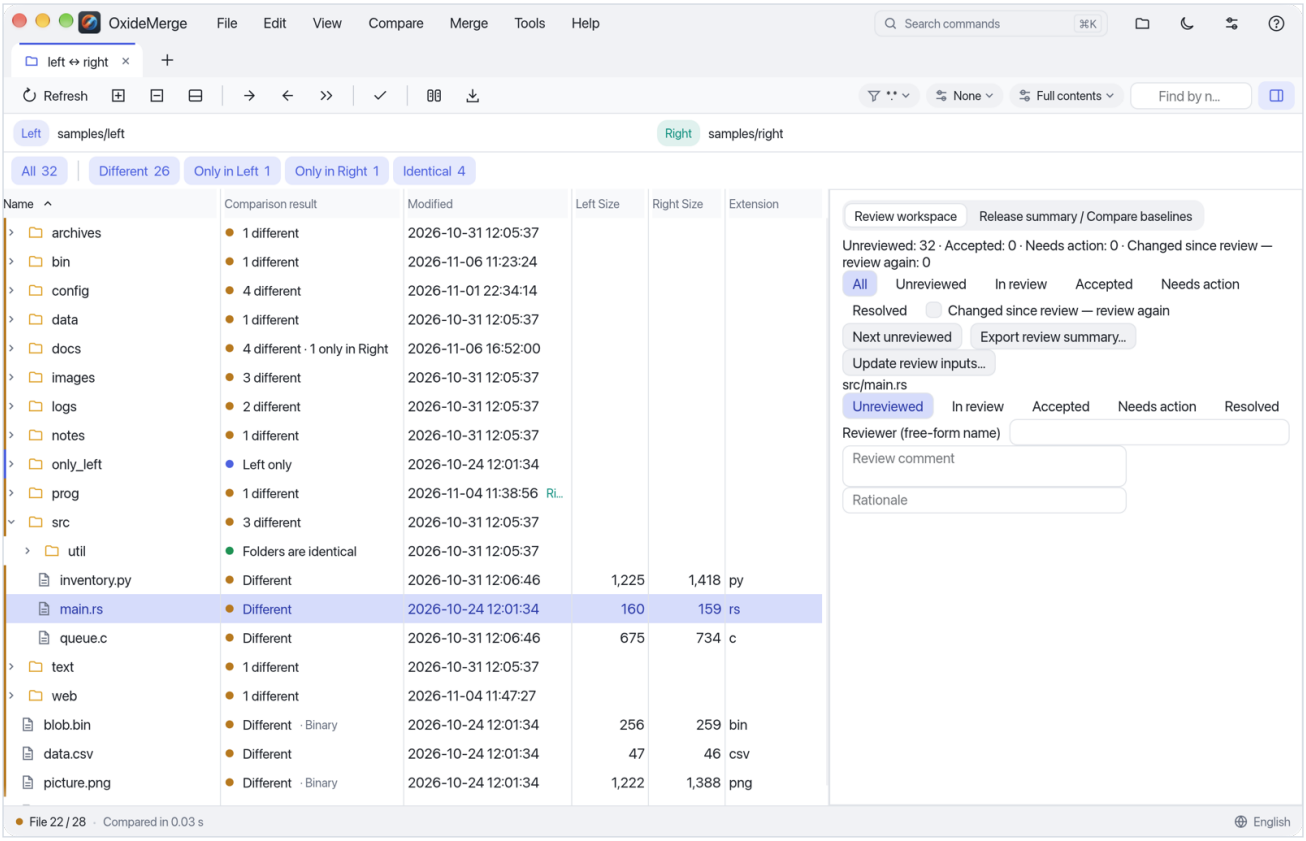


Figure 21. The review workspace

Next unreviewed takes you to the next file that still needs a review. To review single differences, open the file from its folder comparison.

To keep your decisions and your position, save the folder review (see Folder reviews). OxideMerge saves only after the comparison, the file operations and the check of earlier decisions have finished. It refuses to save after a comparison that was stopped or that failed.

12.3.1. Updating the review

Update review inputs... lets you choose new folders to review against your earlier decisions. It keeps the selected compare rules, opens the folders read-only, and marks the decisions that no longer match.

- A decision stays with a file that was renamed without changes, and notes the old name.

- If several new files have the same content, each one is marked for a new review.
- Changes to converted content, to converters or to rules can make earlier decisions invalid.
- A decision that is out of date counts as needing action. It never counts as accepted.
- Editing a reviewed file makes the file’s decision out of date, as well as the decision on each difference whose lines, or the two lines around them, the edit changed. Decisions on other differences remain.
- If the context of a difference repeats or is not clear, you must review that difference by hand.

12.3.2. File versions and safety

- OxideMerge saves `.oxmreview` files in version 2. It opens version 1 files with every item unreviewed, and refuses unknown versions.
- When you open a bundle, OxideMerge checks the digests of the stored files.
- OxideMerge refuses a bundle that has paths leading outside it, that has missing, unlisted or linked items, or that a newer version of OxideMerge saved.

12.4. Release summary / Compare baselines

The **Release summary / Compare baselines** panel compares two releases and gives you an overview of the changes between them. You find it on the start page and in the side drawer of a folder comparison.

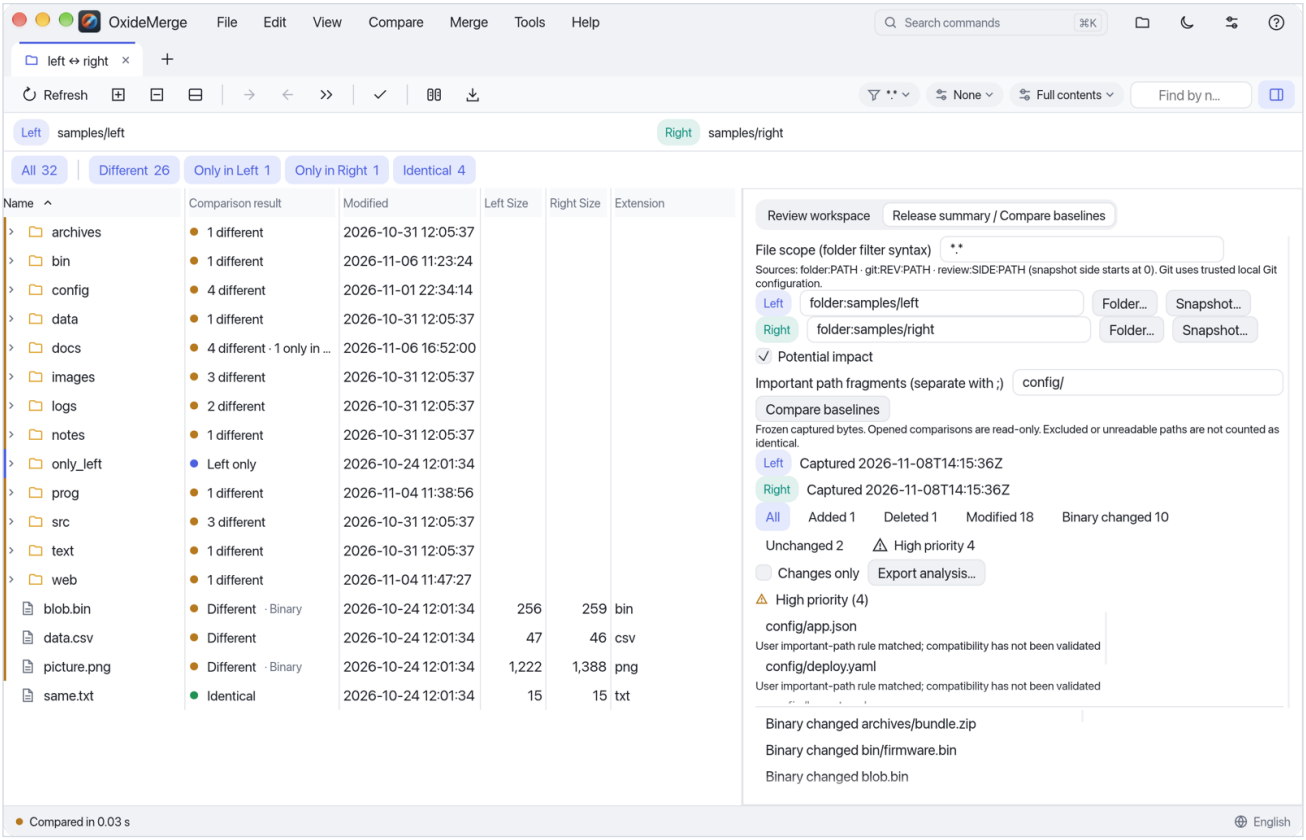


Figure 22. A release summary with potential impact turned on

Choose each side directly; you do not need to compare two folders first. Each side is one of these sources:

Source	Meaning
`folder:PATH`	The current content of a folder.
`git:REV:PATH`	A path in a Git revision. OxideMerge resolves the revision once and records its commit ID. Uncommitted changes are not included.
`review:SIDE:PATH`	One side of a saved folder review. The first side is `0`.

For Git sources, your Git configuration and filters must be trusted, as for Git revision comparisons.

12.4.1. The summary

The summary counts these kinds of changes:

- added, deleted, modified and unchanged files;
- moves of unique files with exactly the same content;
- suspected renames with edits;
- binary changes and changes of file type;
- semantic changes in JSON, XML, TOML and YAML files;

- excluded files.

It also shows when each side was captured.

- The counts above the list work as filters. Click one or more categories to show only those changes.
- **High priority** shows the changes that carry a risk flag: an important path, or conflict markers that are still in a file. These changes are also listed first.
- Select a row to open the captured files read-only. The files come from the baseline, even if the sources changed after the capture.

12.4.2. Renames

OxideMerge suspects a rename with edits when all of the following are true:

- A deleted file and an added file are text files of the same type.
- At least half of their lines, and at least three lines, are the same.
- They are clearly the best match for each other.

The row shows the old name and the share of matching lines.

- Click **Not a rename** to keep the two files as deleted and added. On the command line, use `--split-rename PATH`. OxideMerge warns you if the path is not an added file.
- Click **Detect renames again** to undo this.

12.4.3. Important paths

An important path is text that you want to watch, such as `config/` or `api`. A changed path that contains this text is flagged as high priority, and so is a file that was moved or renamed from such a path. Enter important paths in **Important path fragments (separate with ;)**. On the command line, use `--important-path TEXT`.

You can repeat `--important-path` and `--split-rename`. Both require `--impact`, `--release-report` or `--baseline-left`.



Important paths only help you decide what to review first. They do not guarantee compatibility.

12.5. Potential impact

Potential impact shows you where else a change may have an effect. Turn on **Potential impact** to list the places where the changed files may be used. Each finding shows the file, the line, a digest of the content and an excerpt.

12.5.1. Exact findings

A finding is **Exact** when an import names exactly a file that was changed, deleted or renamed. OxideMerge recognizes these imports:

- JavaScript and TypeScript: relative imports, re-exports and `require`. An import of `./util.js` also finds a changed `util.ts`.
- Python imports.
- C and C++ `#include "…"`, and Objective-C `#import "…"`.
- CSS `@import`.
- Rust `mod name;`, and `use` paths that start with `crate::`, `super::` or `self::`.
- JSON `$ref` file references.

For JavaScript, TypeScript, Python, C, C and Rust, OxideMerge reads the imports from a syntax tree built by Tree-sitter. A `.h` header is read as C, or as C when it can only be read as C++. For the other files, OxideMerge reads the tokens of the code. In Vue and Svelte components, only the `<script>` blocks are read. As a result, an import can span several lines, and text in a comment or a string is not taken as an import. If the parser cannot read part of a file, the file is listed as partly assessed, because some imports and definitions may have been missed.

OxideMerge also compares the top-level functions, types and constants of changed JavaScript, TypeScript, Python, C, C and Rust files. C and C global variables are not included. Rust attributes, such as `#[derive(…)]`, are part of the definition that follows them.

A file that imports a changed or removed definition by name gets an **Exact** finding at the first place where it uses the name. The finding states whether the definition changed or was removed. Imports under another name count too. Examples of such imports:

- `import { parse } from "./parser"`
- `from .parser import parse`
- `use crate::parser::parse`

12.5.2. Heuristic findings

Heuristic findings are possible uses that you should check:

- the first use of a changed definition name in other files of the same language;
- the name or path of a changed file in the text, as a separate word;
- changed JSON, TOML and YAML keys, also as dotted paths such as `db.timeout`.

If a large change produces more than 500 matches of names and keys, only the first 500 are listed, and a note above the findings gives the number of the others. The other kinds of findings are always complete.

12.5.3. What impact analysis does not find

- Imports that are conditional, generated or built by macros.
- Methods, and names used through a namespace, such as `ns.parse`.

OxideMerge never runs builds, tests or scripts of the repository.

12.5.4. Working with findings

- Click **Dismiss finding** to hide a finding. This does not change the files.
- In a folder comparison, **Review** opens the file of the finding in the review workspace.
- If there is no result, the panel says that the impact was not assessed.
- Changing a source clears the old findings.
- If you cancel the analysis, no partial result is shown.

12.6. Limits

The analysis tools work within the following limits:

- Each file can be at most 16 MiB.
- Each baseline can have at most 10,000 items and 256 MiB of captured content.
- Structured files can have at most 64 levels of nesting and 100,000 nodes.
- An impact analysis with more than 2,000 findings fails and asks you to make the scope smaller. Only the matches of names and keys are cut to fit.
- Files over these limits are refused or listed as excluded. They are never reported as identical.

Long analyses show their progress and can be canceled. Some steps, such as parsing one file or merging one file, check for cancellation only before and after they run, so a cancellation may wait until that step finishes. These limits do not apply to the normal comparisons.

12.7. Running analyses from the command line

You can run the analyses from the command line, for example:

```
oxidemerge --merge-plan left.txt base.txt right.txt --report merge-plan.json
oxidemerge --validate-merge merged.txt left.txt base.txt right.txt --report validation.json
oxidemerge --semantic --format json old.json new.json --report semantic.html
oxidemerge --semantic --array-key id old.yaml new.yaml
oxidemerge --semantic left.yaml base.yaml right.yaml --report conflicts.json
oxidemerge --review-summary review.oxmreview --report review-summary.json
oxidemerge --baseline-left git:v1.0:/repo --baseline-right git:v1.1:/repo --release-report release.html
oxidemerge --impact old-folder new-folder --report impact.json
```

Output:

- Without a report path, an analysis writes versioned JSON to the standard output.
- `.xml` reports, and the `.html` reports of most analyses, contain this JSON. Their format is separate from that of the comparison reports.
- An `.html` release report is a readable summary with the counts, the high-priority changes and a section for each change. It also contains the JSON.
- A report cannot replace an input file and cannot be inside a selected input folder.

Exit codes:

Code	Meaning
`0`	Identical, or the plan or summary was created successfully.
`1`	Different, a warning, conflicts left in a plan, or a review summary with decisions that are out of date.
`2`	An error, or the baseline scope is not complete. Files that a filter excludes on purpose are outside the scope, so they are not errors.
`3`	`--validate-merge` only: the check failed.

Other notes:

- `--format` accepts `json`, `xml`, `toml` and `yaml`. The default comes from the extension of the first file.
- Use one analysis option per run.
- `--semantic` only reports. To copy values, open the files and use the semantic view (see Semantic comparison).
- A three-file `--semantic` report lists each path with its `state` (`left`, `right`, `both` or `conflict`) and the changes from each side. It exits with `1` when any path differs, including conflicts.
- Source positions in semantic results are byte ranges of the decoded UTF-8 text and line numbers of the file. They are not offsets in a UTF-16 or legacy-encoded file.

Chapter 13. Command line

The `oxidemerge` command lets you start comparisons from a terminal or from another tool, and run comparisons, reports, patches and folder synchronizations without opening a window. Use it to open files and folders quickly, or to automate recurring work such as nightly reports and backups in scripts and job files. This chapter lists the options, explains the exit codes for scripting, and describes job files.

13.1. Opening comparisons

Run `oxidemerge` without arguments to open the start page. To compare files or folders, give two or three paths. To open a conflict file, a project or a saved comparison, give a single path. Enclose paths that contain spaces in quotes.

The examples use POSIX shell syntax. On Windows, use Git Bash, or adapt the examples to your shell.

```
oxidemerge left.txt right.txt                  # two-way text comparison
oxidemerge mine.txt base.txt theirs.txt        # three-way comparison (base in the middle)
oxidemerge -r old-folder new-folder            # include subfolders
oxidemerge -r -f "*.rs;!target/" old new       # folder comparison with a filter
oxidemerge conflict.txt                        # resolve a file with conflict markers
oxidemerge -t image a.png b.png                # choose the comparison type
oxidemerge https://example.com https://example.org   # web page comparison
oxidemerge left.txt right.txt -o merged.txt    # Save writes the right pane to merged.txt
oxidemerge release.oxmproj                     # open a saved project
oxidemerge -r old.tar.gz new.tar.gz            # compare archives as folders
oxidemerge --git-rev HEAD~1 src/main.rs        # compare with an earlier Git revision
oxidemerge review.oxmcmp                       # open a saved comparison
```

13.2. Options

The options are grouped by purpose in the tables below.

Table 1. Paths and sides

Option	Description
`PATHS…`	Two or three files, folders, web addresses or server paths (`sftp://`, `ftp://`, `ftpes://`, `ftps://`; see Server paths on the command line and in jobs), or a single conflict file, project file or saved comparison.
`-t`, `--compare-as <T>`	Sets the comparison type: `text`, `binary`, `image` or `web`.

Option	Description
`-o`, `--output <PATH>`	Sets where **Save** writes the result: the right pane in a two-way comparison, or the middle pane in a three-way comparison.
`--dl`, `--dm`, `--dr <TEXT>`	Shows these descriptions instead of the left, middle and right paths.
`-L`, `--label <TITLE>`	Sets a title for each side in turn, the way `diff -L` takes them (see Subversion, Mercurial and Perforce). `--dl`, `--dm` and `--dr` take precedence.
`--wl`, `--wm`, `--wr`	Opens the left, middle or right side read-only.
`--ssh-key <FILE>`	Specifies the private key used to sign in to `sftp://` paths (see Server paths on the command line and in jobs).
`--trust-fingerprint <FINGERPRINT>`	Trusts the server of a server path that is not trusted yet, if its key or certificate has this fingerprint, and remembers that server (see Server paths on the command line and in jobs).
`--proxy <URL>`	Connects to server paths through a SOCKS5 (`socks5://[user@]host:port`) or HTTP CONNECT (`http://[user@]host:port`) proxy. The proxy password is read from `OXIDEMERGE_PROXY_PASSWORD` (see Proxies, active mode and client certificates).
`--ftp-active`	Uses active mode for `ftp://`, `ftpes://` and `ftps://` paths (see Proxies, active mode and client certificates).
`--client-cert <FILE>`, `--client-key <FILE>`	Specifies the client certificate (PEM) to present to FTPS servers, and its private key if the key is not in the same file (see Proxies, active mode and client certificates).
`--rev`, `--git-rev <REV>`	Compares the single path given with its version in a Git, Subversion or Mercurial revision (see Comparing with a revision).

Table 2. Folder comparisons

Option	Description
`-r`, `--recursive`	Includes subfolders.
`-f`, `--filter <MASK>`	Applies a filter mask, such as `.c;.h` or `!*.bak`, or a named filter given as `"[F] name"`.
`-m`, `--method <M>`	Sets the compare method: `full`, `quick`, `binary`, `date`, `datesize` or `size`.

Table 3. Comparison options

Option	Description
`--ignore-ws <MODE>`	Sets how whitespace is treated: `compare`, `change` or `all`.
`--ignore-case`, `--ignore-blank`, `--ignore-eol`, `--ignore-numbers`	Ignores case, blank lines, line-ending differences or numbers, respectively.
`--ignore-comments`	Ignores changes in comments.
`--prettify`	Formats JSON and XML before comparing.
`--structured`	In folder comparisons, treats JSON, XML, TOML and YAML files that have the same structure as identical.
`--numeric-tolerance <AMOUNT>`	Treats numbers that are at most this far apart as equal.
`--rules <NAME>`	Selects the compare rules file: a name from the rules folder, or the path of an `.oxrules` file. `""` means no rules file.

Table 4. Reports and patches

Option	Description
`--cli`	Compares without opening a window. For the exit codes, see Scripting and exit codes.
`--report <FILE>`	Writes a report. Folder comparisons: `.html`, `.pdf`, `.csv`, `.txt` or `.xml`. Text comparisons: `.html`, `.pdf` or `.xml`. See PDF reports and printing.
`--report-diffs`	Adds the differences of each changed text file to an HTML or PDF folder report.
`--report-hide-paths`	Names the compared folders Left, Middle and Right in a folder report, rather than by their paths.
`--report-detail <DETAIL>`	How much of the differences an `.html` report shows: `summary`, `standard` (the default) or `full`. See How much a report shows.
`--slideshow`	Writes the `.html` report as a slideshow, with one difference per slide.
`--patch <FILE>`	Writes a patch for two files or folders.
`--patch-style <S>`	Sets the patch format: `normal`, `context` or `unified` (the default).
`--context <N>`	Sets the number of context lines in context and unified patches and in PDF reports. The default is 3.

Option	Description
`--page-size <SIZE>`	Sets the page size of a PDF report: `a4` (the default) or `letter`.
`--portrait`	Uses portrait pages in a PDF report. The default is landscape.
`--unified`	Uses a single column in a two-way PDF report, with removed lines shown above added lines.
`--whole-file`	Shows the whole files in a PDF report, rather than only the differences and their context.

Table 5. Analysis

Option	Description
`--merge-plan`	Writes a read-only merge plan for three files or folders (left, base, right).
`--validate-merge <RESULT>`	Checks a merge result against three files or folders (left, base, right).
`--semantic`	Compares two JSON, XML, TOML or YAML files by their structure. With three files (left, base, right), lists what each side changed from the base, together with the conflicts.
`--format <FORMAT>`	Sets the semantic format: `json`, `xml`, `toml` or `yaml`. Requires `--semantic`.
`--array-key <MEMBER>`	Pairs the objects in arrays by this member. Requires `--semantic`.
`--review-summary <SNAPSHOT>`	Summarizes the decisions in a saved `.oxmreview` folder review.
`--impact`	Lists where the changed files of two folders or baselines may be used.
`--baseline-left <SPEC>`, `--baseline-right <SPEC>`	Specifies release baselines: `folder:PATH`, `git:REV:PATH` or `review:SIDE:PATH`. Give both options instead of the paths.
`--release-report <FILE>`	Writes a release baseline report as `.json`, `.html` or `.xml`.
`--important-path <TEXT>`	Flags changed paths that contain this text. You can repeat this option. Requires `--impact`, `--release-report` or `--baseline-left`.
`--split-rename <PATH>`	Keeps this added path as added, rather than as a suspected rename. You can repeat this option. Requires `--impact`, `--release-report` or `--baseline-left`.

Option	Description
`--sync-plan <MODE>`	Writes a read-only plan to synchronize two folders (source, destination). `MODE` is `mirror` or `update`. The plan is written as JSON to the standard output, or to the `--report` file.
`--sync-links`	With `--sync-plan`, synchronizes symbolic links as links and never follows them (macOS and Linux). The plan records this choice, so `--sync-apply` needs no flag. Without this option, links are left alone.
`--sync-apply <PLAN>`	Applies a plan to the folders it was made for, given in the same order. Requires `--yes`.
`--yes`	Confirms that `--sync-apply`, or the sync jobs of `--job`, may replace and delete files in the destination, and that the jobs of `--job` may write reports and patches outside the job file's folder.
`--sync-recover`	Undoes the folder synchronizations that did not finish.
`--job <FILE>`	Runs the jobs of a job file in order and prints a JSON summary (see Jobs). Takes no paths.

Use only one analysis option per run. Analysis options cannot be combined with `-o`, `--patch`, `--git-rev`, `--slideshow` or `--report-diffs`.

Table 6. Other options

Option	Description
`--lang <CODE>`	Sets the interface language, such as `ja` or `zh-TW`.
`--list-languages`	Prints the available languages.
`--register-shell`, `--unregister-shell`	Adds or removes the file manager entries (see File manager).
`--select-left <PATH>`, `--select-middle <PATH>`	Remembers a side for a later `--compare-to`.
`--compare-to <PATH>`	Compares a path with the remembered left side (and middle side).
`-h`, `--help`	Lists the options. `-h` gives a short summary.
`-V`, `--version`	Prints the version.

On Windows, OxideMerge also accepts WinMerge-style switches, such as `/r`, `/f`, `/o`, `/dl`, `/dr`, `/wl` and `/wr`. Some WinMerge switches are ignored or not supported, so test your scripts before you rely on them.

13.3. Scripting and exit codes

Because OxideMerge can compare without a window, you can call it from scripts and act on the result through its exit code. `--cli`, `--report` and `--patch` run without a window and return these exit codes:

Code	Meaning
`0`	The content is identical under the current options and rules.
`1`	Differences were found. This code is also returned when a report or patch was written successfully.
`2`	An error occurred.



Exit code `1` is a comparison result, not an error. Make sure that your scripts treat it that way.

Folder synchronization (see Synchronizing two folders) uses its own exit codes:

Code	Meaning
`0`	`--sync-plan`: there is nothing to do. `--sync-apply`: every change was applied and checked. `--sync-recover`: everything was undone.
`1`	`--sync-plan`: the plan contains changes or conflicts.
`2`	An error occurred, or the plan was refused and nothing was changed, for example because a file changed after the plan was made.
`4`	`--sync-apply` failed or was stopped, and every change was undone.
`5`	Partly applied: some changes could not be undone. `--sync-recover` also returns this code when it cannot undo everything. The journals named in the output keep the original files.

```
oxidemerge --sync-plan mirror source destination --report plan.json
oxidemerge --sync-apply plan.json --yes source destination
```

`--sync-apply` compares the folders again and refuses a plan that no longer matches them, a plan made with another filter or for other folders, and a plan written by a newer version.

`--sync-apply` and `--sync-recover` print a JSON result with a `status`: `applied`, `not_applied`, `rolled_back`, `recovery_required` or `recovered`. When the result is not a plain success, it also has a `problem`: `stale` (a file or folder changed after the plan was made), `unsupported`, `read_failure`, `write_failure` (for example, no permission or not enough free space), `canceled` or `recovery_needed`. Together with a seventh name, `ambiguous` (another reading of the input is equally likely, as with repeated items that cannot be told apart), these problem names are used consistently throughout

OxideMerge. Any view that leads with the kind of problem before its details uses the same names, so a given situation is called the same thing wherever it appears.

Keep the following in mind when you script OxideMerge:

- The command line uses your saved options, unless you override them with options. For runs that must not depend on your settings, set `OXIDEMERGE_CONFIG_DIR` to a separate folder (see Configuration files).
- Reports are available for text and folder comparisons. For a binary or image comparison, OxideMerge prints a message instead of an HTML report.

```
# Compare without a window. The differences are printed too.
oxidemerge --cli -r dirA dirB

# HTML report of a folder comparison, with the differences of each changed file
oxidemerge -r dirA dirB --report report.html --report-diffs

# HTML report of two files, ignoring changes in comments
oxidemerge old.c new.c --ignore-comments --report diff.html

# Unified diff with 5 lines of context
oxidemerge old.c new.c --patch changes.patch --context 5

# Use a compare rules file
oxidemerge --cli -r build1 build2 --rules "Build logs"
```

In a shell that uses `set -e`, save the exit code explicitly so that differences do not stop the script:

```
result=0
oxidemerge --cli -r old-folder new-folder || result=$?
case "$result" in
  0) echo "No differences" ;;
  1) echo "Differences found" ;;
  *) echo "Comparison failed" >&2; exit "$result" ;;
esac
```

13.4. Jobs

A job file keeps a set of runs in one place, so that a nightly report or backup takes a single command. It is a JSON file, usually with the `.oxjob` extension:

```
{
  "version": 1,
  "continue_on_error": false,
  "jobs": [
    {
      "name": "nightly report",
      "left": "a",
      "right": "b",
```

```
      "recursive": true,
      "report": "report.html"
    },
    {
      "name": "plan",
      "kind": "sync_plan",
      "source": "a",
      "destination": "${env:BACKUP}",
      "mode": "mirror",
      "report": "plan.json"
    },
    {
      "name": "backup",
      "kind": "sync",
      "source": "a",
      "destination": "${env:BACKUP}",
      "mode": "mirror",
      "apply": true
    }
  ]
}
```

```
oxidemerge --job nightly.oxjob          # runs the first two jobs, refuses the sync
oxidemerge --job nightly.oxjob --yes    # runs all three
```

Each job is converted into the options of a command line and runs as that command line would, so a job and its command line give the same result. As on the command line, your saved options apply. `--job` takes no paths and no other options except `--yes` and `--lang`; the options of each job are in the file.

The file has these top-level fields:

`version`

Required; must be `1`. A file of a newer version is refused.

`continue_on_error`

When `false` (the default), the jobs after one that ends with an error (code 2 or higher) are not run. When `true`, every job runs.

`jobs`

The jobs, in order. A field that OxideMerge does not recognize is an error, so a misspelled name is never ignored.

Every job has an optional `name`, which appears in the summary and in error messages, and a `kind`:

`compare` **(the default)**

Requires `left` and `right` (and `base` for a three-way comparison). Options: `recursive`, `report`, `report_diffs`, `report_hide_paths`, `slideshow`, `patch`, `patch_style`, `context`, `compare_as`.

`sync_plan`

Writes a sync plan. Requires `source`, `destination`, `mode` (`mirror` or `update`) and `report` (the plan file).

`sync`

Plans and applies a synchronization in one step. Requires `source`, `destination`, `mode` and `"apply": true`; the run also needs `--yes`. A sync that would delete many items is refused unless the job has `"allow_large_deletion": true`. Both sync kinds accept `"copy_links": true` as the equivalent of `--sync-links`.

All kinds also accept `filter`, `method`, `rules`, `whitespace`, `ignore_case`, `ignore_blank`, `ignore_eol`, `ignore_numbers`, `ignore_comments`, `prettify`, `structured` and `numeric_tolerance`, and the compare kind also accepts `page_size`, `portrait`, `unified` and `whole_file` for a `.pdf` report. These fields have the same meanings as the command-line options of the same names. A field that does not fit the kind, such as `apply` in a compare job, is an error. For server paths, all kinds accept `ssh_key`, `trust_fingerprint`, `proxy`, `ftp_active`, `client_cert` and `client_key`, which correspond to `--ssh-key`, `--trust-fingerprint`, `--proxy`, `--ftp-active`, `--client-cert` and `--client-key` (see Server paths on the command line and in jobs). A `sync` job whose `destination` is a server path writes the changes back to the server.

Paths, including `rules`, are relative to the folder of the job file unless they are absolute paths, URLs or server paths. In paths and in `rules`, `${env:NAME}` is replaced with the environment variable `NAME`; an unset variable is an error. Nothing else is expanded, and a job file names no command and no shell. Comparing a file that one of your unpackers reads (see Unpacker plugins) runs that unpacker, as any comparison does.

A job writes its report or patch as a new file or over a plain file, never through a symbolic link. A report or patch outside the job file's folder, such as `../report.html` or a path under `${env:HOME}`, is written only with `--yes`, so a job file you receive from someone else cannot replace your files.

`--job` prints a JSON summary. Its `exit_code` is the worst of the jobs' exit codes, and it is also the exit code of the run. Each entry of `jobs` has the `name`, the `kind` and the `exit_code`, plus an `error` when the job failed and, for a sync job, the same `sync` result that `--sync-apply` prints. `not_run` lists the jobs skipped after a failure, and `stopped` is `true` when the run was stopped in the program:

```
{
  "exit_code": 2,
  "jobs": [
    { "name": "nightly report", "kind": "compare", "exit_code": 1 },
    { "name": "plan", "kind": "sync_plan", "exit_code": 1 },
    { "name": "backup", "kind": "sync", "exit_code": 2,
      "error": "This job changes … and runs only when allowed: --yes on the command line" }
  ]
}
```

13.4.1. Saving and running jobs in the program

You can also create and run job files from the OxideMerge window.

File → Projects and jobs → Save as job... saves the active folder comparison as a job file. The job records the compared folders, the filter, the compare method, the compare rules file and the comparison options that are turned on, so it compares exactly as the tab does. Choose what the job does:

- **Compare and write a report**: the report file is relative to the job file, and its extension (`.html`, `.pdf`, `.csv`, `.txt` or `.xml`) determines the format.
- **Plan a synchronization**: writes a JSON plan and changes nothing.
- **Synchronize**: plans and applies in one step, left to right or right to left, as a mirror or an update. The job has `"apply": true`; **Allow deleting many files** adds `"allow_large_deletion": true`.

Select **Store paths relative to the job file** if you want to move the job and its folders together. Only a comparison of two folders can be saved as a sync job. A comparison of a revision, a web page or an archive works on temporary copies, so it cannot be saved as a job.

File → Projects and jobs → Run job... runs a job file and lists how each job ended. Click **Open** to open the report or plan that a job wrote, or **Compare** to open its folders in a tab. **Open** is offered only for report files (`.html`, `.pdf`, `.csv`, `.txt`, `.xml`, `.json`, `.diff` or `.patch`). The jobs run as `--job` runs them: with your saved options, not the options that OxideMerge was started with.

Before it runs, a file that holds a sync job, or that writes a report or patch outside its folder, lists the folders it changes and the files it writes. A sync job that is allowed to delete many files is marked as such. The file runs only after you click **Run and allow changes**, which stands for `--yes` for this run only. What runs is the file as it was listed, even if it changes on disk in the meantime; **Run again** reads the file again and asks again. **Stop** stops a synchronization that is being applied, undoes what it changed, and skips the jobs after it. While a job is synchronizing, OxideMerge waits for it to finish before it quits.

Chapter 14. Integrating with other tools

Your version control system can open OxideMerge whenever you review changes or resolve a merge conflict, so that you do this work in the same panes that you use for every other comparison. This chapter shows how to set OxideMerge up as the diff and merge tool that Git, Subversion, Mercurial or Perforce run on their own, and how to add OxideMerge to your file manager. To open a revision from within OxideMerge instead, see Comparing with a revision.

14.1. Git

With a few configuration settings, Git opens OxideMerge for both `git difftool` and `git mergetool`.

Install Git and make sure that `oxidemerge` is in your `PATH`. Then run the following commands in a POSIX shell, such as Bash or Git Bash, to make OxideMerge the diff tool and the merge tool for your user account:

```
git config --global diff.tool oxidemerge
git config --global difftool.oxidemerge.cmd 'oxidemerge "$LOCAL" "$REMOTE"'
git config --global merge.tool oxidemerge
git config --global mergetool.oxidemerge.cmd \
  'oxidemerge "$LOCAL" "$BASE" "$REMOTE" -o "$MERGED" --dl Local --dr Remote --wl --wr'
git config --global mergetool.oxidemerge.trustExitCode false
```

On macOS, use the full path `/Applications/OxideMerge.app/Contents/MacOS/oxidemerge` unless you have linked the program into your `PATH`. On Windows, use the path of `oxidemerge.exe`.

`git difftool`

Opens each changed file in turn. `git difftool --dir-diff` opens a single folder comparison that contains all changed files. In a file, pressing `Alt` + `↓` after the last difference opens the next changed file.

`git mergetool`

Opens a three-way merge with the base in the middle. Use **Auto merge**, or copy differences to the middle, and then save. The middle pane is written to the merged file, never to the base. Files that OxideMerge would otherwise read as text, such as Jupyter notebooks, are merged here as the files they are, so the result that you save is still a valid file.

With `trustExitCode false`, Git checks whether the output file has changed. If the file appears unchanged, Git can ask you whether the merge was successful. Answer yes only after you have saved and checked the result. Closing the window alone does not mean that the merge was successful.

14.2. Subversion, Mercurial and Perforce

The settings in this section let Subversion, Mercurial and Perforce open OxideMerge for diffs and merges.

OxideMerge accepts the title arguments that `diff` takes: `-u` is ignored, and each `-L <TITLE>` names the next side, in the order of the paths. As a result, the pane headers show revision names rather than the paths of temporary files. `--dl`, `--dm` and `--dr` take precedence over `-L`.

A merge can name OxideMerge itself, with no wrapper. Subversion's `merge-tool-cmd` passes five files: the base, theirs, mine, the merged file and the working copy, the last two being the same file. Perforce's `P4MERGE` passes four: the base, theirs, yours and the result, which may not exist yet; all but yours are p4's temporary files, named like `t1t2t3.tmp` on Windows and `tmp.1.2.3` on macOS and Linux. Four or five other files, such as a pattern like `*.txt` that matched them, are refused as too many paths, so a save never overwrites one of them. When the command line contains only those files, OxideMerge opens mine or yours on the left, the base in the middle and theirs on the right, and saves the middle pane to the result. If that file is unchanged when the window closes, OxideMerge exits with code 1 and the conflict stays unresolved. Any other option, including `-o`, turns this recognition off, and the arguments are used as they were written. A wrapper that reorders the files and passes `-o` still works.

14.2.1. Subversion

`svn diff --diff-cmd oxidemerge` compares each changed file in turn. Subversion passes `-u -L <old title> -L <new title> <old> <new>`, so the headers show, for example, `f.txt (revision 12)` and `f.txt (working copy)`. To use OxideMerge for every `svn diff`, set it in `~/.subversion/config` (`%APPDATA%\Subversion\config` on Windows):

```
[helpers]
diff-cmd = oxidemerge
merge-tool-cmd = oxidemerge
```

Do not pass `-x` options to `svn diff`, because Subversion then passes them instead of `-u`. Give the full path of `oxidemerge` when it is not on your `PATH`.

At Subversion's conflict prompt, choose **l** (launch), or run `svn update --accept launch`. Subversion marks the conflict as resolved when OxideMerge exits with code 0, that is, when the merged file changed. Closing the window without saving leaves it unresolved.

14.2.2. Mercurial

Add the following to `~/.hgrc` (`%USERPROFILE%\mercurial.ini` on Windows):

```
[extensions]
extdiff =
```

```
[extdiff]
cmd.oxidemerge = oxidemerge

[diff-tools]
oxidemerge.diffargs = -L $plabel1 -L $clabel $parent $child

[ui]
merge = oxidemerge

[merge-tools]
oxidemerge.args = $local $base $other -o $output --dl $labellocal --dm $labelbase --dr $labelother --wl
--wr
oxidemerge.checkchanged = true
```

`hg oxidemerge` compares the changed files: a single file directly, several files as a folder comparison. `hg oxidemerge -c REV` shows one changeset. Do not also set `opts.oxidemerge`: Mercurial then ignores `diffargs`, and the titles are lost. `hg merge` and `hg resolve` open a three-way merge with the base in the middle. With `checkchanged`, Mercurial asks whether the merge was successful when the merged file has not changed.

14.2.3. Perforce

You can use OxideMerge from the p4 command line, from P4V, or from both. The two are set up separately: p4 reads `P4DIFF` and `P4MERGE`, while P4V keeps its own preferences.

Both require a working connection first. Run `p4 info` in your workspace: it should show your **User name**, your workspace as **Client name** and your server as **Server address**. If it reports **Client unknown** or cannot connect, first set `P4PORT`, `P4USER` and `P4CLIENT`, or the `P4CONFIG` file that your team uses.

The p4 command line

p4 runs the program named by `P4DIFF` with two files: the depot revision (in a temporary file) and your workspace file. It runs the program named by `P4MERGE` with four: base, theirs, yours and the result. OxideMerge takes that order when those are the only arguments.

To set up the p4 command line:

1. Tell p4 to use OxideMerge with `p4 set` (or with environment variables of the same names):

   ```
   p4 set P4DIFF=oxidemerge
   p4 set P4MERGE=oxidemerge
   ```

   If `oxidemerge` is not on your `PATH`, give its full path, for example `/Applications/OxideMerge.app/Contents/MacOS/oxidemerge` on macOS. Do not add `--cli` to `P4DIFF`: with it, OxideMerge prints the differences instead of opening a window, and exits with 1 when the files differ, which p4 on macOS reports as **Execution Failed**.

`p4 set` stores the values in `~/.p4enviro` on macOS and Linux, and in the registry on Windows.

2. Check the settings with `p4 set`; both lines should name OxideMerge:

```
P4DIFF=oxidemerge (enviro)
P4MERGE=oxidemerge (enviro)
```

The text in parentheses shows where p4 found each value: `(enviro)` for `p4 set` on macOS and Linux, `(set)` for `p4 set` on Windows, and `(config '…')` for a `P4CONFIG` file. A value with nothing after it comes from an environment variable, which takes precedence over `p4 set`. If a line names another tool, change the value where it is set.

To compare a file with the depot, run `p4 diff f.txt`. OxideMerge opens with the depot revision on the left and your workspace file on the right. The depot revision is a temporary file named like `tmp.1.2.3`. Only your workspace file can be edited. Because p4 passes the words in `P4DIFF` on to OxideMerge, you can give the panes names instead:

```
p4 set P4DIFF="oxidemerge --dl Depot --dr Workspace"
```

p4 splits `P4DIFF` at spaces, so keep each title to one word.

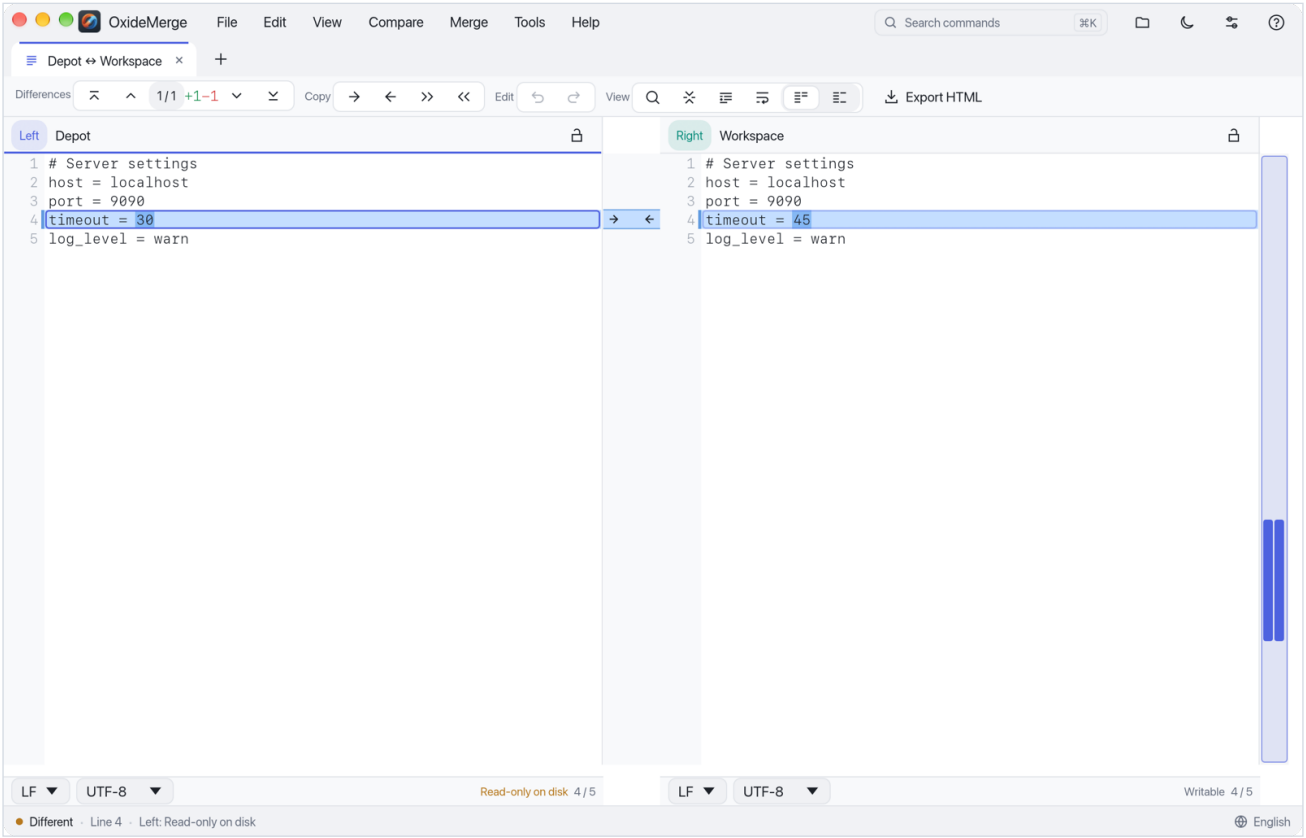


Figure 23. `p4 diff` with the panes named Depot and Workspace

To merge a conflict:

1. Run `p4 resolve f.txt`. p4 shows what it found and suggests an action:

```
/home/you/ws/f.txt - merging //depot/main/f.txt#2
Diff chunks: 0 yours + 1 theirs + 0 both + 1 conflicting
Accept(a) Edit(e) Diff(d) Merge (m) Skip(s) Help(?) e:
```

2. At the prompt, type **m** (merge). OxideMerge opens with **Yours** on the left, **Base** in the middle and **Theirs** on the right. The middle pane starts as the base, and **Save result** writes it to the result file that p4 passed, never to the base.
3. Resolve each conflict, either with **Next conflict** followed by **Take left**, **Take right** or **Take both**, or by editing the middle pane. Check that the toolbar says **No conflicts left**: **Save result** also saves a result that still has conflicts, with the base text in their place.
4. Click **Save result** and close the window.
5. Back at the prompt, p4 suggests **ae** (accept edit), or **at** (accept theirs) when the result is the same as theirs. Type **ae** to accept the merged file as you saved it. p4 then reports, for example, `//ws/f.txt - edit from //depot/main/f.txt`.

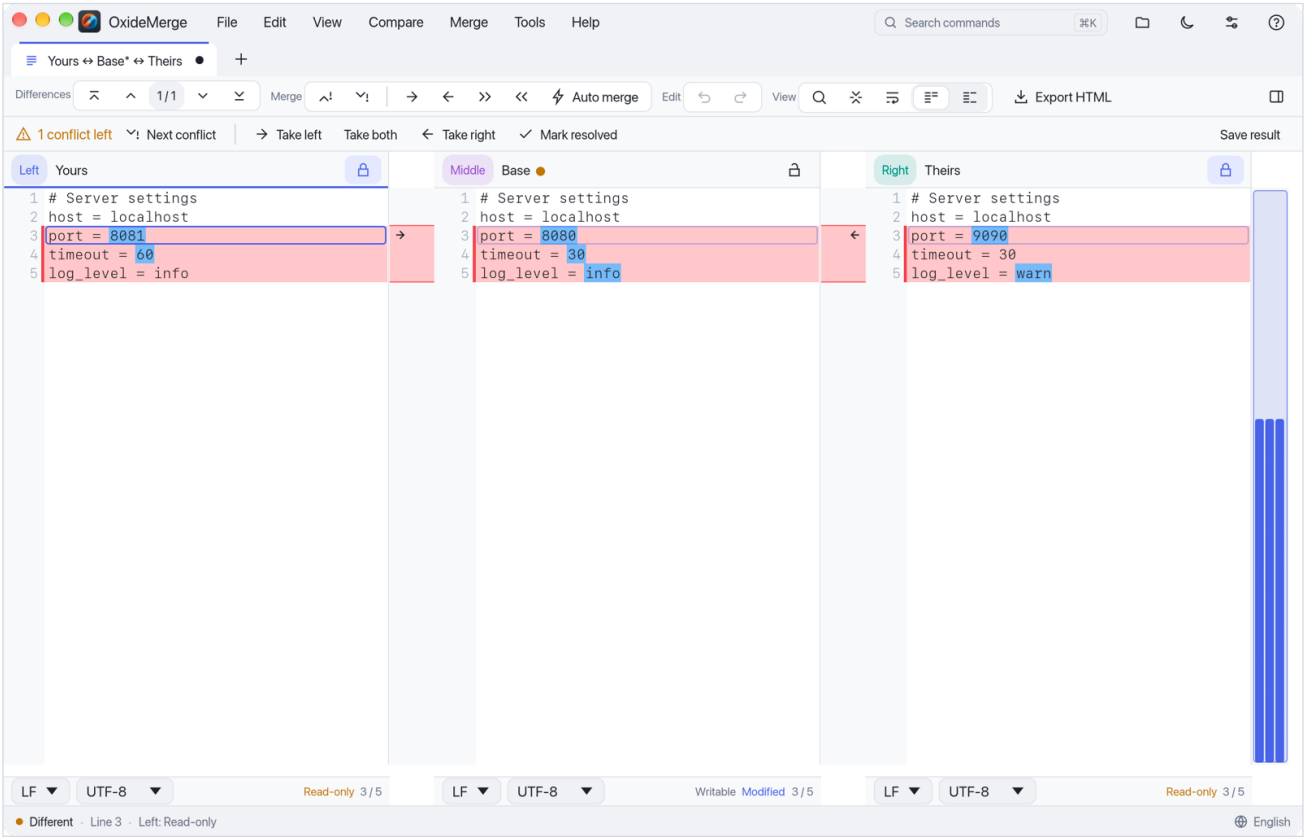


Figure 24. A conflict opened by `p4 resolve`

If you close the window without saving, OxideMerge exits with code 1 and the file stays unresolved: p4 suggests **e** rather than **ae** (on macOS and Linux it also reports **Execution Failed**; Linux then says **No such file or directory**, which is how it reports that exit code), and typing **ae** warns that the file still has change markers. Run `p4 resolve` again to start over.

P4V

P4V does not read `P4DIFF` or `P4MERGE`; it has its own diff and merge settings. Because P4V runs OxideMerge directly, no wrapper is needed.

To set up P4V:

1. In P4V, open **Preferences** (**P4V** → **Preferences** on macOS, **Edit** → **Preferences** on Windows and Linux).
2. Select **Diff** in the list on the left.
3. Select **Other application**. Filling in the fields below it is not enough: as long as **P4Merge** stays selected, P4V keeps using P4Merge.
4. In **Location**, enter the path of the `oxidemerge` program itself, not the path of the app or of a wrapper:

macOS	`/Applications/OxideMerge.app/Contents/MacOS/oxidemerge`
Windows	the path of `oxidemerge.exe` in the folder that you extracted it to
Linux	the output of `command -v oxidemerge`, for example `/usr/bin/oxidemerge`

5. In **Arguments**, type `%1 %2`.

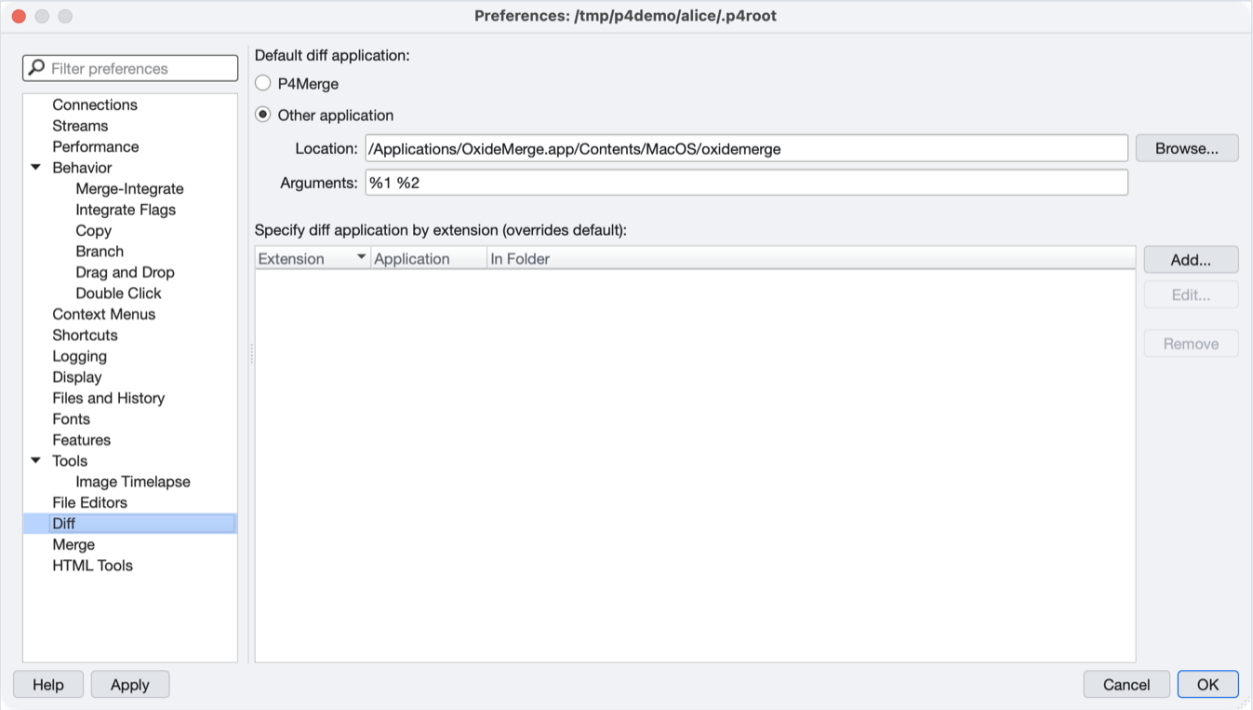


Figure 25. P4V Preferences → Diff

6. Select **Merge** in the list on the left, select **Other application** there as well, and enter the same **Location**.
7. In **Arguments**, type:

```
%2 %b %1 -o %r --dl Yours --dm Base --dr Theirs --wl --wr
```

In P4V's merge arguments, `%1` is theirs, `%2` is yours, `%b` is the base and `%r` is the result.

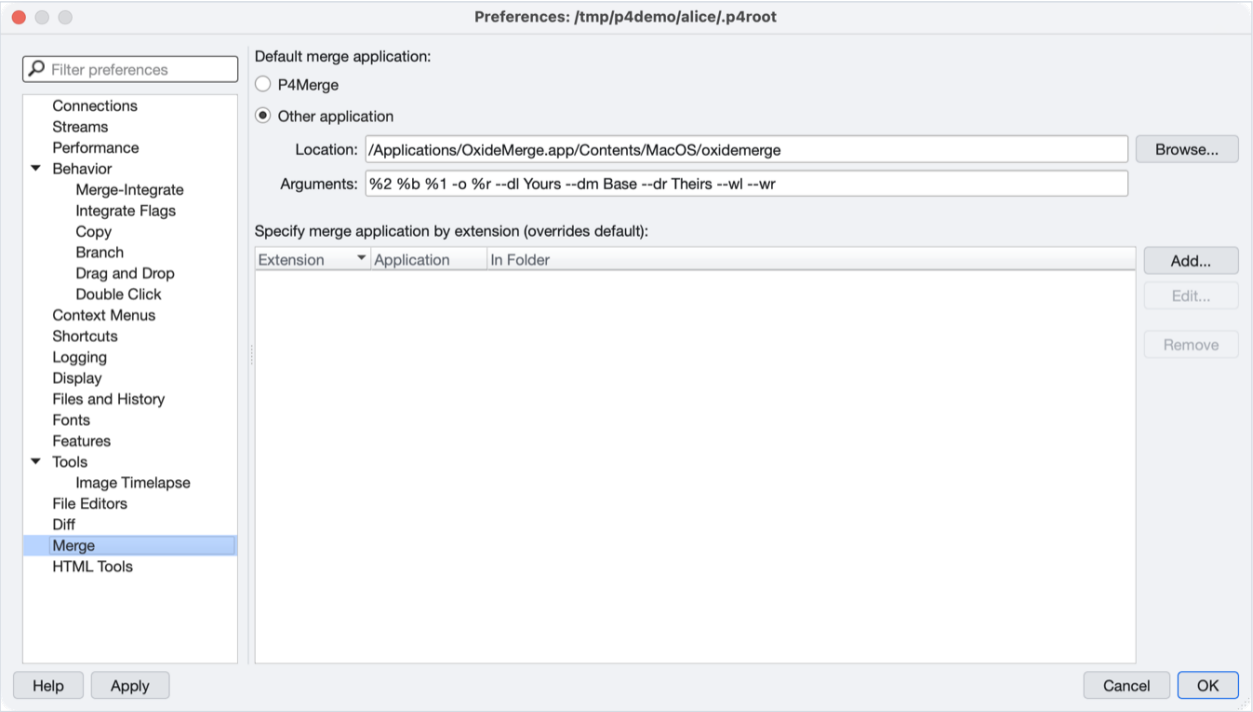


Figure 26. P4V Preferences → Merge

8. Click **OK**.

The list under **Specify diff application by extension** and **Specify merge application by extension** overrides the default for the extensions that it contains. Leave it empty to use OxideMerge for every file. If one kind of file still opens P4Merge, look for its extension there.

To compare a file with the depot:

1. Select the file in the **Workspace** or **Depot** tab and click **Diff** in the toolbar.
2. In the Diff Files dialog, the first path is the latest revision and the second is your workspace file; click **Diff**. OxideMerge opens with the depot revision on the left, in a temporary file with a name such as `server#head.conf`, and your workspace file on the right.

To merge a conflict:

1. Select the file in the **Workspace** or **Pending** tab; a file that needs resolving shows a question mark.
2. Choose **Actions** → **Resolving** → **Resolve...**, or right-click the file and choose **Resolve...**.
3. In the Resolve dialog, click **Run Merge Tool**. OxideMerge opens with **Yours** on the left, **Base** in the middle and **Theirs** on the right. If P4Merge opens instead, go back to **Preferences** → **Merge** and check that **Other application** is selected.

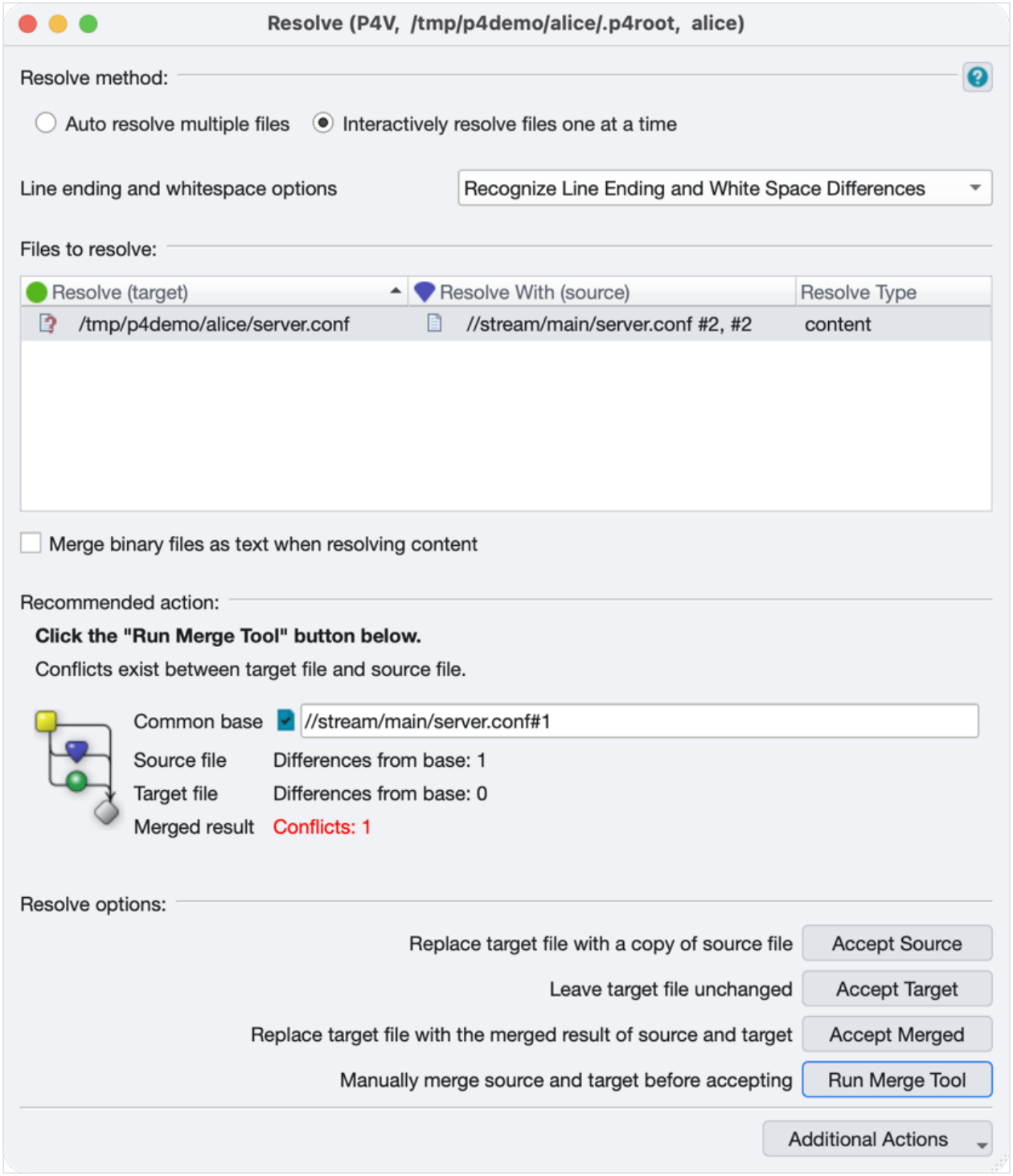


Figure 27. The P4V Resolve dialog

4. Resolve each conflict, click **Save result** and close the window. The result file that P4V passes already contains P4V's own conflict markers; saving replaces them. **Yours** is your workspace file itself rather than a copy, so make your changes in the middle pane: anything that you save in the left pane is written to your workspace file immediately.
5. In the Resolve dialog, click **Accept Merged**, and then click **Yes** when P4V asks whether to replace the file with the merged result. The dialog still shows the conflict count from before the merge; P4V does not read the result again until you accept it. Check the result first: OxideMerge writes the result only when you save, so if you closed it without saving, the result still contains P4V's markers. To take theirs or yours whole, use **Accept Source** or **Accept Target**

instead.



Checked on Windows with SlikSVN 1.14.2, Mercurial 7.1.2 and p4/p4d 2026.1, and on Linux with Git 2.55.0, Subversion 1.14.5, Mercurial 7.2.4 and p4/p4d 2026.1 (3089746), using local repositories and a local Perforce server. **Compare with revision** covers Git, Subversion and Mercurial, including a Subversion file whose name is not ASCII (on Linux that path is passed to `svn` directly) and, on Windows, a Mercurial name the ANSI code page can spell. Merges from `git mergetool`, `svn update --accept launch`, `hg merge` and `p4 resolve` were tried with the settings above, both saved (the conflict is resolved) and closed without saving (it stays unresolved). P4V keeps the argument string above. On Linux this was checked with P4V 2026.1 (2933292): diff opens the depot revision on the left and the workspace file on the right, saving the merge and then **Accept Merged** resolves the file, and closing without saving leaves P4V's markers so the file stays unresolved. The same diff setting, and accepting a merge after saving, were tried earlier on macOS with P4V 2026.3 and were not repeated on Windows. A Subversion or Perforce server reached over the network can still ask for a sign-in that OxideMerge cannot give. Sign in with `svn` or `p4` first. Subversion and Mercurial tests also run on Linux in the Docker test environment.

14.3. File manager

Adding OxideMerge to your file manager lets you start a comparison directly from the files and folders that you are browsing. You can add the entries from the **Tools** menu or with `oxidemerge --register-shell`:

Windows

Tools → **Add to Explorer context menu** adds an **OxideMerge** submenu for files, folders, drives and folder backgrounds. On Windows 11, the submenu is under **Show more options**.

macOS

Tools → **Add Finder Quick Actions** adds **OxideMerge: Compare**, **OxideMerge: Select Left** and **OxideMerge: Select Middle** to the Quick Actions and Services menus of Finder.

Linux

Tools → **Add to file manager context menu** adds the same entries to Nautilus, Nemo and Dolphin.

The entries work as follows:

- **Compare** compares the two or three selected files or folders. With one item selected, it compares that item with the side that you selected before. If you have not selected a side, it opens the **Select files or folders** dialog with the item.
- **Select Left** and **Select Middle** remember an item, for example one from another folder.

Compare then reads **Compare to "name"**.

- **Compare as** (Windows, files only) compares the files as text, binary, image or web page.

The entries point to the program that added them, so if you move the program, add the entries again. To remove them, use the matching **Tools** command or run `oxidemerge --unregister-shell`.

Chapter 15. Options and configuration

OxideMerge lets you adapt its appearance, comparison behavior and file handling to the way you work. This chapter describes the available options, where OxideMerge stores them, and how to run the program with a portable or separate configuration, for example for scripts and tests. It also explains how to add a language of your own.

15.1. Application options

To open the options, choose **Edit** → **Options…**, or click the settings button (two sliders) at the top right. Use **Search settings** to find an option by name, and **Reset to defaults** to restore the default values.

The options are grouped as follows.

General

- **Language**, **Theme** (**Light**, **Dark** or **Match system**), **Font size** and **Tab size**.
- **Line numbers**, **Location pane**, **View whitespace** and **Syntax highlighting**.
- **Highlight files up to**: the size in MB above which a file is shown without syntax highlighting (16 MB by default; see Troubleshooting, *Performance*).
- **Layout** (see the view options in Comparing text files).
- **Keep a .bak copy when saving over a file**, **Use the recycle bin when deleting** and **Confirm copy operations**.
- **Reopen previous comparisons at startup**.
- **Accessibility**:
  - **Color-blind friendly diff colors**: shows added lines in blue, removed lines in orange, changed lines in purple and conflicts in yellow.
  - **High contrast**: uses stronger colors, and gives the text on every line color a contrast ratio of at least 4.5:1, as WCAG requires for text. In every look, the text cursor has a contrast of at least 3:1 on every line color.
  - **Reduce motion**: turns off animations.
  - **Compact density**: uses lower rows and less space.

    You can also switch these four options from the command palette.

    Screen readers read out what changes without moving the keyboard: the difference counter (*Difference 3 of 12*), the merge status of a three-way comparison, the parts of the status bar and each message as it appears; error messages interrupt what is being read. Check boxes report whether they are on.

Compare

The comparison options (see Comparison options).

Compare rules

The compare rules files (see Compare rules).

Folder

Compare method, **Include subfolders**, **Include contents of unique subfolders**, **Skip hidden files and folders**, **Treat JSON, XML, TOML and YAML files with the same structure as identical**, **Date tolerance (seconds)** and **Worker threads (0 = automatic)**.

The structure option works with the **Full contents** method. When the text of a file differs, OxideMerge reads the file as structured data and counts it as identical when the values are the same. As in Semantic comparison, key order, layout and comments do not count. JSON, TOML and YAML numbers compare by value: `30` and `30.0` are the same, even where a program that reads the file tells them apart. When you open such a file, the comparison still shows its text differences; choose **Semantic** to see that the structure is the same. Folder synchronization still copies a file whose contents differ.

Worker threads is the number of files a folder comparison compares at once; **0** uses one thread per processor thread, up to eight. Large text comparisons share their work among up to eight threads whatever this setting is; comparisons running at once share those threads.

Image / Binary

Color distance threshold, **Block size** and **Bytes per line (binary)**.

Plugins

The unpacker plugins (see Unpacker plugins).

15.2. Configuration files

OxideMerge stores its options as JSON in your configuration folder:

System	Settings file
Windows	`%APPDATA%\OxideMerge\config.json`
macOS	`~/Library/Application Support/OxideMerge/config.json`
Linux	`~/.config/OxideMerge/config.json`

Help & Shortcuts shows the location of your settings file.



If OxideMerge cannot read the settings file, it saves the file as `config.json.bad` and uses the default settings. You can then correct the file and put it back.

The configuration folder also contains these folders:

- `Rules`: compare rules files.
- `Filters`: WinMerge `.flt` filter files.
- `Languages`: added or changed language packs.

15.3. Portable and separate configurations

By default, OxideMerge uses the configuration folder of your user account. You can instead keep the configuration with the program, or point OxideMerge to a folder of your choice.

Portable use

Create an empty file named `oxidemerge.json` next to the program. OxideMerge then stores its options in this file, and the `Rules` folder is next to the program. The program folder must be writable.

Separate configuration

Set the environment variable `OXIDEMERGE_CONFIG_DIR` to a folder. OxideMerge then uses the `config.json` in that folder. This setting has priority over portable use and over the default configuration folder. Use it for scripts and tests, or to keep separate work environments.

15.4. Adding a language

You can add a translation of the user interface by providing a language file.

To add a language:

1. Copy `lang/en.json` from the source code to a file named `<code>.json`, such as `nl.json`.
2. Translate the values. Keep each `{placeholder}` exactly as it is.
3. Put the file in a `Languages` folder next to the program, or in the configuration folder.



Messages that are missing, and translations with missing or extra placeholders, are shown in English.

Chapter 16. Keyboard shortcuts

Keyboard shortcuts let you move through differences, merge changes and edit text without reaching for the mouse. The tables in this chapter list the shortcuts by task. For the complete list in the program, including mouse actions, open **Help → Help & Shortcuts** (`F1`).



On macOS, use `⌘` for `Ctrl` and `⌥` for `Alt`.

Everything a comparison needs works without a mouse: `Tab` moves through the controls, and `Enter` or `Space` presses the one that has the focus. You can type the paths on the start page, start the comparison, go to a difference, copy it, save, close the tab and quit with the keyboard alone. A dialog takes the focus when it opens. When it closes, with `Esc` or one of its buttons, the focus goes back to the control that opened it.

Table 7. General

Keys	Action
`F1`, `?`	Show or hide Help & Shortcuts
`Ctrl` + `K`, `Ctrl` + `Shift` + `P`	Command palette
`Ctrl` + `N`	New text comparison
`Ctrl` + `O`	Open
`Ctrl` + `S`	Save
`Ctrl` + `Shift` + `S`	Save all
`Ctrl` + `Shift` + `E`	HTML report
`Ctrl` + `P`	Print: open the PDF report in the system's viewer
`F5`	Refresh
`Ctrl` + `W`, `Ctrl` + `F4`, `Esc`	Close the tab. `Esc` first closes an open dialog, search bar or editor
`Ctrl` + `Tab`, `Ctrl` + `Shift` + `Tab`	Next or previous tab
`Alt` + `F4` (`Cmd` + `Q` on macOS)	Exit. OxideMerge first asks about unsaved changes
`Ctrl` + `+`, `Ctrl` + `-`, `Ctrl` + `0`	Zoom in, zoom out, reset the zoom

Table 8. Differences

Keys	Action
`Alt` + `↓`, `Alt` + `↑`	Next or previous difference
`Alt` + `Home`, `Alt` + `End`	First or last difference

Keys	Action
Alt + PgDn, Alt + PgUp	Next or previous conflict
Alt + →, Alt + ←	Copy the current difference or the selected lines to the right or left. In a folder comparison, copy the selected items
Ctrl + Alt + →, Ctrl + Alt + ←	Copy all differences to the right or left
Alt + S	Set or remove a synchronization point

Table 9. Text editing and navigation

Keys	Action
Ctrl + F, F3, Shift + F3, Esc	Find, find next, find previous, close the find bar
Ctrl + H (Cmd + Alt + F on macOS)	Find and replace
Ctrl + G	Go to a line (to an offset in a binary comparison)
Ctrl + Home, Ctrl + End	First or last line
←, →	Make the left or right pane active
F2, Enter	When there is no cursor, place it at the end of the current line
Ctrl + A	Select the whole file
Shift + ←, Shift + →, Shift + ↑, Shift + ↓, Shift + PgUp, Shift + PgDn	Select text
Ctrl + C, Ctrl + X	Copy or cut the selection or columns, or the whole line
Ctrl + V	Paste at the cursor, or over the selection or columns
Delete, Backspace	Delete a character, the selection or the columns
Ctrl + ←, Ctrl + → (Option + ←, Option + → on macOS)	Move by word; with Shift, select by word
Ctrl + Backspace, Ctrl + Delete (Option + Backspace, Option + Delete on macOS)	Delete the word before or after the cursor
Alt+drag (Option+drag on macOS)	Select columns
Ctrl +], Ctrl + [, Tab, Shift + Tab	Indent or unindent the selected lines
Ctrl + D	Duplicate the selected lines
Ctrl click, kbd:[Ctrl+Alt↑], Ctrl + Alt + ↓	Add a text cursor (see Several cursors)

Keys	Action
Alt + Shift + ↑, Alt + Shift + ↓	Move the selected lines up or down
Ctrl + Z, Ctrl + Y / Ctrl + Shift + Z	Undo, redo
Ctrl + F2, F4, Shift + F4	Set or remove a bookmark, next bookmark, previous bookmark
Alt + C	Collapse unchanged lines
Alt + Z	Word wrap

Table 10. Folder comparison

Keys	Action
Enter	Compare the selected item
Space	Quick preview of the selected file
Ctrl + A	Select all items
→, ←	Expand or collapse a folder in tree mode
Ctrl + F	Find by name
Alt + P	Show or hide the file comparison below the folder list

Chapter 17. Troubleshooting

This chapter helps you solve the problems that you are most likely to meet when you install, start and use OxideMerge. Each entry names the symptom, explains the cause where it is known, and describes the solution.

When an error occurs, first read the error message and note the path that it names. Before you repeat a merge or a save, check the destination, the free disk space and the permissions, and check whether another program changed the file.

17.1. Installation and startup

macOS refuses to open the app

Solution: Try to open the app once, then choose **Open Anyway** in **System Settings** → **Privacy & Security**. Alternatively, remove the download flag in Terminal: `xattr -dr com.apple.quarantine /Applications/OxideMerge.app`.

macOS asks for access to a folder

Cause: macOS asks once before OxideMerge reads Desktop, Documents, Downloads, removable volumes or network volumes.

Solution: Allow the access, or turn it on later in **System Settings** → **Privacy & Security** → **Files and Folders**.

The AppImage does not start

Solution: Make the AppImage executable with `chmod +x`. If the message mentions FUSE, start it with `--appimage-extract-and-run`.

The window stays empty or does not open on Linux

Cause: OxideMerge draws with OpenGL, which needs a working graphics driver. A virtual machine, for example, may not have one.

Solution: Start OxideMerge with `LIBGL_ALWAYS_SOFTWARE=1 oxidemerge`. This uses the Mesa software renderer.

File dialogs do not open on Linux

Cause: File dialogs use the XDG desktop portal.

Solution: Install `xdg-desktop-portal` and the portal for your desktop, such as `xdg-desktop-portal-gtk` or `xdg-desktop-portal-kde`.

The file manager entries are missing

Solution: On Windows 11, look under **Show more options**.

The entries point to the program that added them. If you moved or updated the program to

another folder, add them again from the new program.

On Linux, restart the file manager (for example, `nautilus -q`) to see new entries.

The settings are not saved

When OxideMerge cannot save the settings, it tells you why.

Solution: Check that the configuration folder (see Configuration files) is writable and that the disk is not full.

17.2. Editing and saving

A pane cannot be edited

Cause: The pane is read-only for one of these reasons:

- The lock in its header is closed.
- It was opened with `--wl`, `--wm` or `--wr`.
- It was extracted from a review snapshot or from a web page tree. A file extracted from an archive comparison (see Archives) can be edited; saving it also saves the archive, unless the archive is one that stays read-only, such as a signed package.
- It is a JSON or XML file shown formatted. To edit and merge it, turn off **Format JSON and XML before comparing**.

A file cannot be saved because of its encoding

Cause: The encoding cannot represent some characters in the edited text.

Solution: Undo the change that added these characters, and save again.



If you choose another encoding, OxideMerge reads the file again. Copy your edits to another place first.

A save or copy fails, but the file is writable

Cause: OxideMerge must also be able to create and replace a temporary file in the same folder (see Saving files).

Solution: Check the folder permissions and the free space, and look for programs that lock the file. OxideMerge reports the failure rather than writing over the original.



Keep the comparison open until you have saved your edits successfully.

17.3. Comparison results

Files look identical, but their bytes are different

Cause: Depending on its options and the compare rules, a text comparison can ignore some differences on purpose.

Solution: Check the options for whitespace, case, line endings, encoding and comments, and check the compare rules. To check that files are exactly the same, use a binary comparison, or the **Binary contents** folder method.

Files are missing from a folder comparison

Solution: Check the filter, **Skip hidden files and folders**, **Include subfolders** and the **Show** settings. **Find by name** also hides rows.

All differences uses the filter and the display settings of the folder comparison, and does not include hidden results.

A file is shown as an error

Cause: OxideMerge could not read the file. The message names the file.

Solution: Check the permissions of the file. On macOS, also check the folder access described above.

Automatic folder merge does not start, or merges nothing

Solution: Turn on **Include subfolders** and **Include contents of unique subfolders**, and let the comparison finish.

If a file changed after the comparison, nothing is merged and the file is listed. If a change fails during the merge, all changes made so far are undone. Refresh the comparison and review the merge plan again.

17.4. Web pages

A web comparison is missing content that a browser shows

Cause: OxideMerge compares the HTML from the server, without the content that scripts add. It does not open pages in a browser, so the scripts of the page never run. Content inside frames is not included.

The status bar counts the resources that could not be downloaded, and the resources that were skipped because they are on this computer or on a private network.

17.5. Performance

Large files are slow or use a lot of memory

Cause: A text comparison keeps the decoded text, the comparison data and the edit history in

memory. The memory use depends on the content and on what you do. Binary and image views also load the complete files.

- Valid UTF-8 files of 16 MiB or more load in parts. UTF-16 files and legacy encodings are decoded completely. The text stays editable, and the compare rules work as usual.
- Text comparisons whose files add up to 32 MB or more open in the background: an **Opening** window at the bottom right names the file and counts the seconds, and the rest of the window stays usable. **Cancel** in it stops the comparison, usually within a fraction of a second, and the tab is not opened.
- Syntax highlighting is turned off for a file larger than **Highlight files up to** in **Options → General** (16 MB by default), or with more than 3,125 lines for each of those MB (50,000 lines at 16 MB), and a message says which of the two limits the file is past. Raise the limit to highlight larger files; highlighting takes time and memory for every line.
- Editing, undo, word wrap and unusual compare rules can still use more memory.

Solution: To use less memory, use a binary comparison, or compare folders by size and date instead of by content (see Comparison methods).

Chapter 18. Known limitations

This chapter lists what OxideMerge does not support, or supports only in part. Review it before you plan a workflow that depends on one of these areas.

- Archives: OxideMerge supports ZIP-based formats, TAR (plain, gzip, bzip2, xz and Zstandard) and 7z, including password-protected ZIP and 7z archives. RAR and archives split into parts (`.zip.001`, `.7z.001`) are not supported.
- The text panes are meant for reviewing and merging changes, not as a full editor. The inline view does not support editing.
- Unpacker plugins use external commands. WinMerge plugin DLLs and scripts are not supported.
- Web page comparison does not support cookies or logins. A page that requires a login shows its login page. Page scripts do not run, so content that scripts add is missing. Content inside frames is not compared.
- Semantic comparison copies one value at a time, and only when it can confirm the edit by reading the file again. It does not move array elements or merge all changes at once.
- Folder synchronization works on local folders only, and runs only when you start it. A renamed file appears as a deletion and a copy.
- SFTP and FTP: FTP active mode does not go through a proxy, and rarely works from behind NAT. SSH host certificates signed with RSA SHA-1 or ECDSA P-521 are refused. `Match exec` and `Match canonical` in `~/.ssh/config` never match. A prompt on the terminal can be stopped only with `Ctrl+C`.
- The translations were made with the help of machines and have not been reviewed by native speakers.